

Project Title: Blackjack Game & Learner

Name: Alex de Roeck

Centre Number: 56120

CONTENTS -----

CONTENTS -----	1
ANALYSIS -----	8
The Project Idea:	8
Stakeholders:	8
Research into existing, similar games:	9
Example 1 – “Blackjack” by MSM Games:	10
Example 2 – “Blackjack” by FreeGames:	11
Example 3 – “Blackjack” by The Washington Post Games:	12
Survey Questions, Results and Analysis.....	13
Questions:	13
Survey Results:.....	14
Requirements for my game	17
Identifying Limitations	20
Hardware and Software Requirements	21
Hardware Requirements:	21
Software Requirements:	21
Computational Methods.....	22
Decomposition	22
Abstraction	22
DESIGN -----	23
Development Plan Stages:	23
Stage One	23
Stage Two	23
Stage Three	24
Stage Four.....	24
Stage Five	24

Structure Diagram (for Development Plan Stages):	25
Parts of the Game:	25
GUI Design:	26
The Main Game:	26
Cards:	26
Chips:	27
Buttons:	27
Table:	27
The Menu:	28
Close Menu:	28
'MENU' Title:	28
Menu Options:	29
Extra Details:	29
ITERATIVE DEVELOPMENT -----	30
Development Issue	30
The Issue:	30
Stage 1 – Deck of Cards and Card classes:	30
Stage 2 – Player and Hand classes:	30
Stage 3 – Dealer class:	31
Stage 4 – Betting chips class:	31
Stage 5 – Main Game:	31
Stage 6 – GUI (buttons, boxes, and text output classes):	31
Deck of Cards & Card Classes (Stage 1)	32
Design -----	32
Flow Chart:	32
Data Dictionary:	33
Development -----	34
Developing the Classes:	34
Coding the Class – 'DeckOfCards':	34
Changes:	37
Updated Flow Chart:	37
Re-Coding the Class – 'DeckOfCards':	38

Extra Code:.....	42
Coding the Class – ‘Card’:	43
Testing -----	45
Test 1a – Testing the shuffleDeck() method:.....	45
Test 1b & c – Testing the getCardValue() & dealCard() methods:	47
Player & Hand Classes (Stage 2)	48
Design -----	48
Flowchart:	48
Data Dictionary:.....	50
Development -----	51
Developing the Classes:	51
Coding the Class – ‘Hand’:	51
Extra Changes:.....	53
Re-Coding the Class – ‘Hand’:	53
Coding the Class – ‘Player’:	60
Defining Variables:	60
Extra Code:.....	64
Testing -----	65
Test 2a & b – Testing the getPlayerTurn() & setPlayerTurn() methods:	65
Test 2c – Testing the getHandTotal() method:	66
Dealer Class (Stage 3)	67
Design -----	67
Flowchart:	67
Data Dictionary:.....	68
Development -----	69
Coding the Class – ‘Dealer’:	69
Extra Code:.....	78
Testing -----	79
Test 3a & b – Testing the getHandTotal() & startDealer() methods:	79
Test 3c – Testing the getDealerTurn() method:.....	80
Betting Chips (Stage 4)	81
Design -----	81

Flowchart:	81
Data Dictionary:	83
Development	84
Coding the Parent Class – ‘PokerChips’:	84
Coding the Subclasses:	85
Full Classes’ Code	87
Bugs:	93
Testing	95
Extra Information:	95
Test 4a – Testing PokerChip1’s pressed() method:	95
Test 4b – Testing PokerChip25’s pressed() method:	96
Test 4c – Testing PokerChip200’s pressed() method:	97
Main Game Class (Stage 5)	98
Design	98
Flowchart:	98
Data Dictionary:	99
Development	100
Coding the Class – ‘MainGame’:	100
Testing	104
Test 5a – Testing the MainGame getters methods:	104
Extra Information:	105
Making the GUI (Stage 6)	106
Design	106
Main Game Background Image:	106
Playing Card Images:	107
Button Images:	109
Money Box Image:	109
Poker Chip Images:	110
Development	111
Background Image Code:	111
Playing Card Images Code:	112
Buttons Code:	114

TotalValueCounter Class:	123
InformationOutputter Class:	126
Updating the MainGame Class:	128
Methods:	130
Testing -----	131
Test 6a – Testing the Card’s updated act() method:	131
Test 6b – Testing the Hit Button’s Functionality:	132
Test 6c – Testing the Stand Button’s Functionality:	133
Test 6d – Testing the Double Button’s Functionality:	134
Test 6e – Testing the TotalValueCounter Functionality:	135
Test 6e – Testing the InformationOutputter Functionality:	136
END STAGE TESTING -----	138
Robustness Testing:	138
Test 1:	138
Test 2:	139
Beta Testing & Survey:	140
EVALUATION -----	147
Success Criteria 1 – Menu / Options:	147
Criterion:	147
Fulfillment:	147
Comments:	147
Development:	147
Improvements:	147
Success Criteria 2 – Help Menu:	148
Criterion:	148
Fulfillment:	148
Comments:	148
Development:	148
Improvements:	148
Success Criteria 3 – Stacked Poker Chips:	149
Criterion:	149
Fulfillment:	149

Comments:	149
Development:	149
Improvements:	149
Success Criteria 4 – Toggleable Cheat Sheet:	150
Criterion:	150
Fulfillment:	150
Comments:	150
Development:	150
Improvements:	150
Success Criteria 5 – Moves The Player can make:	151
Criterion:	151
Fulfillment:	151
Comments:	151
Development:	151
Improvements:	151
Success Criteria 6 – The GUI:	152
Criterion:	152
Fulfillment:	152
Comments:	152
Development:	152
Improvements:	152
Success Criteria 7 – Game Rules on the Table:	153
Criterion:	153
Fulfillment:	153
Comments:	153
Development:	153
Improvements:	153
Evaluation – Extra Notes	154
How I instantiated the deck:	154
Dealer Act Method:	154
Player Bust Issue:	154
Money Box Issue:	154

Time Delays:	155
Ace's Values:	155
Insurance:	155
Dealer Blackjack Issue:	155
InformationOutputter Issue:	156
Poker Chip Mini Menu:	156
Player Move Buttons Issue:	156
Maintenance:	157
Use of Functions:	157
Naming Conventions:	157
Code Indentation:	157
Comments and Logging:	158
Limitations:	159
Time Delays:	159
Insurance:	159
Splitting:	159
Menu & Help Menu:	159
Perfect Strategy Cheat Sheet & Card Count Counter:	159
Extra Limitations:	159
Conclusion:	160
Extra References:	161

ANALYSIS -----

The Project Idea:

For my project I came up with the idea to create a Blackjack game / learner, where you play against a computer operated dealer. This game would be used for multiple purposes; It could be used as a game for entertainment/testing your skill, learning how to play, and practicing certain methods such as perfect strategy and card counting.

The game is also free to play and does not use real money to gamble. This means people can play Blackjack without the risk of losing any real money and therefore does not encourage gambling. Additionally, because it is not gambling real money, it is legal for under 18-year-olds to play it.

I got this idea as I often play, and enjoy playing, Blackjack with my friends and family, where we do not gamble with real money. I currently do not know how to card count or play perfect strategy off by heart, which gave me the idea of implementing these strategies into the game as this would help me (and others) practice how to.

For my project, I will research similar, existing games of Blackjack and their different rules to figure out what I will, and will not, implement into my game.

Stakeholders:

The stakeholders for my game are people that are teenagers or adults, however it can be played by people of any age, who already know how to play Blackjack. This is because it will not teach them the basic rules of Blackjack.

Out of the age range of my target audience, I feel that young and middle-aged adults will be the main users of my game; This is because people aged 36-50 statistically play Blackjack the most (35.1%), followed by people aged 21-25 (32%). *[I got these statistics from [‘The Relationship of Demographics to Gaming Preferences and Behavior’](#) Written by Michelle Millar and Seyhmus Baloglu in 2008].* Because of the age of my target audience, the design of game will not be overly simple (because it is not aimed for children) and will be reasonably detailed as older audiences can handle this.

This game is not aimed at any specific gender or race and can be played by any; However, this game is aimed at English speakers as it will be in English.

I believe my stakeholders will use my game for entertainment, practicing, or learning.

Research into existing, similar games:

I have researched existing, similar games to assist in my plan for making my game. I have then noted what features I would, and would not, like to implement in my game from them.

By playtesting these games, I found out some rules (and features) for Blackjack I previously was not aware of which I need to include in my game:

- **Push:** When the player and the dealer stand on the same amount, it is called a push, and the player gets their money back.
- **Dealer hits on soft 17:** Unless the dealer has a hard 17 (the first two cards dealt total to 17), the dealer can keep hitting on a soft 17 (not a hard 17).
- **Insurance:** A side bet offered to players when the dealer's upcard (face up of their two cards) is an Ace, on whether they think the dealer will hit 21. I will make mine pay 2:1 (which is normally what casinos will payout)

Here are the games I researched (see next page):

Example 1 – “Blackjack” by MSM Games:

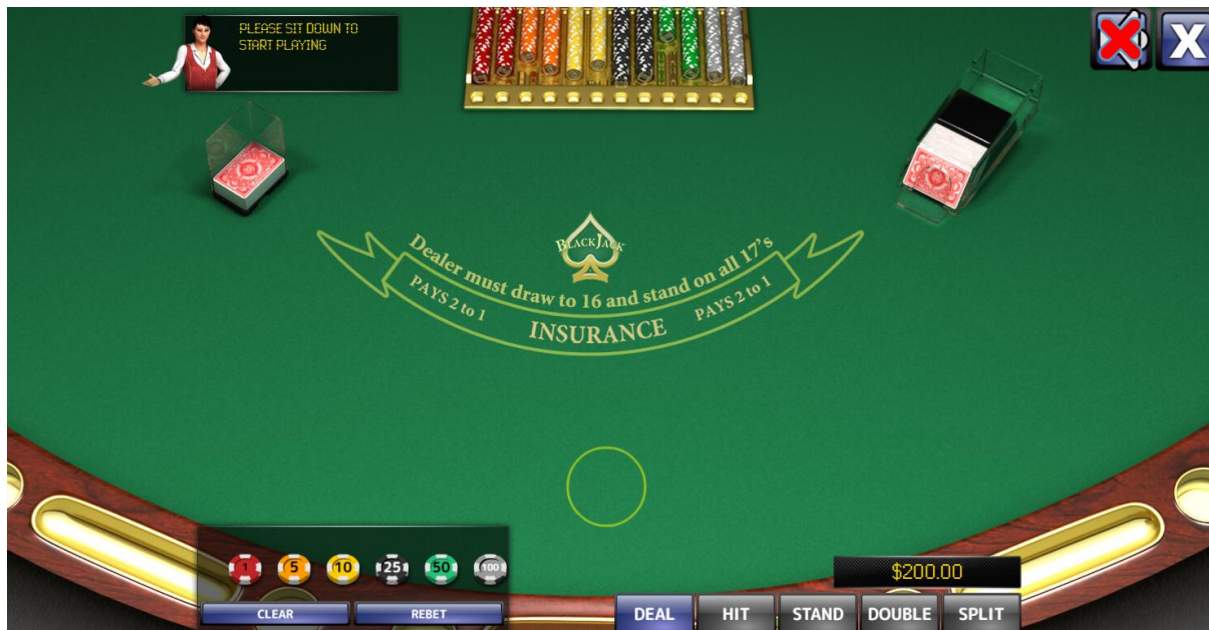


Figure 1 - 'Blackjack' by MSM Games - [Play Blackjack in your browser | Games from MSN](#)

This version of Blackjack feels the most realistic and cleanest with its design and layout; This makes it easy to see what is going on. However, this game does not have the option for multiplayer or a Perfect Strategy cheat sheet, which are things I will attempt to include in mine.

Out of the three existing similar games, the design and layout of this version is my favourite as:

- The buttons for what move you can do and what chips you can bet are constantly on screen (while non obstructive).
- Clear instructions in the top left corner from the “dealer” which also show who won the round.
- Clear numbers next to the chips to show how much you have bet.
- Clear numbers next to the cards to show the total you have.
- Clear counter/scoreboard on how much money the player has.
- Visually appealing with animations and seeing what you have bet in chips stacked up.
- The colour of the table changes (I could implement customisation).
- Game rules on the table.

I am going to base my design and layout similarly to this and implement some of these features.

Example 2 – “Blackjack” by FreeGames:



Figure 2 - 'Blackjack' by FreeGames - [Play Blackjack](#) | 100% Free Online Game | [FreeGames.org](#)

The design of this version feels too childlike which is not my target audience. The reason why it feels like this is because of the round colourful buttons. The layout of the game also feels too squished rather than spaced out. These are features I will not include in mine as it will be aimed for teenage and adult audiences. This version also has quite simple designs and sound effects.

Features of the game I would like to implement in my game:

- Simplistic design.
- Sound effects.

This game also has long pauses between rounds. This is something I will try to avoid when creating my game as it can ruin the flow of the gameplay. Furthermore, you are only able to bet 20, 100, 500 or 1000 in this game, whereas I want the players of my game to be able to bet any amount (like 'Figure 1') as it is more realistic.

Example 3 – “Blackjack” by The Washington Post Games:



Figure 3 - 'Blackjack' by The Washington Post Games - [BlackJack | Instantly Play BlackJack Online for Free!](#)

This version of Blackjack feels the most simplistic layout wise as there are not many things on screen; However, it is clear on what the player can choose from. This game also does not have the option of multiplayer or a Perfect Strategy cheat sheet. I would like these options in my game therefore I will not be using this feature of this game.

This version also has the option to “Cash out and leave” which results to a win/loss screen (see above) which displays different stats; This is different to the other games which only allow you to exit the game. For my game I will allow the player to exit the game, but it will not display stats.

Features of the game I would like to implement in my game:

- Very clear on who won the round.
- It is clear when the player goes bust.
- The ability to exit the game.
- 6 card decks in the Shoe.
- Help menu explaining the rules of Blackjack (If I have time to).

Survey Questions, Results and Analysis

To find out what I should prioritise when creating my game, I created a survey and sent it out to my stakeholders. Here are the questions I asked and why:

Questions:

Q1: *“Have you played Blackjack before? (Also known as “21”)”* - I asked this question to find out how many of my stakeholders have already played Blackjack before (and would likely know the rules).

Q2: *“Did you enjoy it?”* – I have asked this question to find out how many of my stakeholders enjoyed playing the game.

Q3: *“Which would you prefer: A more detailed game, or a more simplistic designed game?”* – I asked this question to find out whether I should make my game more detailed (not super realistic but a more polished feel) or more simplistic.

Q4: *“What is more important to you: Animations or sound effects?”* – I asked this question to find out whether I should focus on the animations or sound effects for my game.

Q5: *“Please rank these uses of the game, of which you would be most likely to use it for: Playing, Practicing, Learning.”* – I asked this question to find out what my stakeholders would use my game for the most.

Q6: *“Please rank these features of the game in importance:*

- *Being able to exit the game*
- *Help menu (for basic rules of Blackjack)*
- *Visual chip stack*
- *Toggleable Card Count counter*
- *Toggleable Perfect Strategy cheat sheet*
- *Having 6 cards in the shoe*
- *Instructions/messages from the (computer) dealer*
- *Option to ask the dealer for a hint”*

– I asked this question to find out what features my stakeholders think are the most important.

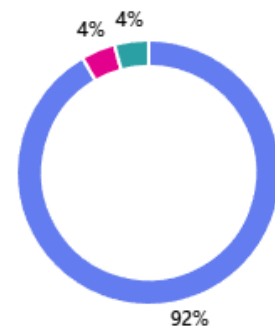
Q7: “*Finally, would you play this game?*” – I asked this question to find out whether my stakeholders would play my game after hearing what it is about and what will be included.

Survey Results:

Q1:

Have you played Blackjack before? (Also known as "21")

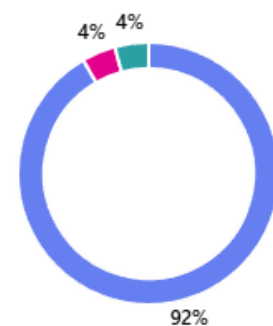
● Yes	22
● No	1
● I don't know what Blackjack (or "21") is.	1



Q2:

Did you enjoy it?

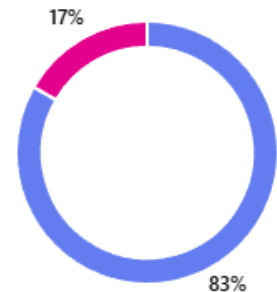
● Yes	22
● No	1
● I have not played it but would be open to trying it.	1
● I have not played it and am not open to trying it.	0



Q3:

Which would you prefer?

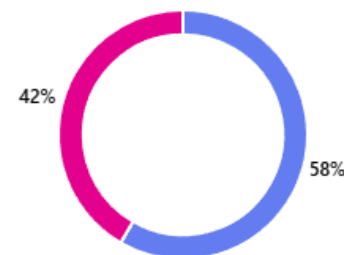
- A more detailed game. (Not super realistic but feels polished) 20
- A more simplistic design. (Not realistic at all boooooo) 4



Q4:

What is more important in this game to you?

- **Animations** (not overly complicated, just cards being dealt, chips being placed, etc..) 14
- **Sound effects** such as: Cards being dealt, Chips being placed, etc... (Possibly shuffle sounds?) 10



Q5:

Please rank these uses of the game, of which you would be most likely to use it for. (*Most likely at the top*)

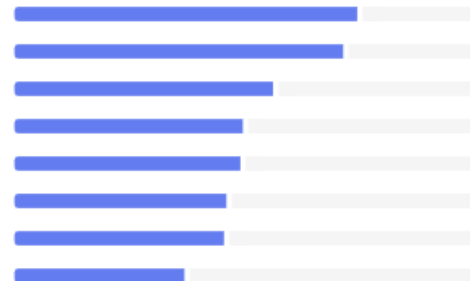
- 1 **Playing** for fun/general entertainment
- 2 **Practicing** your skills at Blackjack. (For legal reasons not to shred casinos 🚫)
- 3 **Learning** Blackjack strategies: Perfect Strategy and Card Counting



Q6:

Please rank these features of the game in importance. (*Most important at the top*)

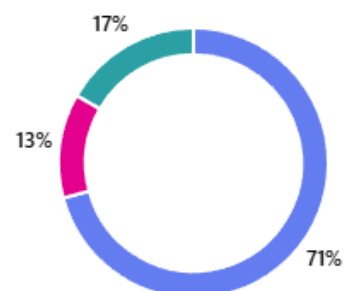
- 1 **Being able to exit the game.** *The rest of the game will be a loop.*
- 2 **Help menu explaining the BASIC rules of Blackjack.** *General rulebook, if you will, of what...*
- 3 **Being able to see all the chips you have bet stacked up in front of you.** *Visually appealing pile...*
- 4 **Toggleable Card Count counter (Maybe true count too?).** *A count based on how likely a card...*
- 5 **Toggleable cheat sheet for Perfect Strategy.** *Perfect Strategy is the best move to do given yours...*
- 6 **Instructions/messages from the (computer) dealer.** *E.g.: "Please place your bet", "Please select..."*
- 7 **Having 6 card decks in the shoe.** *Basically having several card decks shuffled in and in play (normally ...*
- 8 **Option to ask the dealer for a hint.** *Based on Perfect Strategy, the dealer will give you a hint on...*



Q7:

Finally, would you play this game? (*As in this version I will make*)

- Yes 17
- No 3
- Maybe 4



Requirements for my game

A list of the features I want to add to my game.

No.	Feature		Sub-features	Explanation	Justification	Importance
1.	Menu/options	a	Being able to exit the game.	A button that allows the user to exit the game.	Without this, the game would continue for an endless loop.	Important
		b	Toggleable Perfect Strategy cheat sheet.	A button that toggles an on-screen cheat sheet.	This would allow for a learning aspect to my game which is what I intended it to have.	Important
		c	Start button	A button presented to the player when they are not in a game.	Without this, the game would start automatically. By having a start button, it makes the game feel more polished.	Not important
		d	Title screen	When the player is not in a game, a title screen is presented to them.	This would make the game feel more polished and finished. A polished game is more likely to be played than an unpolished game.	Moderately important
		e	Toggleable Card Count counter	A button that toggles an on-screen Card Count counter.	This would further add to the learning aspect of my game.	Slightly important
2.	Help Menu			Navigational menu on the basic rules of Blackjack. May be part of the main menu or will be a separate menu.	Players who do not already know the rules will need this to play the game. Furthermore, people who know the rules may need reminding.	Important
3.	Stacked poker chips			Purely visual. See what you have bet, stacked in front of you.	This would be purely visual as there is already a counter on-screen for the total value you have bet.	Not important

No.	Feature		Sub-features	Explanation	Justification	Importance
4.	Toggleable cheat sheet			Toggleable perfect strategy cheat sheet.	This feature is important as it is what will make my game stand out from other existing similar games.	Important
5.	Moves the player can make	a	Stand	The player's turn ends with what upcards they currently have.	This feature is needed for the game to function correctly.	Essential
		b	Hit	The player gets dealt another card.	This feature is needed for the game to function correctly.	Essential
		c	Split	When the player's (2) upcards are equal, they have the option to split their hand into multiple hands.	This feature is needed for the game to function correctly.	Essential
		d	Double down	The player doubles their initial bet and commits to taking only one additional card. This move is only available before the player has 'hit'.	This feature is needed for the game to function correctly.	Essential
		e	Bust	The player's upcard total goes over 21, they are removed from the round and lose their poker chips.	This feature is needed for the game to function correctly.	Essential
		f	Insurance	Side bet where the player bets that the dealer will get 21.	This feature is not needed for the game to function correctly; however, it is a feature of the game Blackjack.	Not important

No.	Feature		Sub-features	Explanation	Justification	Importance
6.	GUI	a	Current bet total counter	The total amount that the player has currently bet will be displayed on screen.	This will help the player see more clearly how much they have bet.	Essential
		b	Current upcard total counter for the player	The total amount that the player's upcards add up to will be displayed on screen.	This will help the player see more clearly what their running total is.	Essential
7.	Game rules functionality & displayed on the table	a	Blackjack pays 3:2	When a player gets a Blackjack, they will be paid 3:1.	This rule is needed to ensure the amount the player wins is a fair amount.	Essential
		b	Dealer must stand on 17 and draw to 16	If the dealer's upcards add to 17, the dealer must stand. If the dealer does not, the dealer must draw until they have 16.	This rule is needed to make the game more balanced.	Essential
		c	Insurance pays 2:1	If the player wins the insurance side bet, they get paid 2:1.	This feature is needed for the game to function correctly (If I include insurance as a feature).	Important (if I include Insurance)

Identifying Limitations

Below are some limitations and extra features that I am not going to include within my game, explanations for each of them, and reasoning:

Limitations	Explanation
Multiple card decks in the shoe	Although this would make my game more realistic and increase the difficulty, this would be too time consuming for me to do in this project. Because of this, I will be using one card deck (as opposed to a desired 6). This will ensure that I finish my project in time and to a good standard.
Language	My game will be in English. I will not implement language options; This means non-English speakers will not have a translated game. The reason I have decided this is because I will not have enough time to add this to my game.
Customisation	For my game, I am not going to implement customisation. As much as I enjoy customisation in games, I believe my time will be better spent on other parts of the game. If I were to implement customisation, some examples of what I would add would be: - Card back design + colour - Table felt design + colour - Table edge design + colour Another idea would be having unlockable cosmetics that you can purchase.
Leaderboards and Personal scoreboard	Leaderboards comparing different players' stats/data. Personal scoreboard with stats such as: - Highest amount of money - Total Blackjacks - Saved customisation. I am not going to add these features to my game. Although it will improve it, I think it would be better to use my time to improve the main aspects of my game.
Save data	The ability to save player data. I am not going to add this feature as it is quite complex to implement. I would prefer to spend my time on the main parts of my game as opposed to this.

Multiplayer	For my game I originally wanted to implement multiplayer, however, I am not going to add this feature as it is quite complex and time-consuming.
-------------	--

Hardware and Software Requirements

I have chosen to code my game and GUI in Greenfoot. This is because I have used this software before and feel comfortable coding a game using it.

Hardware Requirements:

Any device with a touchscreen user interface or mouse inputs such as:

- Mobile phone
- Tablet/iPad
- Laptop
- Desktop PC

These devices won't need to be very powerful as my game is quite basic with what will need to be loaded.

The device may require speakers if in-game sound effects/music is to be played.

A keyboard (virtual/physical) will **not** be needed as the game does not require text input.

Software Requirements:

A device that uses Windows, macOS, or Linux, or a device that supports a recent Java Virtual Machine (JVM). This is because my game will be made using Greenfoot and has these requirements to play the game I will make.

Computational Methods

I believe that this problem is a computational problem. By having it as a game that is played digitally, it is more convenient and accessible to play. This is because you do not need a physical table to play on, a deck of cards, or poker chips, and allows for portability.

Decomposition

To make my game, I will use **decomposition** to break down the different sections of my game into more manageable, smaller sub-problems such as:

- The betting aspect
- The deck of cards
- The player's moves (I will then break this down into the individual moves themselves such as: Stand, Hit, Split, Double Down)
- Calculating the winner

Abstraction

I will also use **abstraction** for my game; The user will only be presented with necessary information on-screen needed to play the game such as:

- The player's cards and their total value.
- The dealer's cards and their total value.
- The player's bet total.
- The player's possible moves for the game, they won't be presented with options such as: "sit at the table", "get up from table".

Furthermore, by using abstraction, I will not have to show the deck of cards being shuffled, or the dealer itself.

DESIGN -----

Development Plan Stages:

Stage One

To figure out and make a functioning deck of cards, and card objects.

These cards will need to be able to be '**shuffled**', whether it be a random card from the 52 is selected, or if they are put into a random order and **dealt** in that order. Cards will need to be **dealt** as **face-up and face-down** cards. Cards will also need to be **discarded** at the end of a round.

Stage Two

Creating the player moves and hands.

The player will have a choice of moves in which they can do on a turn based on what cards they have. These moves will be:

Stand – Stick with the cards that have been dealt to you so far and end your turn.

Hit – Have another card dealt to you.

Split – If your starting hand is two cards of equal value, the player can split these and play them as separate hands.

Double Down – Put more money down, and be dealt one final card, your turn is then ended after this.

Bust – The player's total upcard values add up to over 21. Their turn ends and they lose the round.

Stage Three

Making a functioning GUI that the user can interact with.

A GUI that the player can see and interact with. This includes:

‘Current bet total counter’ displays the players total bet in that round.

‘Current upcard total counter’ displays the player’s total upcard value.

Face-up & Face-down Playing cards on the table.

Game rules on the table, such as “Blackjack pays 3:2”, “Dealer stands on 17 and draws to 16”, and “Insurance pays 2:1”.

Visually stacked poker chips that the player has bet (purely visual as opposed to the other GUI features).

Stage Four

Making a functioning betting system the user can use.

Bet-able poker chips that the player bets each round. Also, a money system.

These **chips will be clickable** by the player to bet them. There will be a **selection of different valued chips**: ‘£1, £5, £10, £25, £100, £200’. I have chosen these values as they are accurate to what valued chips you may have at a casino. The player won’t be able to bet more money than they have. When the player runs out of money the game ends and they lose.

Stage Five

Making a functioning menu the user can use.

A menu where the player can choose from several options:

Exit to title screen – The player can exit the game to the title screen; this is also where they start when the game itself starts.

Start button – Starts a game of Blackjack.

‘Toggle Perfect Strategy Cheat Sheet’ – Toggles an on-screen Perfect Strategy cheat sheet.

‘Toggle Card Count counter’ – Toggles an on-screen Card Count counter.

Help Menu – Shows the basic rules to Blackjack.

Structure Diagram (for Development Plan Stages):

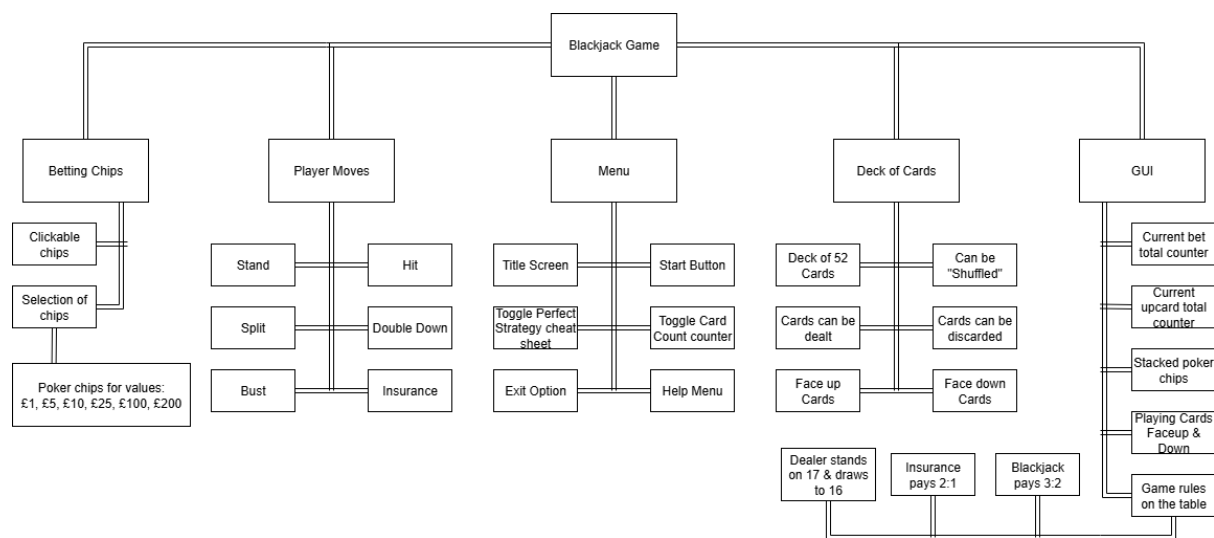


Figure 4 – ‘Project Structure Diagram 2.0’

I have created a structure diagram for my game so I can clearly see what parts to be made and what features each part has. By breaking my game into parts using decomposition, it will make it easier for me to work on my game in parts.

Parts of the Game:

Deck of Cards

Player Moves

GUI

Betting (Poker) Chips

Menu

Player Moves and **Deck of Cards** are the most important part of my game as they are needed to function, therefore I will be making them simultaneously first alongside my **GUI** (as the **GUI** is needed for the game to be used).

Betting Chips will be made next as betting is an important part of Blackjack (despite not being needed for the game to function).

Finally, I will make the **Menu** and title screen and add any extra features (depending on how much time I have left).

I have chosen this order as it is in order of importance for the game to function correctly. The stages I will be spending the most time on is ‘**Deck of cards**’, ‘**Player Moves**’, and ‘**GUI**’, this is because these stages are vital for the game to function. Whereas ‘**Menu**’ and ‘**Betting Chips**’ are not vital for the game’s functionality.

GUI Design:

The Main Game:

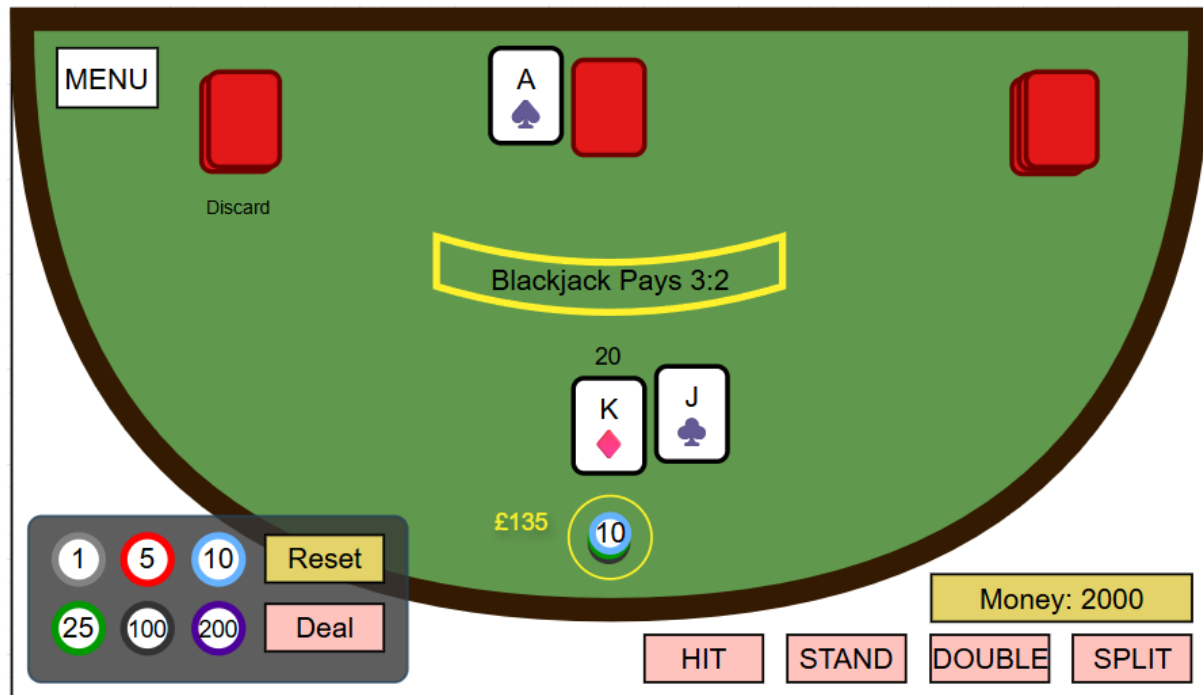


Figure 5 – ‘Blackjack Game Concept GUI’

Cards:

For the cards I have decided to position them where they would be on a real Blackjack table. This is to add to the realism and so that it is in a layout players may be used to.

I have also decided to have the face of the cards be a plain white so that the values are easy to see. This would also be good for people with poor eyesight, potentially due to age or certain conditions, and therefore makes my game more accessible.

I have coloured the card backs red so it is clear it is not the card face; however, I may change these or give them a more designed back when creating the assets.

I will also have a total card value counter next to the player's cards to assist in the game's ease of use (see above 'K ♦ card').

I have also made a discard pile (top left) and draw pile (top right) and put them towards the edges, so they are not obtrusive. I will have text by the discard pile, so people do not get confused as to why there are two piles of cards.

Chips:

I have made poker chips for values: 1, 5, 10, 25, 100, and 200, and coloured them accurately to poker chips in real life to add to the realism and immersion.

I have also added a total bet counter to further improve the games ease of use as it allows the player to see exactly how much they've bet.

Furthermore, I have added a total money counter so the player can see how much money they have overall. This money counter is a warm yellow, this is because it stands out clearly, without being overbearing, and yellow is often associated with money.

The poker chips, once bet, are 'placed' in a yellow circle at the bottom of the table, this is so the player can see what they've bet (making it more immersive). I decided on the colour yellow because it stands out against the green table. Once there are more than 5 (maybe 10 depending on how the final GUI looks), another pile of chips will start to the side, this is so they do not get in the way of the cards if the player was to bet a lot of money.

Buttons:

There are several buttons the player can press. These buttons have been coloured, so they stand out. The player's moves (pale pink) are a different colour to the 'reset' button (warm yellow) as it is to do with the poker chips rather than the player's moves.

There is a menu button that can take the player to the menu, here they will be able to access settings and help. This button is white, this is so it not only stands out on the table, but also from the other buttons.

The player's moves also have buttons. These include Hit, Stand, Double down, and Split.

As extra buttons (and clickables), the player can click on poker chips to bet them, there is a reset button in case the player bets the wrong more/misclicks any poker chips. Finally, there is a 'deal' button, this starts the round, and the 'dealer' deals out cards.

Table:

The table is a green colour, I have chosen this because Blackjack tables in real life, often have a green felt top. I have tried to make mine a similar colour, so the game is more immersive.

I have added the rules on the table: “Blackjack pays 2:1”. However, this looks slightly off and is missing some rules as they did not fit, I will need to fix these problems when I make my assets.

The Menu:

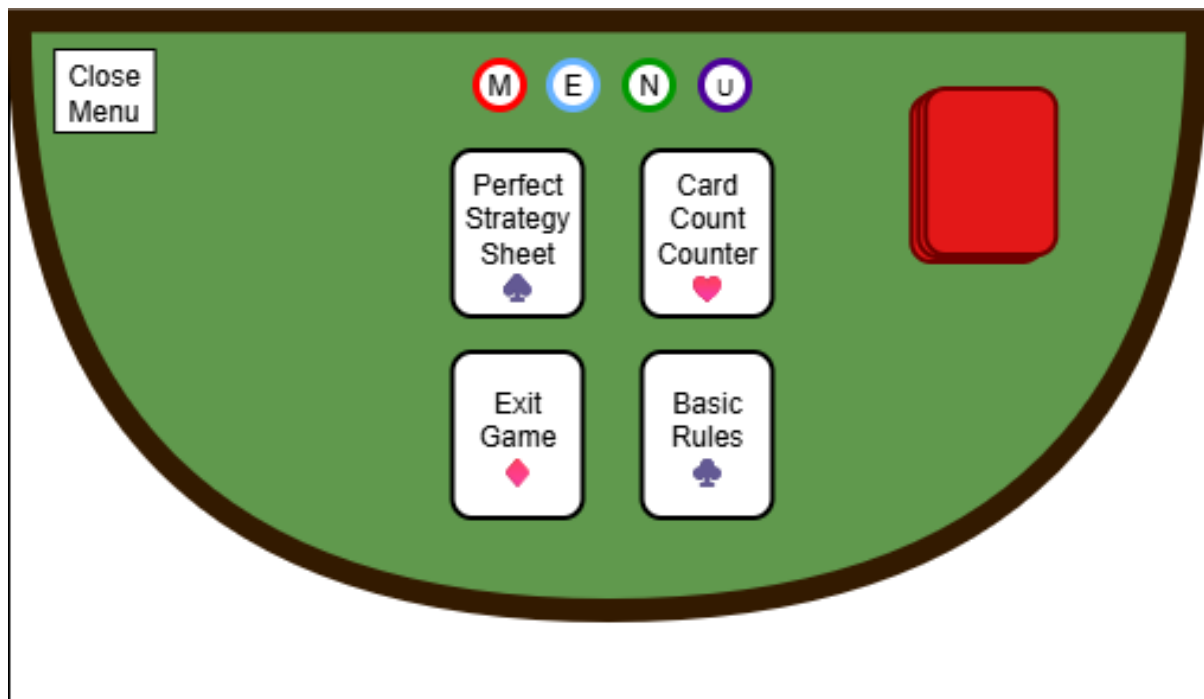


Figure 6 - 'Blackjack Menu Concept GUI'

Close Menu:

I have put the 'Close Menu' in the same place as the 'Menu' button in the main game (see figure 5) as that is normal for some games and will be easy to find. I have made the button white so that it stands out clearly against the green table.

'MENU' Title:

I have made the title 'M E N U' using the most vibrant poker chips from the main game (see figure 5) and changed the values to the letters: 'M', 'E', 'N', 'U'. The vibrant poker chips make it stand out and look appealing.

Menu Options:

There are four menu options: 'Perfect Strategy Cheat Sheet', 'Card Count Counter', 'Exit game', and 'Basic Rules'. I have designed them to look like playing cards shape and colour wise, and each have a playing card suit on them (as there are four menu options).

Perfect Strategy Cheat Sheet

Displays a perfect strategy cheat sheet

Card Count Counter

Displays the current card count

Exit Game

Exits the game

Basic Rules

Displays the basic rules to Blackjack

Extra Details:

I have also put a stack of face down cards; these are roughly in the same position and colour of the face down stack in the main game (see figure 2). I have made all the card shapes larger than the card shapes in the main game to fill out the emptiness of the table.

ITERATIVE DEVELOPMENT -----

Development Issue

The Issue:

As I started to develop my Blackjack game, I realised I would not have time to make a menu as I missed out the dealer and the overall flow of the game itself in my planning. Because of this, I have decided against implementing my menu and need to change my stages and their order.

My new stage order is:

Stage 1 – Deck of Cards and Card classes:

Creating the deck of cards class, and a card class. This is so that the deck of cards class can hold all the card objects and can use them in methods such as shuffling the deck and dealing cards.

I have decided to give the card a separate class as it will have its own methods. The cards will have attributes, such as their total value, that can easily be accessed through getters and setters. With this I will create card objects for all 52 cards which are needed.

Stage 2 – Player and Hand classes:

Creating the player class that will store information such as:

- Their total amount of money
- Total amount they have bet
- Whether they have won or lost
- The player's hands

And creating a separate hand class that can store cards inside of a data structure (such as an ArrayList). This will also mean a hand can hit, stand, double down, or split (rather than the moves being in the player class).

Stage 3 – Dealer class:

Creating the dealer class, including its hand, rules, and the final outcome of the game (who won and lost).

Stage 4 – Betting chips class:

Creating poker chips that can be clicked on that will add to the player's total bet.

Stage 5 – Main Game:

Coding the main flow of the game itself inside the main game world class. Including starting the game/rounds such as the betting, players, and dealers round.

Stage 6 – GUI (buttons, boxes, and text output classes):

Creating and coding a graphical user interface. This will include:

- Buttons for each of the player's moves.
- Buttons for clearing the players bet and starting the game (dealing the cards).
- Boxes that display information such as hand totals, player balance, and total money that has been bet.
- End game output screens e.g.: "YOU WON".

Deck of Cards & Card Classes (Stage 1)

Design

Flow Chart:

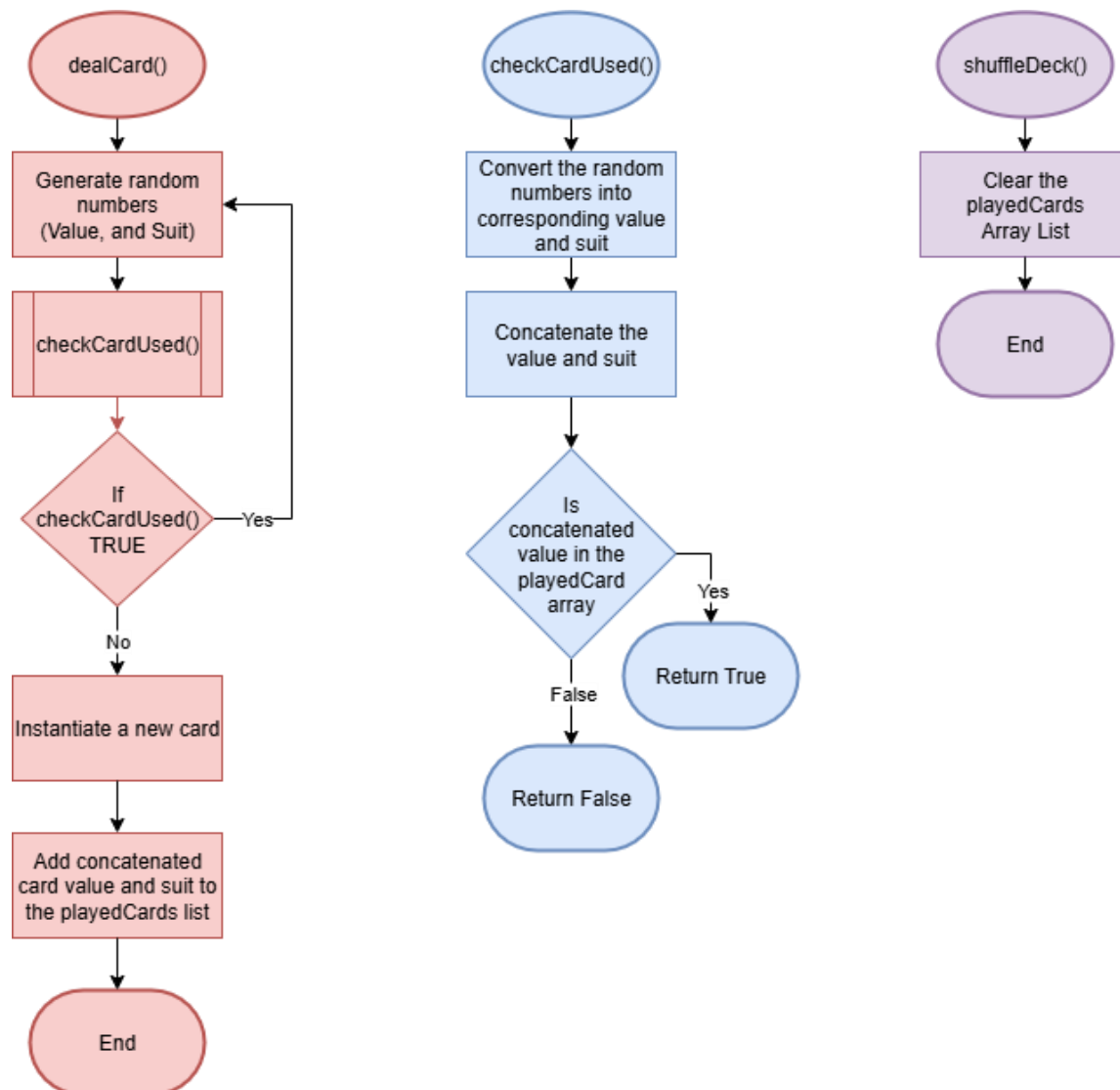


Figure 7 - "Deck of Cards - Flowchart"

I have made a flow chart to see the overall flow of the Deck of Cards methods: `dealCard()`, `checkCardUsed()`, and `shuffleDeck()`. I have coloured each method different colours to make it easier to read and understand.

Data Dictionary:

Class	Name	Data type	Description
DeckOfCards	unusedCards	ArrayList	Stores card objects, represents the draw pile/deck.
DeckOfCards	inUseCards	ArrayList	Stores card objects, represents the cards on the table, currently being played.
DeckOfCards	playedCards	ArrayList	Stores card objects, represents the discard pile.
Card	cardValue	integer	Stores the number of the card 1-13.
Card	cardSuit	character	Stores the first letter of the card's suit. E.g.: S - spades
Card	faceDown	Boolean	Stores whether the card is face down or face up.

Development -----

Developing the Classes:

I have several options for making my game:

- 1 – When cards are dealt, it is a randomly generated card, meaning you could get one card dealt repeatedly forever (despite the probability being extremely low). This would mean the game has an ‘infinite’ number of cards in the shoe which is not good.
- 2 – Have an array of what cards have been played already. When a new card is dealt, it is randomly generated. However, it will first check the generated card to see if it is in the list of played cards, if it is, it will attempt to generate a new card until it is unique. I would need to use a search algorithm to do this.
- 3 – Having an array of card objects that can be shuffled, and cards removed from as they are dealt.

I am going to go with option 2, as this is a solution that is not too complex but will still function the same way as I planned for my game.

Coding the Class – ‘DeckOfCards’:

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
import java.util.ArrayList; // For array lists

public class DeckOfCards extends Actor
{
    ArrayList<String> playedCards = new ArrayList<String>(); //Played cards go here
```

For the **DeckOfCards** class, I have created an Array List, **playedCards**, to store the cards that have already been dealt in the game. This is because if they have already been dealt, I can check a generated card against this list, and they should not be able to be dealt again.

As this could potentially be a large list of cards (or a small list), it made more sense to me to have a non-static data structure to store them in. Because of this, I have chosen to use an Array List, however I have not coded them in Java before, so I had to research how to do it. I used this website: [Java ArrayList](#). To use an Array List, I had to import ‘java.util.ArrayList’.

Defining Methods:

dealCard():

```

public void dealCard(){
    // Randomly select a card
    String cardValue = "N";
    String cardSuit = "N";
    int rand1 = -1;
    int rand2 = -1;

    // Might need to check if there are cards left in the deck later on *****
    while (checkCardUsed(rand1, rand2) == true){
        rand1 = Greenfoot.getRandomNumber(13); // (0 <= x < 13), one more than array because array starts at 0
        rand2 = Greenfoot.getRandomNumber(4);
    }

    if (rand1 < 10){
        cardValue = String.valueOf(rand1+1);
    }
    else{
        switch (rand1){
            case 10:
                cardValue = "J";
                break;
            case 11:
                cardValue = "Q";
                break;
            case 12:
                cardValue = "K";
                break;
        } //Converts >10 valued cards to court cards
    } //Converts rand1 into the value of the card

    switch (rand2){
        case 0:
            cardSuit = "S";
            break;
        case 1:
            cardSuit = "H";
            break;
        case 2:
            cardSuit = "C";
            break;
        case 3:
            cardSuit = "D";
            break;
    } // Sets cardSuit to the card's suit

    String cardIs = cardValue + cardSuit;
    playedCards.add(cardIs);
    Card card = new Card(rand1, cardSuit, cardIs);
    //instantiate new card with rand1 as value and cardSuit as suit
}

```

The `dealCard()` method simulates the dealing of a card from the top of a shuffled deck. Because I have not made a deck of cards that can be shuffled, I am generating a random card in this method.

It does this by generating a random number (see 'Extra References') for the value of the card, and a random number for the suit of the card. It then converts these values to the respective string values and concatenates them to make the variable 'cardIs'. This variable will then be used in other classes.

The method ends by instantiating a card object.

shuffleDeck():

```
public void shuffleDeck()  
{  
    // Resets the deck, empties the 'played cards' array.  
    playedCards.clear();  
}
```

I have made a method for 'shuffling' the deck. This method empties the 'played cards' array, meaning they can be dealt again, effectively 'shuffling' the played cards back into the deck.

Changes:

As I started to code the DeckOfCards class, I realised my game would be more efficient if I coded it using the *third* option I came up with, rather than the *second*. My third option was to have an array of card objects that can be shuffled, and cards removed from as they are dealt.

I am now going to recode my already written code to meet this option. I will then continue to code it this way.

Updated Flow Chart:

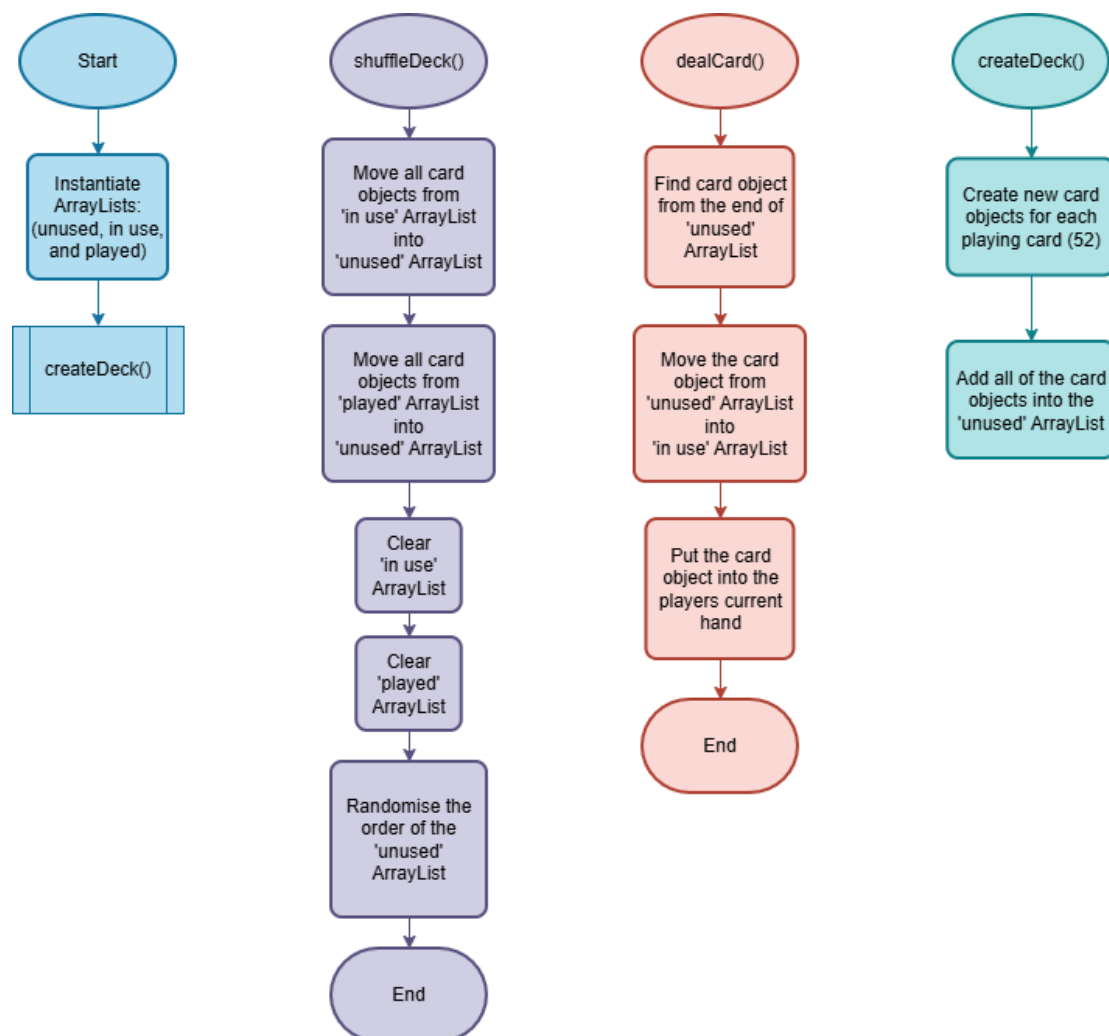


Figure 8 - "Deck of Cards - Updated Flowchart"

I have updated the flowchart as it was incorrect for the new way I am going to code my game. It is now updated with the new / modified methods, and I have coloured each method different colours to make it easier to read and understand.

Re-Coding the Class – ‘DeckOfCards’:

Defining Updated Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
import java.util.ArrayList; // For array lists
import java.util.Collections; // Used to randomise the order of data in an ArrayList

public class DeckOfCards extends Actor
{
    private ArrayList<Card> unusedCards; // UNUSED cards go here
    private ArrayList<Card> inUseCards; // IN USE cards go here
    private ArrayList<Card> playedCards; // PLAYED cards go here
}
```

I only had one array before which stored the cards that had already been played, whereas I need *more than one array*.

I have created two more Array Lists called ‘**unusedCards**’ and ‘**inUseCards**’. These two ArrayLists, and ‘**playedCards**’ store Card objects.

These are to store the cards that have:

- *Not yet been dealt/used (unusedCards)*
- *Cards that are currently being used in the game, have been dealt but not yet played (inUseCards)*
- *Have already been dealt and used in the game.*

This is so I can easily move specific card objects between these ArrayLists.

As these ArrayLists will change in how many objects they are each storing, because they will move between ArrayLists, it made more sense to me to have a non-static data structure to store them in. This will also reduce the amount of storage the program will use. Because of this, I have chosen to use Array Lists.

Importing Updated Libraries:

To use Array Lists and Randomise the order of their data, I have imported the libraries: ‘ArrayList’ and ‘Collections’. (see more detail in ‘Defining Updated Variables & Attributes and Importing Libraries’)

Creating the Constructor:

DeckOfCards():

```
public DeckOfCards () {  
    this.unusedCards = new ArrayList<Card>();    // UNUSED cards go here  
    this.inUseCards = new ArrayList<Card>();    // IN USE cards go here  
    this.playedCards = new ArrayList<Card>();    // PLAYED cards go here  
    createDeck();  
}
```

I have created a constructor which instantiates the three ArrayLists needed for the deck to function. (For more detail on the ArrayLists see 'Coding the Class – 'Card', 'Defining Variable and Attributes')

Defining Updated Methods:

shuffleDeck():

```
public void shuffleDeck()  
{  
    // Puts all playedCards into the unusedCards ArrayList and randomises their order.  
    unusedCards.addAll(inUseCards);  
    unusedCards.addAll(playedCards);  
    inUseCards.clear();  
    playedCards.clear();  
    Collections.shuffle(unusedCards);    // Randomises the order of the data in 'unusedCards'  
}
```

The '**shuffleDeck**' method adds all card objects to 'unusedCards', it then removes any cards from the other two ArrayLists: 'inUseCards' and 'playedCards'. The ArrayList 'unusedCards' is then "shuffled", this is done using the method 'shuffle', which is part of the 'Collections' library.

Because I didn't know how to use ArrayLists in Java until this project, I had to research on how to randomise the order of the data in an ArrayList. For other ArrayList methods such as 'addAll' and 'clear' I used a website to learn the methods. I used these websites:

[Getting Random Elements from ArrayList in Java - GeeksforGeeks](#) (Randomising order)

[Java ArrayList Reference](#) (ArrayList methods)

useCard(Card c) & playCard(Card c):

```
public void useCard(Card c){  
    // Adds a passed card object to the 'inUseCards' ArrayList  
    inUseCards.add(c);  
    unusedCards.remove(c);  
}  
  
public void playCard(Card c){  
    // Adds a passed card object to the 'playedCards' ArrayList  
    playedCards.add(c);  
    inUseCards.remove(c);  
}
```

I have made these methods so I can quickly move card objects between ArrayLists.

These methods work by taking card objects as parameters, then adding them to an ArrayList and removing it from the previous one.

The “**useCard**” method adds the passed card object to the “inUseCards” ArrayList and removes it from the “unusedCards” ArrayList.

The “**playCard**” method adds the passed card object to the “playedCards” ArrayList and removes it from the “inUseCards” ArrayList.

createDeck():

```
public void createDeck(){
    // Adds all card to 'unusedCards'
    // Card (int cardValue, int valueOfCard, String cardSuit, String cardIs, boolean faceDown)

    // SPADES -----
    unusedCards.add(new Card(1, 11, "S", "AS", false)); //Ace of Spades
    unusedCards.add(new Card(2, 2, "S", "2S", false)); //Two of Spades
    unusedCards.add(new Card(3, 3, "S", "3S", false)); //Three of Spades
    unusedCards.add(new Card(4, 4, "S", "4S", false)); //Four of Spades
    unusedCards.add(new Card(5, 5, "S", "5S", false)); //Five of Spades
    unusedCards.add(new Card(6, 6, "S", "6S", false)); //Six of Spades
    unusedCards.add(new Card(7, 7, "S", "7S", false)); //Seven of Spades
    unusedCards.add(new Card(8, 8, "S", "8S", false)); //Eight of Spades
    unusedCards.add(new Card(9, 9, "S", "9S", false)); //Nine of Spades
    unusedCards.add(new Card(10, 10, "S", "10S", false)); //Ten of Spades
    unusedCards.add(new Card(11, 10, "S", "JS", false)); //Jack of Spades
    unusedCards.add(new Card(12, 10, "S", "QS", false)); //Queen of Spades
    unusedCards.add(new Card(13, 10, "S", "KS", false)); //King of Spades
    //-----

    // HEARTS -----
    unusedCards.add(new Card(1, 11, "H", "AH", false)); //Ace of Hearts
    unusedCards.add(new Card(2, 2, "H", "2H", false)); //Two of Hearts
    unusedCards.add(new Card(3, 3, "H", "3H", false)); //Three of Hearts
    unusedCards.add(new Card(4, 4, "H", "4H", false)); //Four of Hearts
    unusedCards.add(new Card(5, 5, "H", "5H", false)); //Five of Hearts
    unusedCards.add(new Card(6, 6, "H", "6H", false)); //Six of Hearts
    unusedCards.add(new Card(7, 7, "H", "7H", false)); //Seven of Hearts
    unusedCards.add(new Card(8, 8, "H", "8H", false)); //Eight of Hearts
    unusedCards.add(new Card(9, 9, "H", "9H", false)); //Nine of Hearts
    unusedCards.add(new Card(10, 10, "H", "10H", false)); //Ten of Hearts
    unusedCards.add(new Card(11, 10, "H", "JH", false)); //Jack of Hearts
    unusedCards.add(new Card(12, 10, "H", "QH", false)); //Queen of Hearts
    unusedCards.add(new Card(13, 10, "H", "KH", false)); //King of Hearts
    //-----

    // CLUBS -----
    unusedCards.add(new Card(1, 11, "C", "AC", false)); //Ace of Clubs
    unusedCards.add(new Card(2, 2, "C", "2C", false)); //Two of Clubs
    unusedCards.add(new Card(3, 3, "C", "3C", false)); //Three of Clubs
    unusedCards.add(new Card(4, 4, "C", "4C", false)); //Four of Clubs
    unusedCards.add(new Card(5, 5, "C", "5C", false)); //Five of Clubs
    unusedCards.add(new Card(6, 6, "C", "6C", false)); //Six of Clubs
    unusedCards.add(new Card(7, 7, "C", "7C", false)); //Seven of Clubs
    unusedCards.add(new Card(8, 8, "C", "8C", false)); //Eight of Clubs
    unusedCards.add(new Card(9, 9, "C", "9C", false)); //Nine of Clubs
    unusedCards.add(new Card(10, 10, "C", "10C", false)); //Ten of Clubs
    unusedCards.add(new Card(11, 10, "C", "JC", false)); //Jack of Clubs
    unusedCards.add(new Card(12, 10, "C", "QC", false)); //Queen of Clubs
    unusedCards.add(new Card(13, 10, "C", "KC", false)); //King of Clubs
    //-----

    // DIAMONDS -----
    unusedCards.add(new Card(1, 11, "D", "AD", false)); //Ace of Diamonds
    unusedCards.add(new Card(2, 2, "D", "2D", false)); //Two of Diamonds
    unusedCards.add(new Card(3, 3, "D", "3D", false)); //Three of Diamonds
    unusedCards.add(new Card(4, 4, "D", "4D", false)); //Four of Diamonds
    unusedCards.add(new Card(5, 5, "D", "5D", false)); //Five of Diamonds
    unusedCards.add(new Card(6, 6, "D", "6D", false)); //Six of Diamonds
    unusedCards.add(new Card(7, 7, "D", "7D", false)); //Seven of Diamonds
    unusedCards.add(new Card(8, 8, "D", "8D", false)); //Eight of Diamonds
    unusedCards.add(new Card(9, 9, "D", "9D", false)); //Nine of Diamonds
    unusedCards.add(new Card(10, 10, "D", "10D", false)); //Ten of Diamonds
    unusedCards.add(new Card(11, 10, "D", "JD", false)); //Jack of Diamonds
    unusedCards.add(new Card(12, 10, "D", "QD", false)); //Queen of Diamonds
    unusedCards.add(new Card(13, 10, "D", "KD", false)); //King of Diamonds
    //-----
}
```

I have made a method, “createDeck” for ‘creating’ the deck (see *previous page*). This method creates 52 card objects and adds them to the ‘unusedCards’ ArrayList. This is effectively putting 52 cards into a ‘deck’/draw pile.

Each card object is instantiated with five attributes (see *Card class*). This method would only be run once at the beginning of the game to initialise the 52 Card objects.

dealCard(Card c):

```
public Card dealCard(){  
    // Take card from deck  
    Card newCard = unusedCards.get(unusedCards.size()-1);  
    useCard(newCard);  
    return newCard;  
}
```

I have created a method that gets a card object from the end of the unusedCards ArrayList, moves it to inUseCards, and returns it. I chose to take the card from the end of the unusedCards ArrayList as an ArrayList will automatically resize itself to only fit the remaining cards. This means I don’t have to run any sorts on unusedCards, making my program more efficient.

Extra Code:

In the MainGame World class:

```
public class MainGame extends World  
{  
    DeckOfCards deck = new DeckOfCards();  
  
    public DeckOfCards getDeck()  
    {  
        return deck;  
    }  
}
```

I have instantiated a deck object from the DeckOfCards class in my MainGame class so I can run its methods in my game.

I have also created a method, ‘getDeck()’, which returns the DeckOfCards object. This is so the deck can be accessed from other classes such as the Hand class which will need to access the deck to deal a card to itself.

Coding the Class – ‘Card’:

Defining Variables & Attributes:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
```

```
public class Card extends Actor
{
    private int cardValue;
    private int valueOfCard;
    private String cardSuit;
    private String cardIs;
    private boolean faceDown;
```

I have made 5 attributes for the Card objects:

cardValue – Stores an Integer of the card’s value e.g.: 1 for ace, 11 for jack, 13 for king.

valueOfCard – Stores an Integer of the value of the card within Blackjack e.g.: 10 for ten, jack, queen, king.

cardSuit – Stores a String of the card suit initial e.g.: S - Spade, H – Heart, C – Clubs, D – Diamonds.

cardIs – Stores a String of the card’s value and suit e.g.: AH – Ace of Hearts, KS – King of Spades, 2C – Two of Clubs.

faceDown – Stores a Boolean of whether the card is face up or face down.

Defining Constructor & Methods:

Card():

```
public Card (int cardVal, int valOfCard, String cardSu, String cardIS, boolean faceDOWN){
    this.cardValue = cardVal;
    this.valueOfCard = valOfCard;
    this.cardSuit = cardSu;
    this.cardIs = cardIS;
    this.faceDown = faceDOWN;
```

I have created a constructor which takes the five attributes (set in DeckOfCards) as parameters and sets the object’s variables to them.

(For more detail on the attributes see ‘Coding the Class – ‘Card’, ‘Defining Variable and Attributes’)

Getters & Setters:

```
public int getCardValue(){  
    return this.valueOfCard;  
}  
  
public void faceDown(){  
    this.faceDown = true;  
}  
  
public void faceUp(){  
    this.faceDown = false;  
}  
  
public boolean getFaceDown(){  
    return this.faceDown;  
}
```

(For more detail on the attributes see ‘Coding the Class – ‘Card’, ‘Defining Variable and Attributes’)

getCardValue():

I have created a getter method that returns the Integer value of the card in Blackjack .

faceDown():

I have created a setter method that sets the attribute **faceDown** to true.

faceUp():

I have created a setter method that sets the attribute **faceDown** to false.

getFaceDown():

I have created a getter method that returns the **faceDown** attribute’s value.

Testing -----

Test 1a – Testing the **shuffleDeck()** method:

Expected Result: The unusedCards ArrayList is not in the same order before and after running the method.

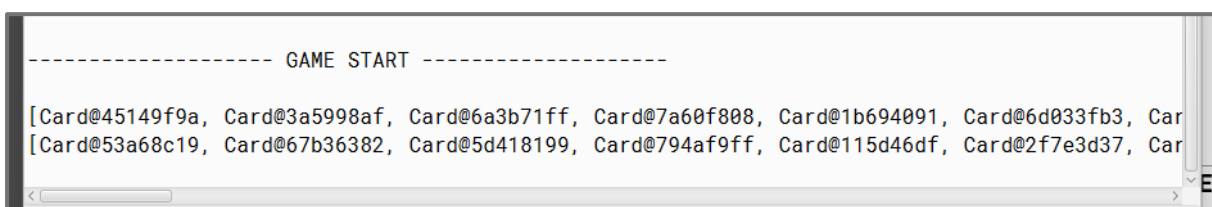
To test this, I have written some code in the MainGame world class that will output the unusedCards ArrayList before and after running the method:

```
// TESTING
System.out.println(deck.getUnusedCards());
deck.shuffleDeck();
System.out.println(deck.getUnusedCards());
```

I also had to make a getter method in the DeckOfCards so I can access the ArrayList directly from the MainGame class as I couldn't before:

```
// FOR TESTING
public ArrayList<Card> getUnusedCards(){
    return this.unusedCards;
}
```

Result (Passed):



```
----- GAME START -----
[Card@45149f9a, Card@3a5998af, Card@6a3b71ff, Card@7a60f808, Card@1b694091, Card@6d033fb3, Car
[Card@53a68c19, Card@67b36382, Card@5d418199, Card@794af9ff, Card@115d46df, Card@2f7e3d37, Car
```

The first output is the ArrayList before the shuffleDeck method is run, and the second output is the ArrayList after the method has been run (as there are 52 card objects I cannot show the entire output in one screenshot; however, it does output all 52).

I can see that the Card objects in the ArrayList are in a different order after the method was run, this means the shuffleDeck method has worked.

As I was testing this back-to-back, I noticed that the ArrayList is in a different order when I run the program each time:

```
----- GAME START -----  
[Card@300ceb3c, Card@3ba7f7e8, Card@547d7a96, Card@602fc3a6, Card@37ad9194,  
[Card@50b61a7e, Card@644277af, Card@23169673, Card@70c049e3, Card@46e9ae4f,  
  
----- GAME START -----  
[Card@45149f9a, Card@3a5998af, Card@6a3b71ff, Card@7a60f808, Card@1b694091,  
[Card@53a68c19, Card@67b36382, Card@5d418199, Card@794af9ff, Card@115d46df,
```

I am unsure why this happens, however this is fine as I won't need the deck in order at any point in my game, as long as the ArrayList order changes after running the method, then it works correctly.

Test 1b & c – Testing the **getCardValue()** & **dealCard()** methods:

Expected Result: The Card getter method should return a Card object's value and the dealCard() method should return the first card from the ArrayList every time.

To test this, I have written some more code in the MainGame world class that will output the first card in the ArrayList using the dealCard() method, therefore testing them both:

```
// TESTING
System.out.println(deck.getUnusedCards());
System.out.println("Card Value = " + deck.dealCard().getCardValue());
deck.shuffleDeck();
System.out.println(deck.getUnusedCards());
System.out.println("Card Value = " + deck.dealCard().getCardValue());
```

Result (Passed):

```
----- GAME START -----

[Card@3f505748, Card@4e76d168, Card@5288819, Card@4ea54fc8, Card@73f2ab43, Card@b77c9c4, Card@506a0287, Card@5d1fab5c, Card@3bf5edd9, Card@52d298b1, Card@9833f75, Card@23a169cd, Card@3b334380, Card@6c317c8, Card@55fd8f34, Card@36bbc607, Card@17b5ac13, Card@9d6847a, Card@306b2e5b, Card@15c1d768, Card@26c40dd7, Card@488bb33a, Card@72c7eacf, Card@4b125fe1]
Card Value = 10
[Card@506a0287, Card@5d1fab5c, Card@3bf5edd9, Card@52d298b1, Card@9833f75, Card@23a169cd, Card@3b334380, Card@6c317c8, Card@55fd8f34, Card@36bbc607, Card@17b5ac13, Card@9d6847a, Card@306b2e5b, Card@15c1d768, Card@26c40dd7, Card@488bb33a, Card@72c7eacf, Card@4b125fe1]
Card Value = 11

----- GAME START -----

[Card@3f505748, Card@4e76d168, Card@5288819, Card@4ea54fc8, Card@73f2ab43, Card@b77c9c4, Card@506a0287, Card@5d1fab5c, Card@3bf5edd9, Card@52d298b1, Card@9833f75, Card@23a169cd, Card@3b334380, Card@6c317c8, Card@55fd8f34, Card@36bbc607, Card@17b5ac13, Card@9d6847a, Card@306b2e5b, Card@15c1d768, Card@26c40dd7, Card@488bb33a, Card@72c7eacf, Card@4b125fe1]
Card Value = 10
[Card@306b2e5b, Card@15c1d768, Card@26c40dd7, Card@488bb33a, Card@72c7eacf, Card@4b125fe1, Card@3f505748, Card@4e76d168, Card@5288819, Card@4ea54fc8, Card@73f2ab43, Card@b77c9c4, Card@506a0287, Card@5d1fab5c, Card@3bf5edd9, Card@52d298b1, Card@9833f75, Card@23a169cd, Card@3b334380, Card@6c317c8, Card@55fd8f34, Card@36bbc607, Card@17b5ac13, Card@9d6847a, Card@306b2e5b, Card@15c1d768, Card@26c40dd7, Card@488bb33a, Card@72c7eacf, Card@4b125fe1]
Card Value = 4
```

I can see that the Card objects have been correctly returned by the **dealCard()** method and that the getter method **getCardValue()** has also worked as it has output

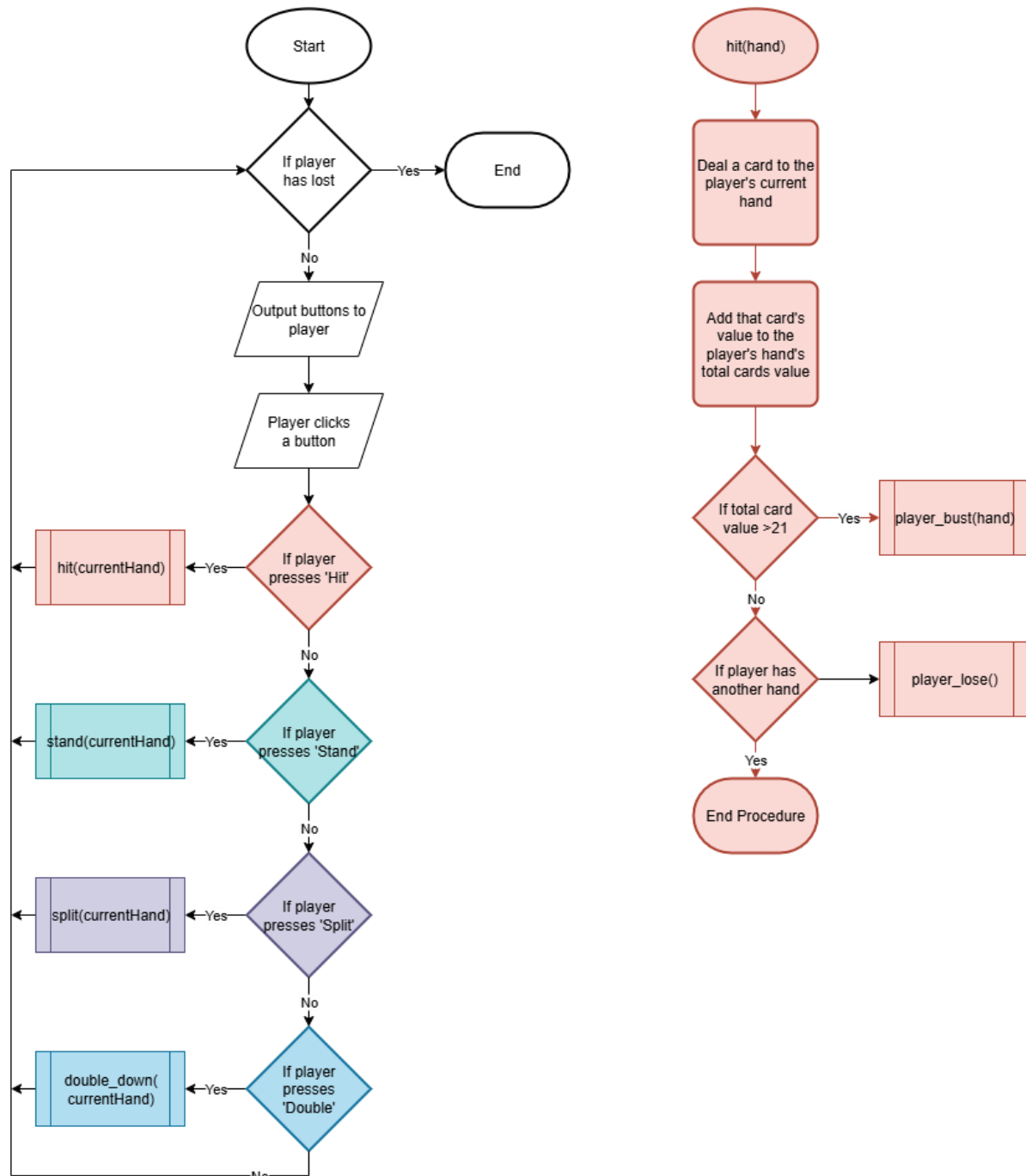
Extra Notes:

As I was testing this back-to-back, I noticed that the previous issues I noticed about the deck starting in a random order was false; It *does* start in the correct order, but it generates a random name for the object each time I run the code which led me to believe they were different card objects.

Player & Hand Classes (Stage 2)

Design

Flowchart:



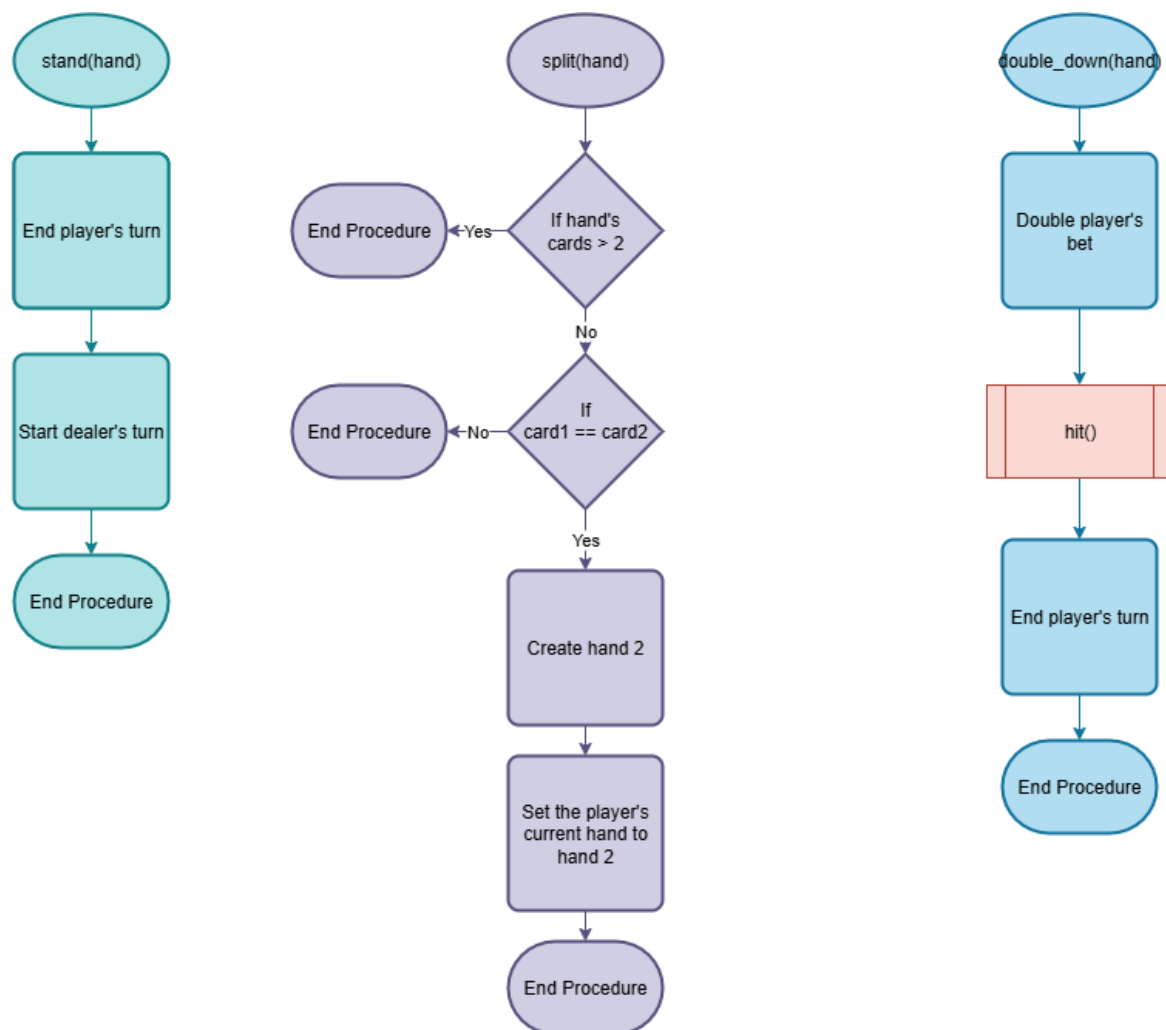


Figure 9 - 'Player Moves - Flowchart'.

I have made this flowchart as a rough overview of how the Player's Moves will work in my game. I have coloured the flowchart for easier viewing, with subroutines and related code different colours to each other.

When it is the player's turn, if they click on certain buttons, they will run different subroutines. These subroutines include:

- 'hit(hand)'
- 'stand(hand)'
- 'split(hand)'
- 'double_down(hand)'

These subroutines take 'hand' as a parameter so the program knows where to run the moves. This is because players can have more than one card hand (due to the option of 'splitting', where the player can split their hand into two separate hands).

Data Dictionary:

Name	Data type	Description	Validation
playerLost	Boolean	Stores whether the player has lost the round or not.	'true' or 'false'
handTotal	Integer	Stores the total value of the cards in a player's hand.	≥ 0
card1Value	Integer	Stores the value of the first card dealt to a hand.	≥ 0
card2Value	Integer	Stores the value of the second card dealt to the hand.	≥ 0
currentHand	Integer	Stores the hand (or number hand) that is currently in play.	Starts equal to 0

Development -----

Developing the Classes:

Instead of creating just the player moves for this stage, I am going to create a hand class which will store the player's cards, and I am going to create a player class which will have the player's moves as methods and attributes such as how much money they have and how many hands they have.

Coding the Class – 'Hand':

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
import java.util.ArrayList; // For array lists

public class Hand extends Actor
{
    private int handTotal;
    private int card1Value;
    private int card2Value;
    private boolean handLost;
    private ArrayList<Card> cardsInHand;

    // An ArrayList for the card objects in each hand.
```

Figure **NUMBER SOMETHING**

For the **Hand** class, I have created 5 variables:

handTotal – Used to store an integer value of the hand's total value.

card1Value – Used to store an integer value of the *first* card dealt to the hand. Used with *card2Value* to check whether a split is possible.

card2Value – Used to store an integer value of the *second* card dealt to the hand.

handLost – Used to store a Boolean value of whether the hand has gone bust (lost).

I have created an Array List, **cardsInHand**, this will store the card objects that are in each hand.

*Creating the Constructor:**Hand():*

```
public Hand () {  
    World world = getWorld();  
    MainGame mainGame = (MainGame)world;  
    DeckOfCards deck = mainGame.getDeck();  
  
    this.cardsInHand = new ArrayList<Card>();  
  
    Card card1 = deck.dealCard();  
    this.cardsInHand.add(card1);  
    this.card1Value = card1.getCardValue();  
  
    Card card2 = deck.dealCard();  
    this.cardsInHand.add(card2);  
    this.card2Value = card2.getCardValue();  
  
    this.handTotal = card1Value + card2Value;  
}
```

I have made a constructor class that creates a hand object.

This creates an ArrayList that will store cards in the hand. It then uses the dealCard() method from the deck object (see *Stage 1 of iterative development*) twice and adds the two cards to the **cardsInHand** ArrayList. It then updates the **handTotal** to be the values of the two cards combined.

When I tried running my game, it came back with a Null Pointer Exception Error (see *below*), this is because the Hand constructor was attempting to access the world class before it had been created in the game.

```
java.lang.NullPointerException: Cannot invoke "MainGame.getDeck()" because "mainGame" is null  
    at Hand.startHand(Hand.java:39)  
    at MainGame.<init>(MainGame.java:21)
```

To fix this I am going to make a separate method, **startHand(World world)**, from code that is currently in the Hand constructor, that will access the world class through a passed in parameter.

Extra Changes:

As I started to code the Hand class, I was doing some research into the rules of Blackjack to check what I know about the game. While doing this I found out that normally, dependant on specific casino(s) rules, the player can only split up to three time (resulting in four total hands). I originally though you could have an unlimited number of hands, which would've made my game a lot harder to code.

Now that I know this, I am going to code my game to allow a maximum of four hands.

Re-Coding the Class – 'Hand':

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
import java.util.ArrayList; // For array lists

public class Hand extends Actor
{
    private boolean handLost;
    private boolean blackjack;
    private ArrayList<Card> cardsInHand; // An ArrayList for the card objects in each hand.
    private int card1Value;
    private int card2Value;
    private int handTotal;
    private int lastPos;
```

Figure **NUMBER SOMETHING**

For the **Hand** class, I have created 7 variables:

handLost – Used to store a Boolean value of whether the hand has gone bust (lost).

blackjack – Used to store a Boolean value on if a hand has Blackjack or not (if a hand's card 1 value + card 2 value = 21).

I have created an Array List, **cardsInHand**, this will store the card objects that are in each hand. I have chosen to use an ArrayList as it will store different numbers of cards throughout the game. An ArrayList is dynamic, so it fits this.

card1Value – Used to store an integer value of the *first* card dealt to the hand. Used with *card2Value* to check whether a split is possible.

card2Value – Used to store an integer value of the *second* card dealt to the hand.

handTotal – Used to store an integer value of the hand's total value.

lastPos – Used to store an integer value for x value of where the previous card was dealt.

I also imported the **java.util.Arraylist** library so I could use ArrayLists and its methods.

*Creating the Constructor:**Hand():*

```
//FOR FIRST HAND -----  
public Hand () {  
    this.handLost = false;  
    this.blackjack = false;  
    this.lastPos = 575;  
    this.cardsInHand = new ArrayList<Card>();  
}
```

I have created a constructor which instantiates the hand object, it sets **handLost** and **blackjack** to false, sets the **lastPos** to 575 so that the cards are added in the correct part of the world. Finally, it instantiates a **cardsInHand** ArrayList to store the card objects in the hand.

*Defining Methods:**startHand():*

```
public void startHand(World world) {  
    MainGame mainGame = (MainGame)world;  
    DeckOfCards deck = mainGame.getDeck();  
    Player player = mainGame.getPlayer();  
  
    player.addToHandList(this);  
  
    Card card1 = deck.dealCard();  
    this.cardsInHand.add(card1);  
    this.card1Value = card1.getCardValue();  
    mainGame.addObject(card1, 490, 430);  
  
    Card card2 = deck.dealCard();  
    this.cardsInHand.add(card2);  
    this.card2Value = card2.getCardValue();  
    mainGame.addObject(card2, 575, 430);  
  
    this.handTotal = card1Value + card2Value;  
}
```

I have made a method for 'starting' the hand. This will be run after the betting round I will make. As explained, I have moved a lot of code from the constructor to this separate method (startHand) as it caused null pointer exception errors. Doing this fixed it.

Hand(Card cardTwo):

```
//FOR SPLITTING -----  
public Hand (Card cardTwo){  
    this.handLost = false;  
    this.blackjack = false;  
  
    World world = getWorld();  
    MainGame mainGame = (MainGame)world;  
    DeckOfCards deck = mainGame.getDeck();  
    Player player = mainGame.getPlayer();  
  
    player.addToHandList(this);  
  
    this.cardsInHand = new ArrayList<Card>();  
  
    Card card1 = cardTwo;  
    this.cardsInHand.add(card1);  
    this.card1Value = card1.getCardValue();  
  
    Card card2 = deck.dealCard();  
    this.cardsInHand.add(card2);  
    this.card2Value = card2.getCardValue();  
  
    this.handTotal = card1Value + card2Value;  
} //polymorphism for splitting
```

I have started to make another constructor using polymorphism (overloading) for when a hand splits. This runs very similar code to the original constructor, but it takes a parameter of the first hand's second card and sets it to be the first card of the second hand.

hit():

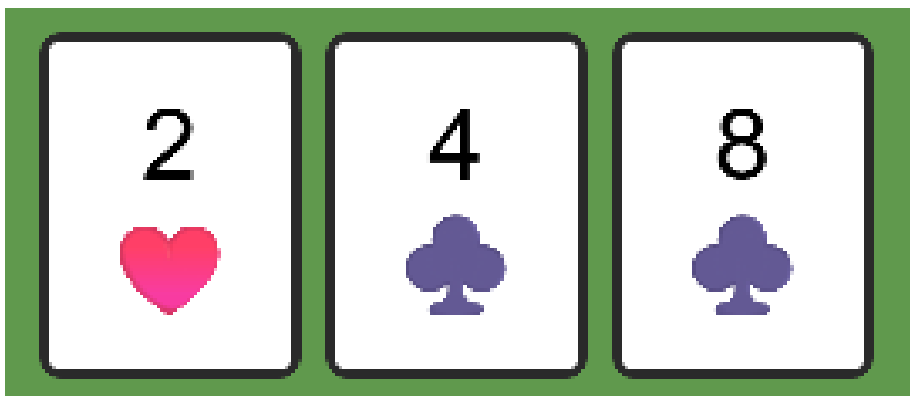
```
public void hit(World world)
{
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();
    Player player = mainGame.getPlayer();

    Card newCard = deck.dealCard();
    this.cardsInHand.add(newCard);
    this.handTotal = this.handTotal + newCard.getCardValue();
    mainGame.addObject(newCard, this.lastPos + 85, 430);
    lastPos = lastPos + 85;

    checkHandStatus();

    System.out.println("*Hit, Hand total = " + this.handTotal);
}
```

I have created a method that gets a card object from the deck and adds it to the hand using the method. It also updates the hands attributes such as **handTotal**, including adding 85 to **lastPos** so that the next card is placed visually in the correct position, so they don't overlap and aren't too close. Example:



Spaced correctly. Not too close that they overlap and not too far apart to take up space.

Finally, the method outputs details such as the hand total to the terminal for future testing to check the game is running correctly after hitting.

To remind myself on how to output text to the terminal in Java, I used this website:

https://www.w3schools.com/java/java_output.asp

stand():

```
public void stand(World world)
{
    //
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();
    Dealer dealer = mainGame.getDealer();

    player.setPlayerTurn(false);

    System.out.println("*Stood, Hand total = " + this.handTotal);

    //if( player.getCurrentHand() == player.getLastHand() ){
    dealer.dealerTurn(world);
    //}
}
```

I have created a method that allows the player to stand. It does this by changing the player's turn to false (see player class for setPlayerTurn() method). It again outputs that the player has stood, and the hand total to the terminal. Finally, it starts the dealers turn (see mainGame class).

doubleDown():

```
public void doubleDown(World world)
{
    //
    MainGame mainGame = (MainGame)world;

    this.hit(mainGame);
    this.stand(mainGame);

    System.out.println("*Doubled Down, Hand total = " + this.handTotal);
}
```

I have created a method that allows the player to double down. This method runs the hit method, followed by the stand method. This method also has logging outputted to the terminal so I can test my game later on.

split():

```
public void split()
{
    World world = getWorld();
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();
    Player player = mainGame.getPlayer();

    if (card1Value == card2Value && (player.getPlayerTurn() == true)){
    }
}
```

I have started making a method for the player to split. I don't think I am going to finish implementing splitting into my game due to time constraints; however, I have started to put code in place if I still have time left over.

checkHandStatus():

```
public void checkHandStatus(){
    if (this.handTotal >= 22){
        this.handLost = true;
    }
    else{
        this.handLost = false;
    }

    if ((this.card1Value + this.card2Value) == 21){
        this.blackjack = true;
    }
    else{
        this.blackjack = false;
    }

    System.out.println("*Hand total = " + this.handTotal);
    System.out.println("*Blackjack = " + this.blackjack);
    System.out.println("*Hand Lost = " + this.handLost);
}
```

I have made a method that is run after all the player's actions (Hit, Stand, Double Down, and Split). This checks the hand total and updates attributes (**handLost** and **blackjack**) on whether the hand has gone bust (lost), and whether the hand has achieved a 'Blackjack' and got a hand total of 21 with their two starting cards. It originally checked whether their hand total was 21, but then I realised that getting Blackjack is when only your original two cards total 21.

The method also outputs details about the hand to a terminal window to be checked during testing to make sure the game is running correctly.

Getters & Setters:

```
public int getHandTotal(){  
    return (this.handTotal);  
}  
  
public boolean getBlackjack(){  
    return (this.blackjack);  
}  
  
public boolean getHandLost(){  
    return this.handLost;  
}  
  
public void setHandLost(Boolean value){  
    this.handLost = value;  
}
```

(For more detail on the attributes see 'Coding the Class – 'Hand', 'Defining Variables and Attributes')

getHandTotal():

I have created a getter method that returns the Integer value of the hand total, this method will be used a lot in other classes as it will be used to compare values a lot.

getBlackjack():

I have created a getter method that returns a Boolean value of whether the hand has blackjack or not.

getHandLost():

I have created a getter method that returns a Boolean value of whether the hand has gone bust (lost) or not.

setHandLost():

I have created a setter method that takes a Boolean value as a parameter and sets **handLost** to it.

Coding the Class – ‘Player’:

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
import java.util.LinkedList; // For linked lists

public class Player extends Actor
{
    private int playerMoney;
    private int totalBet;
    private boolean playerLost;
    private int currentHand;
    private LinkedList<Hand> handsList;
    private int lastHand;
    private boolean isTurn;
    // Have a List for the hands.
    // Will need to check if the list is empty.
```

For the **Hand** class, I have created 7 variables:

playerMoney – Used to store an Integer value of the player’s total money.

totalBet – Used to store an Integer value of the total money the player has bet.

playerLost – Used to store a Boolean value on if the player has lost.

currentHand – Used to store an Integer value of the index of the player’s hand that is current active/playing.

I have created a Linked List, **handsList**, to store a list of the player’s hands. I have chosen to use a LinkedList as it stores data more efficiently than other data structures. As I haven’t coded using a linked list before, I had to research how to and what built in methods I could use with it. I used this website to these things out: [Java LinkedList](#).

lastHand – Used to store an Integer value of the index of the player’s last hand.

isTurn – Used to store a Boolean value of if it is the player’s turn.

*Creating the Constructor:**Player():*

```
public Player ()
{
    this.playerMoney = 500;
    this.totalBet = 0;
    this.playerLost = false;
    this.currentHand = 0;
    this.handsList = new LinkedList<Hand>();
    this.lastHand = 0;
    this.isTurn = false;
}
```

I have created a constructor which instantiates the player object, it sets **playerMoney** to 500 as I believe that is a good starting amount as they will then be able to play several rounds regardless of if they lose if they don't bet loads of money.

It also sets **totalBet** to 0 as the player has not bet yet, **playerLost** to false as they also won't have lost yet. **currentHand** and **lastHand** are also set to 0, this is because they player will only have one hand to begin with and it will be at index position 0. Finally, it sets **isTurn** to false, as it is not the players turn when the game begins, it is a betting round first.

The constructor also instantiates a new **handsList** Linked List to store the player's hand(s) in.

Defining Methods:

Getters & Setters:

<pre>public int getLastHand (){ return(this.lastHand); } public void setLastHand (int value){ this.lastHand = value; } public Boolean getPlayerTurn (){ return(this.isTurn); } public void setPlayerTurn (Boolean value){ this.isTurn = value; }</pre>	<pre>public Hand getPlayerHand (int index){ if (!this.handsList.isEmpty()){ return (this.handsList.get(index)); } return(null); } public void addToHandList (Hand hand){ this.handsList.add(hand); } public int getCurrentHand(){ return this.currentHand; } public int getPlayerMoney(){ return this.playerMoney; }</pre>
---	---

For more detail on the attributes see ‘Coding the Class – ‘Hand’, ‘Defining Variables and Attributes’)

getLastHand():

I have created a getter method that returns the Integer value of the index of the player’s last hand.

setLastHand():

I have created a setter method that sets the Integer value of the index of the player’s last hand to a passed in value.

getPlayerTurn():

I have created a getter method that returns the Boolean value of if it is the player’s turn.

setPlayerTurn():

I have created a getter method that sets the Boolean value of if it is the player’s turn.

getPlayerHand():

I have created a getter method that returns the hand in the **handsList** LinkedList at a passed in index if the LinkedList is not empty, otherwise it will return a null value.

addToHandList():

I have created a method that takes a hand object as a parameter and adds it to **handsList**.

getCurrentHand():

I have created a getter method that returns the Integer value of the index of the player’s hand that is currently in use.

getPlayerMoney():

I have created a getter method that returns the Integer value of the player’s total balance.

```
public int getTotalBet()
{
    return totalBet;
}

public void addToTotalBet(int value)
{
    this.totalBet = this.totalBet + value;
}

public void emptyBet()
{
    this.totalBet = 0;
}

public void addPlayerMoney(int value)
{
    this.playerMoney = this.playerMoney + value;
}

public void subPlayerMoney(int value)
{
    this.playerMoney = this.playerMoney - value;
}
```

getTotalBet():

I have created a getter method that returns the Integer value of the player's total bet.

addToTotalBet():

I have created a setter method that adds a passed in value to the players total bet and updates the Integer value of the player's total bet to the new total.

emptyBet():

I have created a setter method sets the player's total bet to 0.

addPlayerMoney():

I have created a setter method that adds a passed in value to the players total balance and updates the Integer value of the player's total balance to the new total.

subPlayerMoney():

I have created a setter method that subtracts a passed in value from the players total balance and updates the Integer value of the player's total balance to the new total.

Extra Code:

In the MainGame World class:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class MainGame extends World
{
    DeckOfCards deck = new DeckOfCards();

    Player player = new Player();
    Hand hand1 = new Hand();

    public Player getPlayer()
    {
        return player;
    }
}
```

I have instantiated a player and hand object from the Player and Hand classes in my MainGame class so I can run its methods in my game.

I have also created a method, `getPlayer()`, which returns the Player object. This is so the player can be accessed from other classes such as the poker chip class which will need to access the player to adjust their total bet.

Testing -----

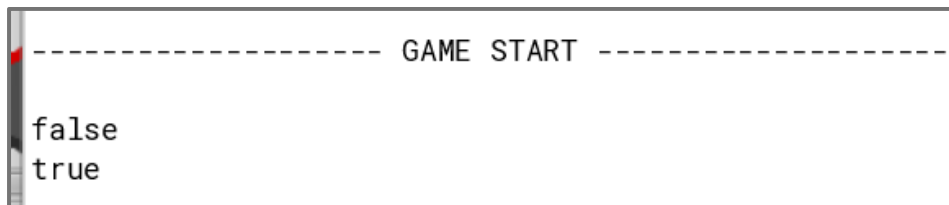
Test 2a & b – Testing the **getPlayerTurn()** & **setPlayerTurn()** methods:

Expected Result: The Player getter method returns the correct value, and the setter method updates the attribute correctly.

To test this, I have written some code that will output whether it is the Player's turn (which should be false when the game begins), it then sets the player's turn to true, and then outputs the player's turn again (which should be true):

```
System.out.println(player.getPlayerTurn());  
player.setPlayerTurn(true);  
System.out.println(player.getPlayerTurn());
```

Result (Passed):



```
----- GAME START -----  
false  
true
```

The program outputs the correct values for player turn before and after the setter method. This means the two methods have passed the test correctly.

Test 2c – Testing the `getHandTotal()` method:

Expected Result: The Hand getter method returns the correct value.

*To test this, I have written some code that will output the hand's total, which should be zero as it has not run the method **starthand()** yet:*

```
System.out.println("Hand total is: " + hand1.getHandTotal());
```

Result (Passed):

```
----- GAME START -----  
  
Hand total is: 0
```

The program outputs the correct values of zero for the hand total.

Dealer Class (Stage 3)

Design -----

Flowchart:

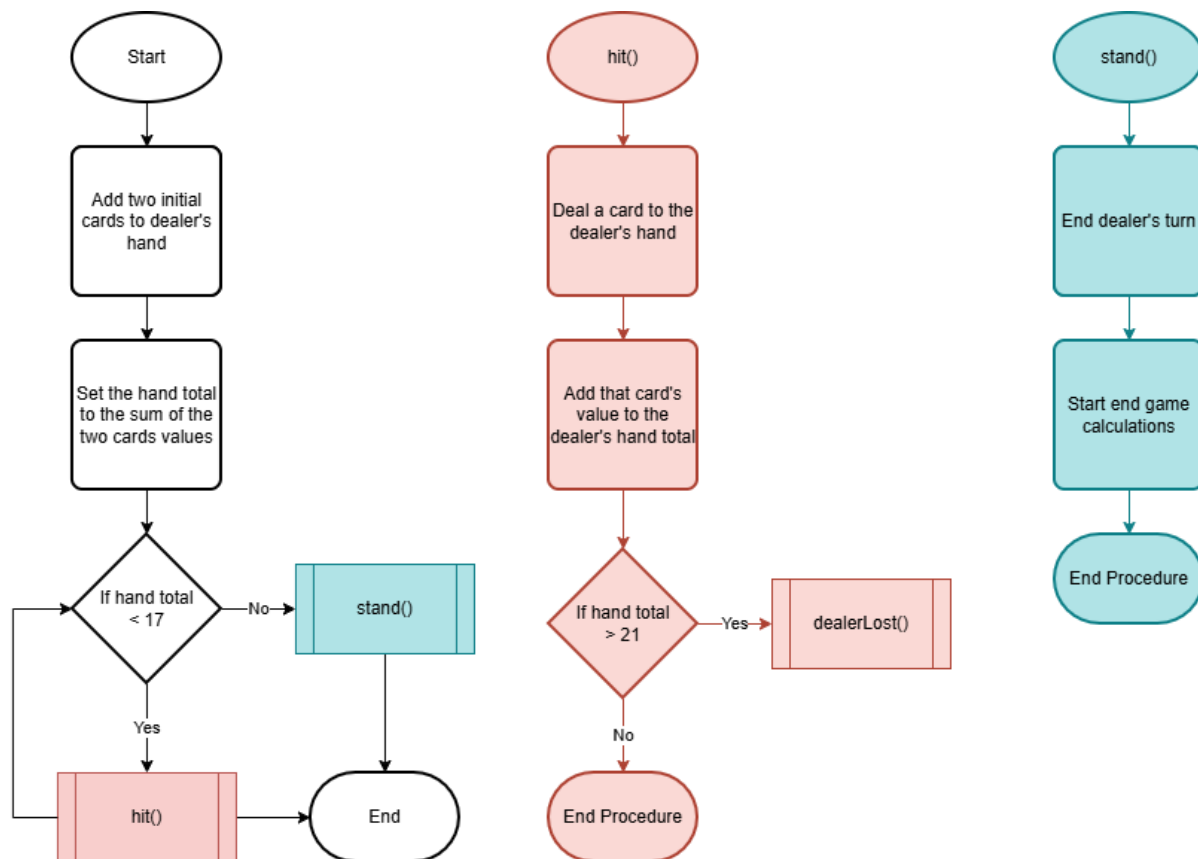


Figure 10 – 'Dealer Turn – Flowchart'

I have made this flowchart as a rough overview of how the Dealer's turn will work in my game. I have coloured the flowchart for easier viewing, with subroutines and related code different colours to each other.

Data Dictionary:

Name	Data type	Description	Validation
dealerLost	Boolean	Stores whether the dealer has gone bust.	'true' or 'false'
handTotal	Integer	Stores the total value of the cards in the dealer's hand.	≥ 0
card1Value	Integer	Stores the value of the first card dealt to the hand.	≥ 0
card2Value	Integer	Stores the value of the second card dealt to the hand.	≥ 0
isTurn	Boolean	Stores whether it is the dealer's hand.	'true' or 'false'
dealerHand	ArrayList	Stores the dealer's card objects.	N / A

Development -----

Coding the Class – ‘Dealer’:

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
import java.util.ArrayList; // For Array lists

public class Dealer extends Actor
{
    private boolean dealerLost;
    private boolean isTurn;
    private boolean handLost;
    private boolean blackjack;
    private ArrayList<Card> dealerHand; // An ArrayList for the card objects in each hand.
    private int card1Value;
    private int card2Value;
    private int handTotal;
    private int lastPos;
    private int winnings;
```

For the **Dealer** class, I have created 10 variables:

dealerLost – Used to store a Boolean value of whether the dealer has lost the game.

isTurn – Used to store a Boolean value of whether it is the dealers turn.

handLost – Used to store a Boolean value of whether the dealer has gone bust.

blackjack – Used to store a Boolean value of whether the dealer has gotten blackjack.

I have created an Array List, **dealerHand**, this will store the card objects that are in the dealer’s hand.

card1Value – Used to store an integer value of the *first* card dealt to the hand.

card2Value – Used to store an integer value of the *second* card dealt to the hand.

handTotal – Used to store an integer value of the dealer’s hand’s total value.

lastPos – Used to store an integer value of the x coordinates of where the last dealer’s card was placed.

winnings – Used to store an integer value of how much the player has won. This is stored in the dealer class as the dealer will have a method to check the outcome of the game.

Creating the Constructor:

Dealer():

```
public Dealer () {
    this.dealerLost = false;
    this.isTurn = false;
    this.handLost = false;
    this.blackjack = false;
    this.dealerHand = new ArrayList<Card>();

    World world = getWorld();
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();

    Card card1 = deck.dealCard();
    this.dealerHand.add(card1);
    this.card1Value = card1.getCardValue();

    Card card2 = deck.dealCard();
    this.dealerHand.add(card2);
    this.card2Value = card2.getCardValue();

    this.handTotal = card1Value + card2Value;
}
```

This is my original Dealer constructor code. My game had Null Pointer Exception for the dealer, this is because the constructor was trying to call a world method before the world had been created.

I fixed this by creating a new method separate to the constructor that has the world object passed into it (see **startDealer(World world)** method).

This is the new updated code (see below). This has fixed the Null Pointer Exception error as it does not attempt to call any world methods before it has been created.

```
public Dealer () {
    this.dealerLost = false;
    this.isTurn = false;
    this.handLost = false;
    this.blackjack = false;
    this.lastPos = 625;
    this.dealerHand = new ArrayList<Card>();
    this.winnings = 0;
}
```

I have created a constructor for the dealer, this sets the attributes **dealerLost**, **isTurn**, **handLost**, and **blackjack** to false and **winnings** to 0 as these are the values the attributes should be when the game starts. It also sets **lastPos** to 625 as this is where the first card will be dealt. Finally, it creates a new ArrayList, **dealerHand**, to store the dealer's cards.

For more detail on the attributes, see previous page 'defining variables'.

Defining Methods:

startDealer():

```
public void startDealer(World world){
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();

    Card card1 = deck.dealCard();
    this.dealerHand.add(card1);
    this.card1Value = card1.getCardValue();
    mainGame.addObject(card1, 710, 120);

    Card card2 = deck.dealCard();
    card2.faceDown();
    this.dealerHand.add(card2);
    this.card2Value = card2.getCardValue();
    mainGame.addObject(card2, 625, 120);

    this.handTotal = card1Value;
}
```

I have made a method for ‘starting’ the dealer. This will be run after the player’s turn. As explained on the previous page, I have moved a lot of code that was originally in the constructor to this method (startDealer) as it caused null pointer exception errors. Again, doing this fixed it.

This method also sets the dealer’s second card’s **facedown** value to true as until it is the dealer’s turn, so no one can see what the value is (as this is a rule in Blackjack). I have also made sure the dealer’s total is only the first (face up) card to begin with.

act():

```
public void act()
{
    // Add your action code here.
    if (handTotal > 21){
        this.handLost = true;
    }

    if (handTotal == 21){
        this.blackjack = true;
    }
    else{
        this.blackjack = false;
    }
}
```

I have modified the Dealer class's act method (repeatedly executed while the program is being run) to check for if the dealer has gotten blackjack or has lost and update the respective attributes.

hit():

```
public void hit(World world){
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();

    Card newCard = deck.dealCard();
    this.dealerHand.add(newCard);
    this.handTotal = this.handTotal + newCard.getCardValue();
    mainGame.addObject(newCard, this.lastPos - 85, 120);
    lastPos = lastPos - 85;
}
```

I have created a hit method for the dealer. This deals a card using the DeckOfCards method to the **dealerHand** ArrayList. It then updates the **handTotal** attribute. Finally, it places the object in the world at the correct position using the **lastPos** attribute.

stand():

```
public void stand()
{
    this.isTurn = false;
}
```

I have created a method that allows the dealer to stand, this sets the attribute **isTurn** to false.

revealCard():

```
public void revealCard () {
    System.out.println("dealer card face status before = ");
    for (int i = 0; i <= ( dealerHand.size() - 1 ); i++){
        System.out.println("Card index " + i + " Face Down = " + this.dealerHand.get(i).getFaceDown());
    }
    this.dealerHand.get(1).faceUp();
    this.handTotal = this.handTotal + dealerHand.get(1).getCardValue();
    System.out.println("dealer card face status after = " + this.blackjack);
    for (int i = 0; i <= ( dealerHand.size() - 1 ); i++){
        System.out.println("Card index " + i + " Face Down = " + this.dealerHand.get(i).getFaceDown());
    }
}
```

As the dealer's second card is face down to begin with, I have made this method to change the card's **faceDown** attribute to false (was true), which will change the image for the card to be the face up image. This will also output logs to the terminal so I can test it is working correctly.

lost():

```
public void lost (World world){
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();

    this.winnings = 2 * player.getTotalBet();
    this.handLost = true;
    System.out.println("Player wins, the dealer went bust with a hand total of: " + this.handTotal);

    for (int i = 0; i <= player.getLastHand(); i++){
        //CHECK FOR PLAYER IF THEY HAVE BUSTED THEN THEY LOSE BET NOT WIN IT, CHECK FOR EACH HAND
        if ( (player.getPlayerHand(i).getHandLost() == false) ){
            player.addPlayerMoney(this.winnings);
            player.emptyBet();
        }
        else{
            player.emptyBet();
        }
    }
}
```

I have made this method to be called when the dealer has lost (gone bust). This method calculates the winnings for the player, sets the dealer's **handLost** attribute to false, outputs to the terminal that the dealer went bust and the dealer's total (for future testing to check it was meant to go bust).

It then checks if they player has gone bust. If they haven't, it pays them their winnings and empties their bet. If they have gone bust, it empties their bet and does *not* pay them.

dealerTurn() – 1/3:

```
public void dealerTurn(World world)
{
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();
    revealCard();
    if ( !player.getPlayerHand(0).getHandLost() ){
        while (getHandTotal() < 17){
            System.out.println("*Dealer has less than 17, dealer has " + getHandTotal() + ", hits*");
            hit(mainGame);
        }
        if (getHandTotal() > 21){
            System.out.println("*Dealer went bust, dealer stands* " + getHandTotal());
            //What if player also goes bust
            lost(mainGame);
        }
    }
}
```

I have made a method for the dealer's turn. In this part of the code, it runs the **revealCard** method. After that, if the player hasn't gone bust and the dealer's hand total is less than 17, it will run the hit method until the **handTotal** attribute is 17 or over. Then if the dealer's hand total is over 21 (gone bust), it will run the **lost** method and output logging to the terminal that it has gone bust and output its hand total.

However, if the player has already gone bust, then the dealer won't draw any extra cards and moves onto to the next piece of code (see *dealerTurn()* – 3/3).

For outputting the logging, I originally tried to use commas instead of plus symbols to concatenate the text to the values, this did not work.

To figure out what the issue was, I researched outputting text in Java. By reading through a website I found, I realised I needed to use a plus symbol to concatenate the string for printing it. This is the website I used:

<https://medium.com/javarevisited/java-output-a-beginners-guide-to-printing-text-7aad93774110>

dealerTurn() – 2/3:

```
else{
    System.out.println("Dealer hasn't gone bust, dealer stands on " + getHandTotal());
    stand();
    System.out.println();
    for (int i = 0; i <= player.getLastHand(); i++){
        // IF THE PLAYER'S HAND HAS ALREADY GONE BUST (DEALER HAND WINS)
        if ( player.getPlayerHand(i).getHandLost() ){
            System.out.println("Player's hand " + (i + 1) + " has lost, it went bust.");
        }

        // IF THE PLAYER'S HAND TOTAL IS LESS THAN DEALER'S (DEALER HAND WINS)
        else if ( (player.getPlayerHand(i).getHandTotal() < getHandTotal())
        || player.getPlayerHand(i).getHandLost() ){
            System.out.println("Player's hand " + (i + 1) + " has lost, the dealer beat it.");
        }

        // IF THE PLAYER'S HAND TOTAL IS MORE THAN DEALER'S (PLAYER HAND WINS)
        else if ( (player.getPlayerHand(i).getHandLost() == false)
        && player.getPlayerHand(i).getHandTotal() > getHandTotal() ){
            System.out.println("Player's hand " + (i + 1) + " has won, the dealer hand had less.");
            this.winnings = 2 * player.getTotalBet();
            player.addPlayerMoney(winnings);
        }

        // IF THE PLAYER'S HAND TOTAL IS EQUAL TO THE DEALER'S (DRAW)
        else if ( player.getPlayerHand(i).getHandTotal() == getHandTotal() ){
            System.out.println("Player's hand " + (i + 1) + " is equal to the dealer's hand.");
            player.addPlayerMoney(player.getTotalBet());
        }
    }
}
```

In this code, the dealer will check who has won, lost, and drawn. It will then run the respective code to make sure the player is paid out if needed or not paid out if the dealer has one. It then outputs logging to the terminal the outcome of the game so I can test if it has calculated it correctly.

dealerTurn() – 3/3:

```
// IF THE PLAYER GETS BLACKJACK (21) THEN THEY WIN
if ( player.getPlayerHand(i).getBlackjack()){
    System.out.println("BLACKJACK!! The Player's hand " + (i + 1) + " has won. Dealer had " + getHandTotal());
    this.winnings = player.getTotalBet() + (int)Math.floor(1.5 * player.getTotalBet());
    player.addPlayerMoney(winnings);
    //REFERENCE INT MATH.FLOOR
}
}

else{
    System.out.println("Player's hand " + i + " has lost, it went bust.");
}

this.isTurn = false;
mainGame.displayEndGameInfo();
System.out.println();
System.out.println("----- GAME OVER -----");
System.out.println();
```

If the player has gone bust during their turn, it will output logging to the terminal that the player's hand has gone bust.

The end of the method runs regardless of what happens. The dealer's attribute **isTurn** is set to false, and it runs a method in the mainGame class which I have not coded yet. This method will display the outcome of the game for the user such as if they have won, lost, or drawn, and how much money they won, lost, or kept.

It also outputs to the terminal some spacers and a "GAME OVER" message to make it clear in the logging which outputs are for the current game or previous games.

Getters & Setters:

```
public int getHandTotal (){
    return(this.handTotal);
}

public boolean getDealerTurn (){
    return(this.isTurn);
}

public int getWinnings (){
    return(this.winnings);
}

public void setDealerTurn (Boolean value)
{
    this.isTurn = value;
}

public boolean checkDealerLost(){
    return (this.handLost);
}
```

(For more detail on the attributes see 'Coding the Class – Dealer, 'Defining Variable and Attributes')

getHandTotal():

I have created a getter method that returns the Integer value of the Dealer's hand total.

getDealerTurn():

I have created a getter method that returns the Boolean value of the Dealer's attribute **isTurn**.

getWinnings():

I have created a getter method that returns the Integer value of the player's total winnings, stored in the dealer class.

setDealerTurn():

I have created a setter method that sets the attribute **isTurn** to a passed in Boolean value.

checkDealerLost():

I have created a getter method that returns the Boolean value of the Dealer's attribute **handLost**.

Extra Code:

In the MainGame World class:

```
public class MainGame extends World
{
    DeckOfCards deck = new DeckOfCards();
    Dealer dealer = new Dealer();
    Player player = new Player();

    public Dealer getDealer()
    {
        return dealer;
    }
}
```

I have instantiated a dealer object from the Dealer class in my MainGame class so I can run its methods in my game.

I have also created a method, `getDealer()`, which returns the Dealer object. This is so the dealer can be accessed from other classes such as the Poker Chip class (when I code it) as it will need to know if it is still the dealer's turn or not.

Testing -----

Test 3a & b – Testing the **getHandTotal()** & **startDealer()** methods:

Expected Result: Dealer's getter and startDealer methods should work correctly at returning and adjusting values.

*To test this, I have written some code in the MainGame class that will output the dealer's hand total at the beginning of the program (see first image), I have also added the same code at the end of the dealer method, **startDealer()** (see second image) so I can see the values before and after **startDealer()** is run:*

```
System.out.println("Dealer hand total is: " + dealer.getHandTotal());
```

```
public void startDealer(World world){
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();

    Card card1 = deck.dealCard();
    this.dealerHand.add(card1);
    this.card1Value = card1.getCardValue();
    mainGame.addObject(card1, 710, 120);

    Card card2 = deck.dealCard();
    card2.faceDown();
    this.dealerHand.add(card2);
    this.card2Value = card2.getCardValue();
    mainGame.addObject(card2, 625, 120);

    this.handTotal = card1Value;

    System.out.println("Dealer hand total is: " + getHandTotal());
}
```

Result (Passed):

```
----- GAME START -----
Dealer hand total is: 0
```

```
Dealer hand total is: 4
```

The first output is the dealer's hand when the program starts, this is correct as the dealer has not been dealt any cards yet.

The second output is after the **startDealer()** method has been run, this is correct because it should be between 0 and 11 (as it has been dealt only one card).

Test 3c – Testing the `getDealerTurn()` method:

Expected Result: Getter method should return the correct value; this is necessary for the game to function.

*To test this, I have written some code in the MainGame class, this will output the current value of the dealer's **isTurn** attribute at the beginning of the game, this should be false:*

```
System.out.println("The dealer isTurn attribute is currently: " + dealer.getDealerTurn());
```

Result (Passed):

```
----- GAME START -----  
The dealer isTurn attribute is currently: false
```

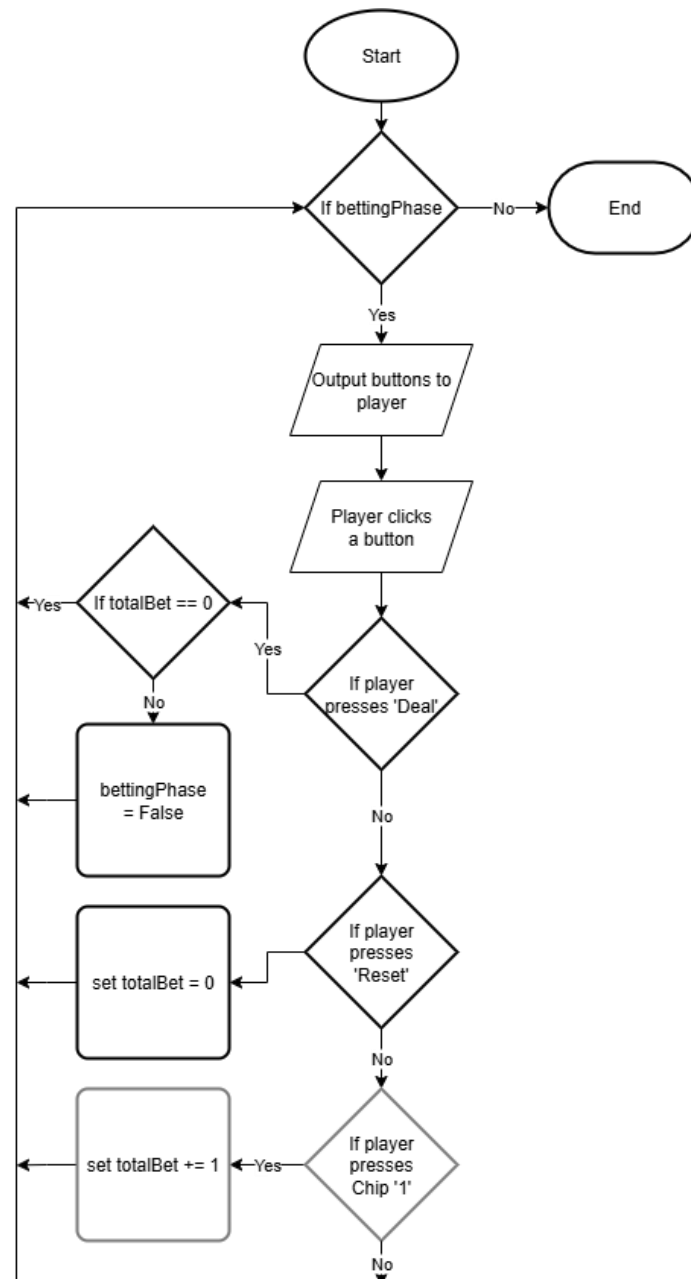
The program has output that the attribute is false. This value is correct as it should be false at the start of the game.

If it were true at the beginning of the game, the program would crash as the dealer would try to have their turn and run code, which checks for the outcome of the player's turn, and wouldn't be able to find any needed values.

Betting Chips (Stage 4)

Design

Flowchart:



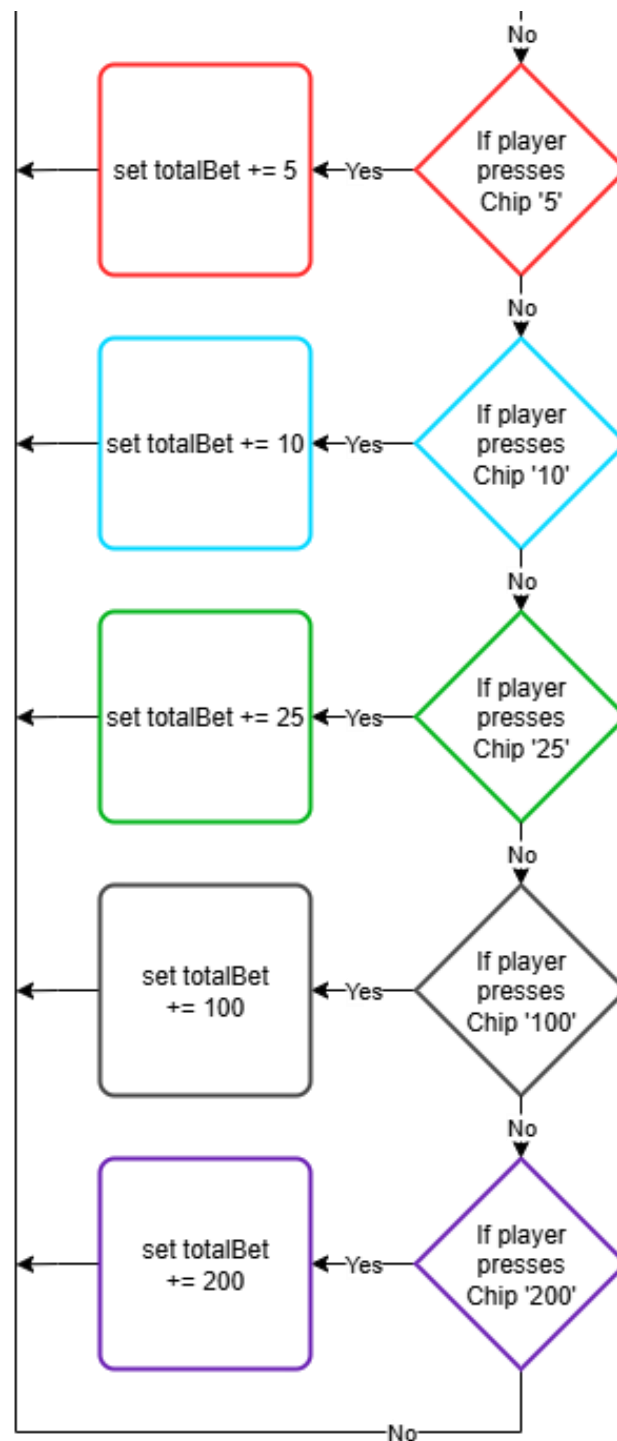


Figure 11 - 'Betting Chips - Flowchart'

I have made this flowchart to summarise how the poker chips betting system will work. I have also coloured this flowchart for easier viewing. The player can click on 6 different poker chips, depending on what one(s) they bet, they will bet the corresponding amount of money.

The player can also press 'Reset' which resets their bet, and 'Deal' which will start the game with the amount of money they have bet, if they have bet an amount more than zero.

Data Dictionary:

Name	Data type	Description	Validation
myValue	Integer	Stores the value of the poker chip	1, 5, 10, 25, 100, or 200
pressed()	void Method	Updates the player's total bet and balance.	Method runs correctly

Development -----

Coding the Parent Class – ‘PokerChips’:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChips extends Actor
{
    public void act()
    {
        // Add your action code here.
    }

    public void setChipSize(){
        GreenfootImage image = getImage();
        image.scale(60, 60);
        setImage(image);
    }
}
```

This parent class has the method **setChipSize()**. This method takes the object's image, scales it to be 60 by 60 pixels, and then updates its image to this new image.

This method will be inherited by the subclasses for each poker chip (e.g.: 1, 5, 25, 100)

Coding the Subclasses:

PokerChip1, PokerChip5, PokerChip10, PokerChip25, PokerChip100, PokerChip200

Defining Variables:

(Example Class – *PokerChip*):

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChip1 extends PokerChips
{
    private int myValue;
```

For the subclasses, they have one attribute which is '**myValue**' which will be used to add to the player's bet when they are clicked on.

In each of their constructor classes they set this attribute to their corresponding value, e.g.: PokerChip1's would be 1, PokerChip25's would be 25.

The constructor also runs the **setChipSize()** method inherited from the PokerChips parent class which sets the poker chip's image to the correct size (see *Coding the Parent Class – 'PokerChips'*).

Defining Methods:

(Example Class – *PokerChip*):

```
public void act()
{
    if (Greenfoot.mouseClicked(this)){
        this.pressed();
    }
}
```

act():

I have modified each of the PokerChips subclasses' act methods (repeatedly executed while the program is being run) to check for if the user has clicked on them.

It does this by using an IF statement and a built in Greenfoot method **mouseClicked()** which detects when a mouse has clicked on the object passed in. For this code I passed in each poker chip object, so they detect if they have been clicked.

If they have been clicked, then it will run the **pressed()** method.

As I originally could not remember how to use the **mouseClicked()** method, I used the built in Greenfoot documentation to remind myself.

(Example Class – *PokerChip*):

```
public void pressed()
{
    World world = getWorld();
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();
    Dealer dealer = mainGame.getDealer();

    if ((player.getPlayerMoney() >= this.myValue) && (player.getPlayerTurn() == false)
    && (dealer.getDealerTurn() == false) ){
        player.subPlayerMoney(this.myValue);
        player.addToTotalBet(this.myValue);
        System.out.println("Updated Player Money and Total Bet");
    }
    else{
        System.out.println("Insufficient Funds");
    }
}
```

pressed():

Each of the subclasses have a method called **pressed()**. This method checks three conditions before it takes its value away from the player's money and adds it to the player's total bet, it then outputs logging to the terminal that it has updated the player's money and total bet.

- The first condition it checks is that the player has enough money to bet. It checks this by comparing the player's total money to the value of the poker chip they are trying to bet. If the player has more or an equal amount of money as the poker chips' value, then they pass this condition.
- The second condition is that it is not the player's turn yet (they are not current playing the game with the option's **hit**, **stand**, **double**, **split**). This is checked with the player method **getPlayerTurn**.
- The third condition is that it is not the dealer's turn either as this would mean the game is still in play. This is checked with the dealer's method **getDealerTurn**.

If all three of these conditions are met, then it will update the players total money and bet. However, if one or more of these are not met, then the method won't do anything and output logging to the terminal: "Insufficient Funds".

Full Classes' Code

PokerChip1:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChip1 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if ((player.getPlayerMoney() >= this.myValue) && (player.getPlayerTurn() == false)
        && (dealer.getDealerTurn() == false) ){
            player.subPlayerMoney(this.myValue);
            player.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }

    public PokerChip1 ()
    {
        this.myValue = 1;
        setChipSize();
    }
}
```

PokerChip5:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)
```

```
public class PokerChip5 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if (player.getPlayerMoney() >= this.myValue && (player.getPlayerTurn() == false) && (dealer.getDealerTurn() == false) ){
            player.subPlayerMoney(this.myValue);
            player.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }

    public PokerChip5 ()
    {
        this.myValue = 5;
        setChipSize();
    }
}
```

PokerChip10:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChip10 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if (player.getPlayerMoney() >= this.myValue && (player.getPlayerTurn() == false) && (dealer.getDealerTurn() == false) ){
            player.subPlayerMoney(this.myValue);
            player.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }

    public PokerChip10 ()
    {
        this.myValue = 10;
        setChipSize();
    }
}
```

PokerChip25:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChip25 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if (player.getPlayerMoney() >= this.myValue && (player.getPlayerTurn() == false) && (dealer.getDealerTurn() == false) ){
            player.subPlayerMoney(this.myValue);
            player.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }

    public PokerChip25 ()
    {
        this.myValue = 25;
        setChipSize();
    }
}
```

PokerChip100:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChip100 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if (player.getPlayerMoney() >= this.myValue && (player.getPlayerTurn() == false) && (dealer.getDealerTurn() == false)) {
            player.subPlayerMoney(this.myValue);
            player.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }

    public PokerChip100 ()
    {
        this.myValue = 100;
        setChipSize();
    }
}
```

PokerChip200:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChip200 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

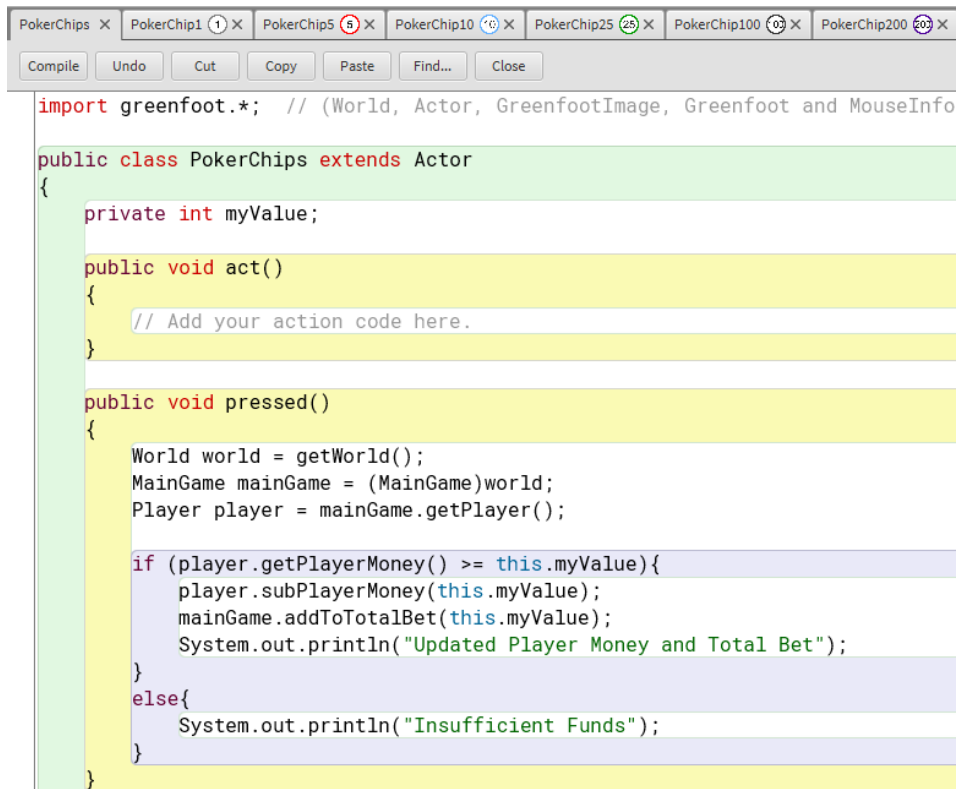
    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if (player.getPlayerMoney() >= this.myValue && (player.getPlayerTurn() == false) && (dealer.getDealerTurn() == false) ){
            player.subPlayerMoney(this.myValue);
            player.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }

    public PokerChip200 ()
    {
        this.myValue = 200;
        setChipSize();
    }
}
```


Bugs:

Pressed() Method Bug:



```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class PokerChips extends Actor
{
    private int myValue;

    public void act()
    {
        // Add your action code here.
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();

        if (player.getPlayerMoney() >= this.myValue){
            player.subPlayerMoney(this.myValue);
            mainGame.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }
}
```

I originally tried to have the **pressed()** method be defined in the PokerChips parent class and inherited in the subclasses. However, it wouldn't work as the subclasses were trying to access the parent class's attribute, **myValue**, rather than their own.

```
public class PokerChip1 extends PokerChips
{
    private int myValue;

    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();

        if (player.getPlayerMoney() >= this.myValue){
            player.subPlayerMoney(this.myValue);
            mainGame.addToTotalBet(this.myValue);
            System.out.println("Updated Player Money and Total Bet");
        }
        else{
            System.out.println("Insufficient Funds");
        }
    }
}
```

By moving the **pressed()** method into each of the subclass' code, it meant that they would use their own attributes rather than the parent class's. This fixed the bug.

Testing -----

Extra Information:

For testing the Poker Chip Classes, I have only tested three as all 6 poker chips code is almost identical with the only difference being a change in an attribute's value.

Test 4a – Testing PokerChip1's **pressed()** method:

Expected Result: Player's balance decreases by 1 and total bet increases by 1.

To test this, I have written some code in the MainGame class that checks the player's balance and total bet before and after attempting to bet a £1 chip:

```
System.out.println("Balance: " + player.getPlayerMoney());  
System.out.println("Total Bet: " + player.getTotalBet());  
  
PokerChip1 chip1 = new PokerChip1();  
chip1.pressed();  
  
System.out.println("Balance: " + player.getPlayerMoney());  
System.out.println("Total Bet: " + player.getTotalBet());
```

Result (Passed):

Before:	Balance: 500 Total Bet: 0	After:	Balance: 499 Total Bet: 1
---------	--	--------	--

The player's balance and total bet attributes successfully updated after running the press method.

Test 4b – Testing PokerChip25's **pressed()** method:

Expected Result: Player's balance decreases by 25 and total bet increases by 25.

To test this, I have written some code in the MainGame class that checks the player's balance and total bet before and after attempting to bet a £25 chip:

```
System.out.println("Balance: " + player.getPlayerMoney());  
System.out.println("Total Bet: " + player.getTotalBet());  
  
PokerChip25 chip25 = new PokerChip25();  
chip25.pressed();  
  
System.out.println("Balance: " + player.getPlayerMoney());  
System.out.println("Total Bet: " + player.getTotalBet());
```

Result (Passed):

Before:	Balance: 500 Total Bet: 0	After:	Balance: 475 Total Bet: 25
---------	--	--------	---

The player's balance and total bet attributes successfully updated after running the press method.

Test 4c – Testing PokerChip200's **pressed()** method:

Expected Result: Player's balance decreases by 200 and total bet increases by 200.

To test this, I have written some code in the MainGame class that checks the player's balance and total bet before and after attempting to bet a £200 chip:

```
System.out.println("Balance: " + player.getPlayerMoney());  
System.out.println("Total Bet: " + player.getTotalBet());  
  
PokerChip200 chip200 = new PokerChip200();  
chip200.pressed();  
  
System.out.println("Balance: " + player.getPlayerMoney());  
System.out.println("Total Bet: " + player.getTotalBet());
```

Result (Passed):

Before:	Balance: 500 Total Bet: 0	After:	Balance: 300 Total Bet: 200
---------	--	--------	--

The player's balance and total bet attributes successfully updated after running the press method.

Main Game Class (Stage 5)

Design -----

Flowchart:

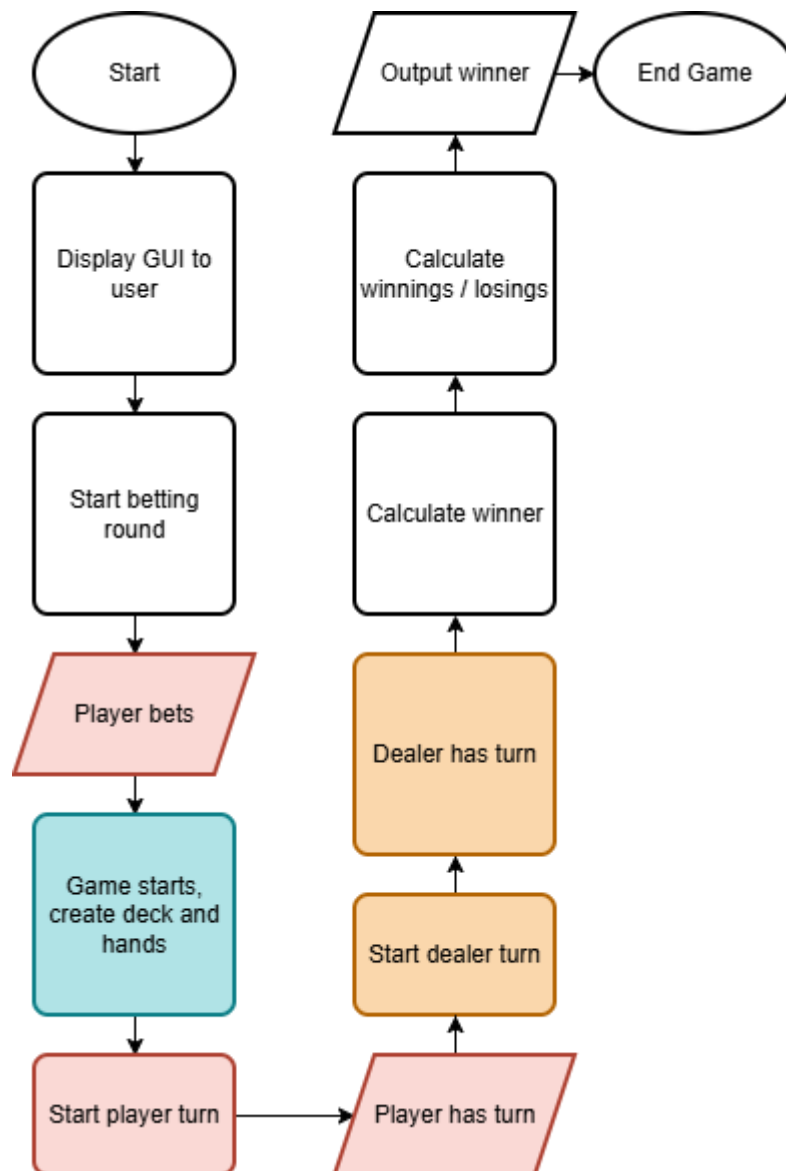


Figure 12 - 'Main Game - Flowchart'

I have made this flowchart as a rough overview of how the Dealer's turn will work in my game. I have coloured the flowchart for easier viewing, with subroutines and related code different colours to each other.

Data Dictionary:

Name	Object	Description	Validation
deck	DeckOfCards	The DeckOfCards object that will be used in the game.	N / A
dealer	Dealer	The Dealer object that will be used in the game.	N / A
player	Player	The Player object that will be used in the game.	N / A
hand1	Hand	The Hand object that will be used in the game.	N / A
chip1	PokerChip1	The PokerChip1 object that will be used in the game.	N / A
chip5	PokerChip5	The PokerChip5 object that will be used in the game.	N / A
chip10	PokerChip10	The PokerChip10 object that will be used in the game.	N / A
chip25	PokerChip25	The PokerChip25 object that will be used in the game.	N / A
chip100	PokerChip100	The PokerChip100 object that will be used in the game.	N / A
chip200	PokerChip200	The PokerChip200 object that will be used in the game.	N / A

Development -----

Coding the Class – ‘MainGame’:

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class MainGame extends World
{
    DeckOfCards deck = new DeckOfCards();
    Dealer dealer = new Dealer();
    Player player = new Player();
    Hand hand1 = new Hand();
}
```

At the beginning of the MainGame class, I have instantiated:

- A DeckOfCards object ‘deck’
- A Dealer object ‘dealer’
- A Player object ‘player’
- A Hand object ‘hand1’

These are so their methods can be run in the program.

Defining Methods:

Getters:

```
public DeckOfCards getDeck()
{
    return deck;
}

public Player getPlayer()
{
    return player;
}

public Dealer getDealer()
{
    return dealer;
}
```

I have made getter methods for the deck, player, and dealer objects. This is so their methods can be accessed from other classes.

playerTurn():

```
public void playerTurn(){
    player.setPlayerTurn(true);
}
```

I have made a method, **playerTurn**, which uses a player setter method to set the player's turn to true.

dealerTurn():

```
public void dealerTurn(){
    player.setPlayerTurn(false);
    dealer.setDealerTurn(true);
}
```

I have made a method, **dealerTurn**, which uses a player setter method to set the player's turn to false, and a dealer setter method to set the dealer's turn to true.

gameStart():

```
public void gameStart(){
    deck.shuffleDeck();
    dealer.startDealer(this);
    hand1.startHand(this);
    playerTurn();
}
```

I have made a **gameStart()** method that will be run to begin the game.

In this method it starts by running the **shuffleDeck()** method from the deck object, this makes sure the card objects are in a random order in the deck Array List ready to start.

Next it runs **startDealer()** and **startHand()** to ensure that the dealer and hand objects are ready for the game to start (*for more information see stage 2 and stage 3 for hand and dealer methods*).

Finally, it runs the **playerTurn()** method (*See previous page*).

Constructor - MainGame():

```
public MainGame()  
{  
    super(1200, 700, 1);  
    System.out.println();  
    System.out.println();  
    System.out.println("----- GAME START -----");  
    System.out.println();  
}
```

In the MainGame constructor, it creates the world and outputs some breaks and some text to the terminal, so it is easy and clear to see what logging outputs are from which game, whether it be a previous or current game.

```
// CHIPS -----  
  
PokerChip1 chip1 = new PokerChip1();  
addObject(chip1, 50, 560);  
  
PokerChip5 chip5 = new PokerChip5();  
addObject(chip5, 125, 560);  
  
PokerChip10 chip10 = new PokerChip10();  
addObject(chip10, 200, 560);  
  
PokerChip25 chip25 = new PokerChip25();  
addObject(chip25, 50, 645);  
  
PokerChip100 chip100 = new PokerChip100();  
addObject(chip100, 125, 645);  
  
PokerChip200 chip200 = new PokerChip200();  
addObject(chip200, 200, 645);
```

It also creates a poker chip object for each poker chip and adds them to the world at the correct coordinates.

Testing -----

Test 5a – Testing the MainGame **getters** methods:

Expected Result: No compilation errors in any code attempting to use these getters.

To test this, I will check that my written code does not show any errors:

```
public DeckOfCards getDeck()
{
    return deck;
}

public Player getPlayer()
{
    return player;
}

public Dealer getDealer()
{
    return dealer;
}
```

Result (Passed):

```
public void startHand(World world){
    MainGame mainGame = (MainGame)world;
    DeckOfCards deck = mainGame.getDeck();
    Player player = mainGame.getPlayer();

    player.addToHandList(this);

    Card card1 = deck.dealCard();
    this.cardsInHand.add(card1);
    this.card1Value = card1.getCardValue();
    mainGame.addObject(card1, 490, 430);

    Card card2 = deck.dealCard();
    this.cardsInHand.add(card2);
    this.card2Value = card2.getCardValue();
    mainGame.addObject(card2, 575, 430);

    this.handTotal = card1Value + card2Value;
}
```

```
public void pressed()
{
    World world = getWorld();
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();
    Dealer dealer = mainGame.getDealer();

    if ((player.getPlayerMoney() >= this.myValue) && (player.getPlayerTurn() == false)
    && (dealer.getDealerTurn() == false) ){
        player.subPlayerMoney(this.myValue);
        player.addToTotalBet(this.myValue);
        System.out.println("Updated Player Money and Total Bet");
    }
    else{
        System.out.println("Insufficient Funds");
    }
}
```

Both method's classes had this message when I went to compile the code:

Class compiled - no syntax errors

This shows that the MainGame class's getters have worked correctly as they are all tested between these two methods, the deck, player, and dealer objects.

Extra Information:

For testing the MainGame class, I cannot fully test all the code yet as it relies on the GUI to function, I will test the game's full functionality later in the project once I have coded and implemented the GUI.

Making the GUI (Stage 6)

Design -----

Main Game Background Image:

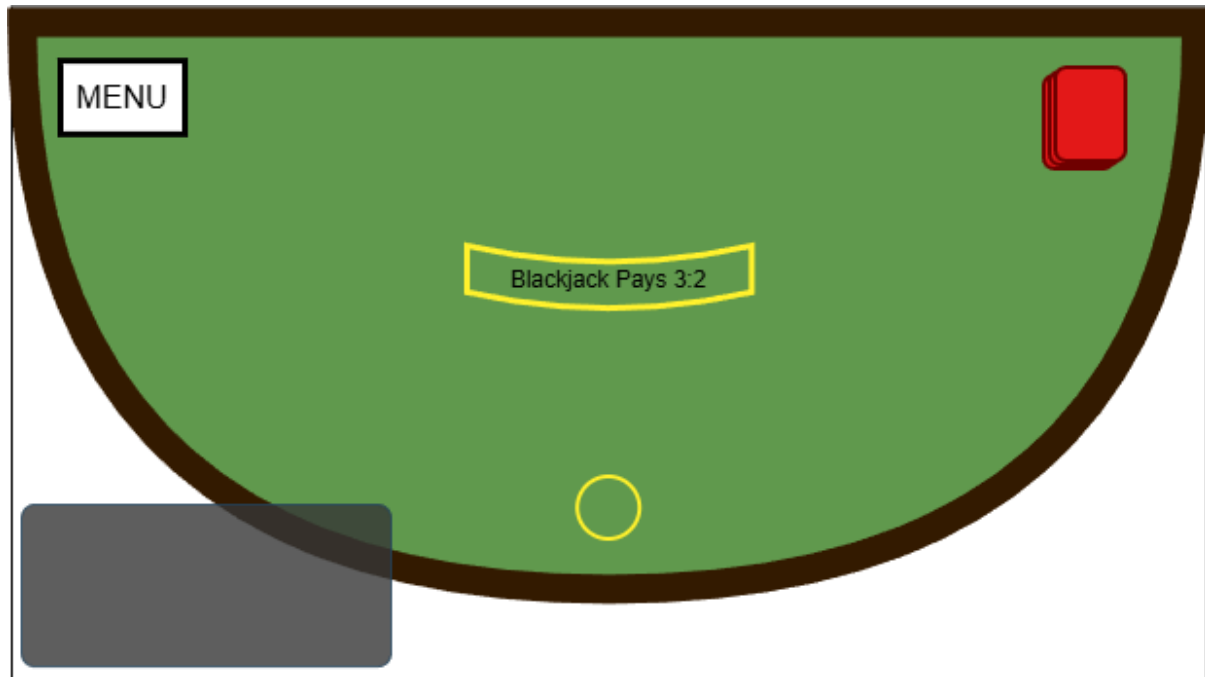


Figure 13 - 'MainGame Background'

For the main game background, I took my original concept photo (see *GUI design*) and removed all the interactable features as I will need to add them as objects with images later (excluding the MENU button as I will not be able to make a menu due to time constraints). I made this using draw.io, same as for my concept design.

What is left for the background is the:

- Plain white background.
- Green tabletop (and brown edges).
- Grey translucent box for the clickable poker chips and betting related buttons to go inside.
- Stacked deck of cards in the top right (this should be updated to the new card back design seen in stage 1, rather than my very plain simplistic design I came up with in my concept design).
- Rules on the table ("Blackjack Pays 3:2").
- Yellow circle for bet poker chip images to be placed inside of.

Playing Card Images:

The Card Back:

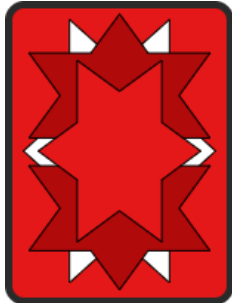
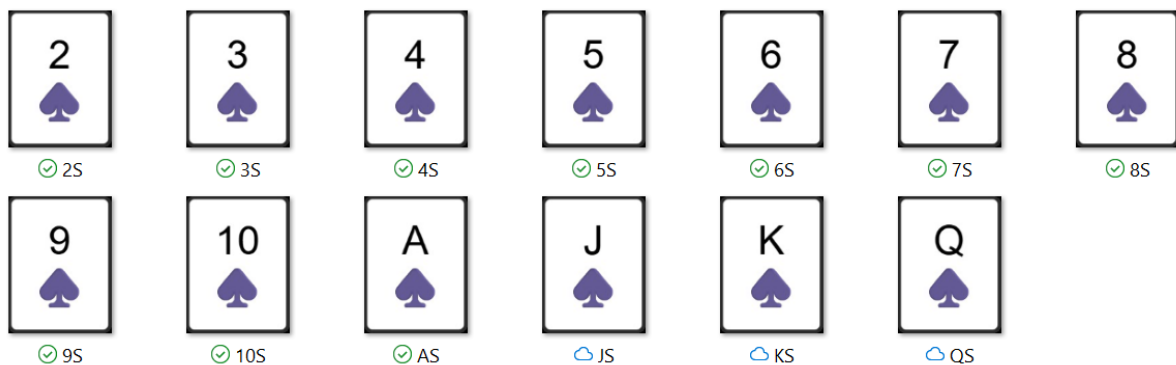


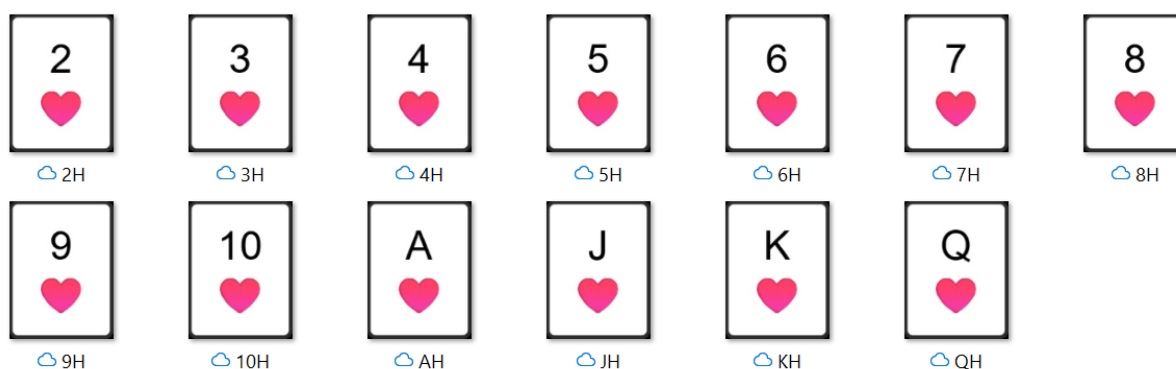
Figure 14 - 'CardBack.png'

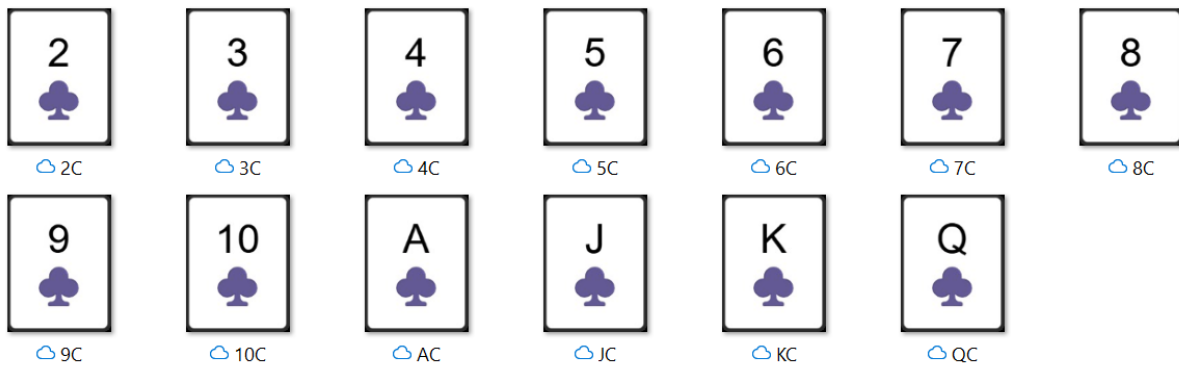
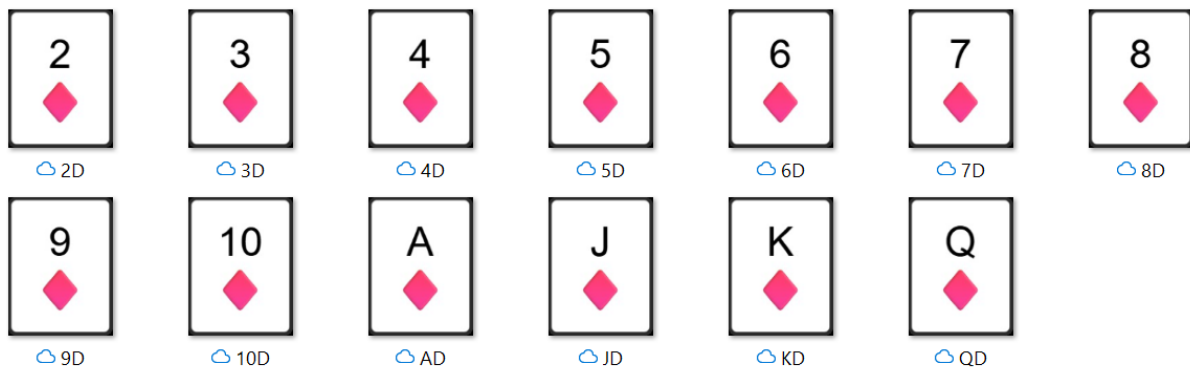
I have updated the back design of the playing cards to look nicer as they were quite plain and simplistic before, I have made these cards have rounded corners as I think it looks appealing and is like a real-life card. I made this using draw.io.

The Spade Cards:



The Heart Cards:



The Club Cards:*The Diamond Cards:***Explanation:**

I have kept the face design for the cards the same as before, this is because they only have the necessary detail on them without being too simplistic.

I have made these cards have rounded corners, like the back of the card design, as I think it looks more appealing than just square corners and is like a real-life card.

I made this using draw.io.

Button Images:

Player Moves Buttons:

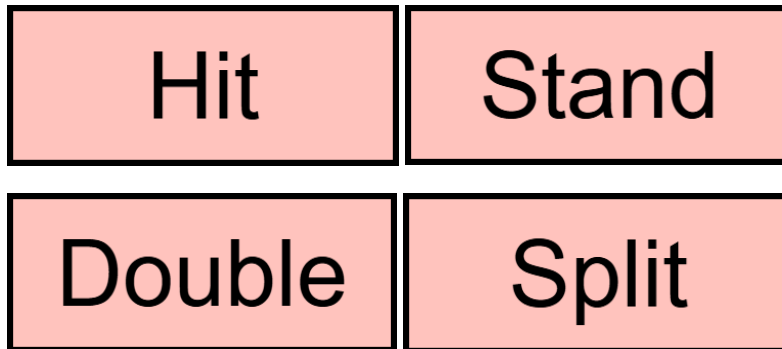


Figure 15-18 – ‘Hit Button’, ‘Stand Button’, ‘Double Button’ & ‘Split Button’

I have made these button images, using draw.io, for each of the player’s moves in the game: Hit, Stand, Double down, and Split.

These buttons will run each of the moves once I have code them.

Betting Round Buttons:



Figure 19 & 20 – ‘DealButton’ & ‘ResetButton’

I have made these button images, using draw.io, that can only be used in the betting phase of the game.

The Reset button will empty the current bet and give the player their money back, this will be used for if the player accidentally bets the wrong amount.

The Deal button will end the betting round and start the game itself.

Money Box Image:



Figure 21 – ‘MoneyBox’

I have made this empty box image, using draw.io, that I will put text inside that will show the player’s total money/balance. This is so the player can see how much money they have.

Poker Chip Images:



Figure 22-27 – ‘PokerChip1’, ‘PokerChip5’, ‘PokerChip10’, ‘PokerChip25’, ‘PokerChip100’ & ‘PokerChip200’

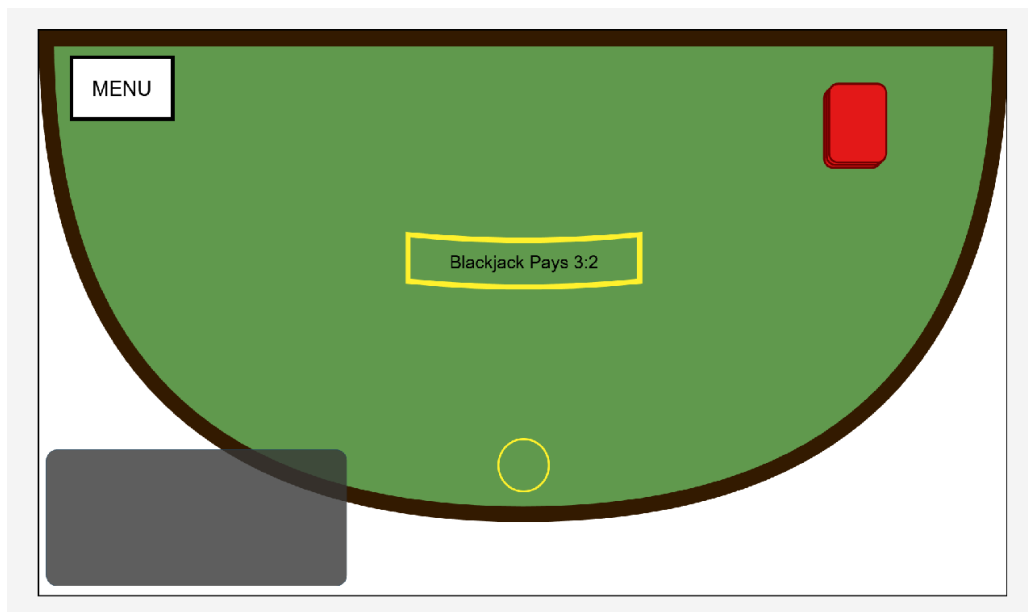
I have made images for each of the poker chips using draw.io. These will be clickable so that the player can bet. Whichever one they click will move that amount of money from their balance, and into their bet.

Development -----

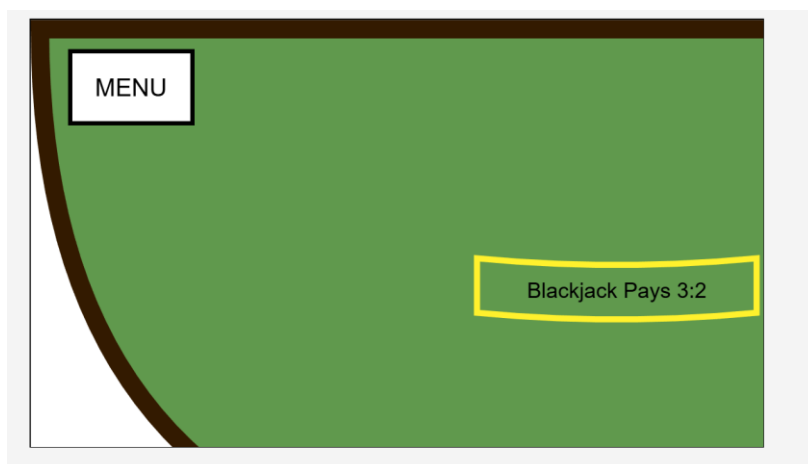
Background Image Code:

```
// BACKGROUND -----  
GreenfootImage image = getBackground();  
image.scale(1205, 700);  
setBackground(image);
```

I have written code in the MainGame class's constructor to set the background image scale to the right size.



Originally, it did not fit correctly (see below).



In order to fix it, I had to research about resizing images on Greenfoot. This was the website I used:

<https://www.greenfoot.org/topics/1509>

Playing Card Images Code:

Card class Method - act():

```
public void act()
{
    // Make image the face up card unless face down
    if (faceDown){
        setImage("CardBack.png");
    }
    else{
        String imageFile = cardIs + ".png";
        setImage(imageFile);
    }
    setCardSize();
}
```

I have modified the Card class's act method (repeatedly executed while the program is being run) to set the Card objects' images to either a back of a card (if their attribute **faceDown** is true), or the corresponding face of a card (if **faceDown** is false).

It also runs setCardSize() to ensure that the card size is set correctly (*see next page*).

For this to work I created an image for the back of the card, and images for every card face (using Draw.io).

When I saved the images, I made the names of each face card image the same name as their respective Card object's **cardIs** attribute. This is so I could easily set the card images. Examples below: 'CardBack', 'AS', '2H', '3C'.



(see more in folder: A2 Coursework > Parts > Iterative Development > Game Images > Card PNGs)

Card class Method - setCardSize():

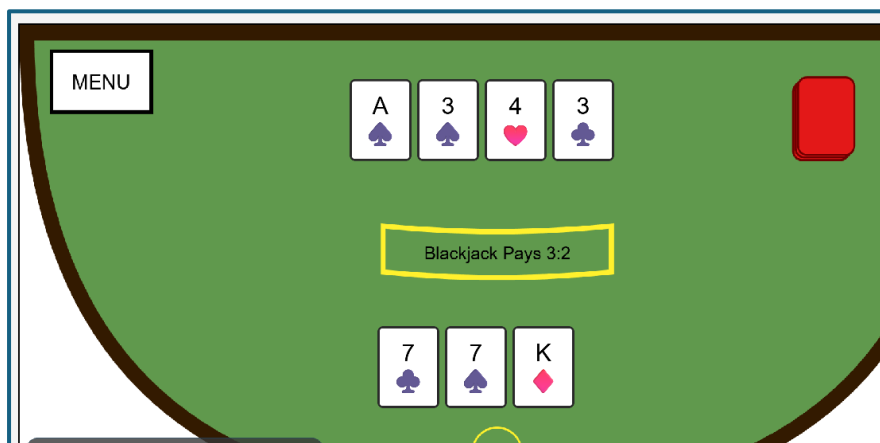
```
public void setCardSize(){  
    GreenfootImage image = getImage();  
    image.scale(78, 103);  
    setImage(image);  
}
```

I have made this method to correctly resize the images of cards to be the correct size in game. This is called when they are instantiated (see *updated Card() method/constructor below*) and in the act() method (see *previous page*).

Card class Constructor - Card():

```
public Card (int cardVal, int valOfCard, String cardSu, String cardIS, boolean faceDOWN){  
    this.cardValue = cardVal;  
    this.valueOfCard = valOfCard;  
    this.cardSuit = cardSu;  
    this.cardIs = cardIS;  
    this.faceDown = faceDOWN;  
    setCardSize();  
}
```

I have edited the Card class constructor so that it runs the setCardSize() method. This is to ensure that the card image's size is correct so that it fits on the background (see *image below*).



Buttons Code:

Button - Parent Class:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class Button extends Actor
{
    public void act()
    {
        // Add your action code here.
    }

    public void setMoveButtonSize(){
        GreenfootImage image = getImage();
        image.scale(120, 50);
        setImage(image);
    }

    public void setMoneyBoxSize(){
        GreenfootImage image = getImage();
        image.scale(265, 50);
        setImage(image);
    }
}
```

The button parent class has two methods that will be inherited by the subclasses.

The method **setMoveButtonSize()** adjusts the player move button images to the correct scale and size.

The method **setMoneyBoxSize()** adjusts the size of the money box image to the correct scale and size. This method will only be run by the money box class.

Button Subclasses:

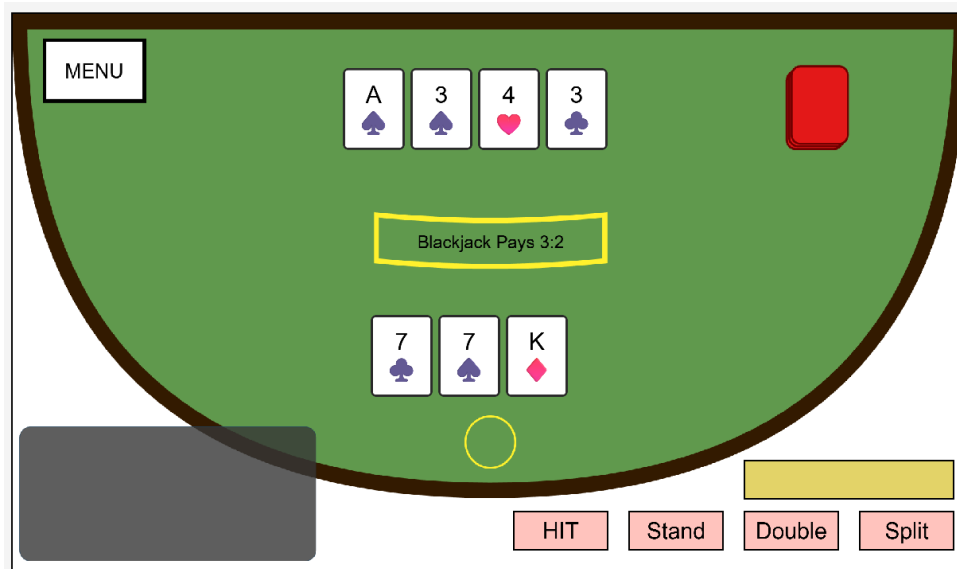
For the subclasses, they have a **pressed()** method that runs code based on the player's moves the buttons are labelled as.

They also each have an **act()** method which I have modified. The act method is repeatedly executed while the program is being run. I have made it check for if the mouse has clicked on the button (similarly to the poker chip subclasses, see *stage 4*).

It does this by using an IF statement and a built in Greenfoot method, **mouseClicked**, which detects when a mouse has clicked on the object passed in. For this code I passed in each button object, so they detect if they have been clicked. If they have been clicked, then it will run the **pressed()** method.

Finally, in all the subclasses' constructors, they run the **setMoveButtonSize()** method.

Buttons (and money box) in the correct place and sizing in game:



HitButton Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class HitButton extends Button
{
    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed(){
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();

        Hand hand = player.getPlayerHand(player.getCurrentHand());

        if (hand.getHandTotal() < 21 && (player.getPlayerTurn() == true)){
            hand.hit(world);
        }
    }

    public HitButton (){
        setMoveButtonSize();
    }
}
```

For the HitButton class, it checks that the player's hand total is less than 21 (using a getter method) and that it is the player's turn. If it meets both of these conditions, it will run the **hit** method in the hand object (see stage 2).

StandButton Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class StandButton extends Button
{
    public void act()
    {
        // Add your action code here.
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed(){
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        Hand hand = player.getPlayerHand(player.getCurrentHand());

        hand.stand(world);
    }

    public StandButton (){
        setMoveButtonSize();
    }
}
```

For the StandButton class, it gets the player's current hand and runs the **stand** method from that hand object (see stage 2).

DoubleButton Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class DoubleButton extends Button
{
    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed(){
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();

        Hand hand = player.getPlayerHand(player.getCurrentHand());

        if ( hand.getHandTotal() < 21 && (player.getPlayerTurn() == true)
        && player.getPlayerMoney() > player.getTotalBet() ){
            player.subPlayerMoney(player.getTotalBet());
            player.addToTotalBet(player.getTotalBet());
            System.out.println("Updated Player Money and Total Bet");
            hand.doubleDown(world);
        }
    }

    public DoubleButton (){
        setMoveButtonSize();
    }
}
```

For the DoubleButton class, it checks:

- The player's hand total is less than 21 (using a getter method),
- That it is the player's turn,
- That the player's balance is more than their total bet (as doubling down means that the player will match their own bet for which they need to have enough money to do so.

If it meets all of these conditions, it will run the **doubleDown** method in the hand object (see stage 2).

SplitButton Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class SplitButton extends Button
{
    public void act()
    {
        // Add your action code here.
    }

    public SplitButton () {
        setMoveButtonSize();
    }
}
```

For the SplitButton class, in its constructor it runs the **setMoveButtonSize()** method.

I have not coded this button fully as I do not have enough time to (see *Development Issue*).

ResetButton Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class ResetButton extends Button
{
    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if(player.getPlayerTurn() == false && dealer.getDealerTurn() == false){
            player.addPlayerMoney(player.getTotalBet());
            player.emptyBet();
        }
    }

    public ResetButton ()
    {
        setMoveButtonSize();
    }
}
```

For the ResetButton class, it checks that it is neither the player's nor the dealer's turn. If it meets these conditions, it will add the total bet back to the player's balance (where it was moved from when they bet) and then run the **emptyBet()** method in the player object (see *stage 2*) which sets the player's total bet to 0.

This effectively moves the players bet back into their balance.

DealButton Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class DealButton extends Button
{
    public void act()
    {
        if (Greenfoot.mouseClicked(this)){
            this.pressed();
        }
    }

    public void pressed()
    {
        World world = getWorld();
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        if(player.getPlayerTurn() == false && dealer.getDealerTurn() == false){
            if(player.getTotalBet() == 0){
                System.out.println();
                System.out.println("--- PLEASE BET BEFORE ATTEMPTING TO START GAME ---");
                System.out.println();
            }
            else{
                mainGame.gameStart();
                player.setPlayerTurn(true);
            }
        }
    }

    public DealButton ()
    {
        setMoveButtonSize();
    }
}
```

For the DealButton class, it checks that it is neither the player's nor the dealer's turn. If it meets these conditions, it will then check whether the player has bet (by checking if the total bet is 0) in an embedded selection/IF statement.

If they have not bet anything, nothing happens and it outputs logging to the terminal saying, "PLEASE BET BEFORE ATTEMPTING TO START GAME".

Otherwise, if they have bet, it will run the MainGame method **gameStart()** and set the player's turn to true.

MoneyBox Code:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class MoneyBox extends Button
{
    public void act()
    {
        // Add your action code here.
    }

    public MoneyBox() {
        setMoneyBoxSize();
    }
}
```

For the MoneyBox class, in its constructor, it runs the **setMoneyBoxSize()** method inherited from parent class.

TotalValueCounter Class:

This class will create text on screen that shows total values such as the dealer's hand total and the player's total balance.

Defining Variables:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class TotalValueCounter extends Actor
{
    private Hand handCounted;
    private Dealer dealerCounted;
    private Player playerCounted;
    private World worldCounted;
    private int handTotal;
    private int totalValue;
    private boolean isDealer = false;
    private boolean isPlayerMoney = false;
    private boolean isTotalBet = false;
```

For this class I have created 9 attributes:

handCounted – Used to store a hand object of the hand total that will be used.

dealerCounted – Used to store the dealer object of the dealer's hand total that will be used.

playerCounted – Used to store the player object of the player's total money and total bet that will be used.

worldCounted – Used to store the world object that will be used to access the other classes.

handTotal – Used to store an integer value of either the player's or the dealer's hand total

totalValue – Used to store an integer value of either the player's total money or total bet.

isDealer – Used to store a Boolean value of whether the TotalValueCounter object will be outputting a counter relating to the dealer's hand total.

isPlayerMoney – Used to store a Boolean value of whether the TotalValueCounter object will be outputting a counter relating to the player's total money.

isTotalBet – Used to store a Boolean value of whether the TotalValueCounter object will be outputting a counter relating to the player's total bet.

Defining Methods:

Constructors:

```
public TotalValueCounter (Hand hand)
{
    this.isDealer = false;
    this.handCounted = hand;
    this.handTotal = handCounted.getHandTotal();

    setImage(new GreenfootImage("Hand Total: " + handTotal, 20, Color.BLACK, new Color(0, 0, 0, 0)));
}

public TotalValueCounter (Dealer dealer)
{
    this.isDealer = true;
    this.dealerCounted = dealer;
    this.handTotal = dealerCounted.getHandTotal();

    setImage(new GreenfootImage("Hand Total: " + handTotal, 20, Color.BLACK, new Color(0, 0, 0, 0)));
}

public TotalValueCounter (Player player)
{
    this.isPlayerMoney = true;
    this.playerCounted = player;
    this.totalValue = player.getPlayerMoney();

    setImage(new GreenfootImage("Balance: " + totalValue, 35, Color.BLACK, new Color(0, 0, 0, 0)));
}

public TotalValueCounter (Player player, boolean value)
{
    this.isTotalBet = value;
    this.playerCounted = player;
    this.totalValue = playerCounted.getTotalBet();

    setImage(new GreenfootImage("Total Bet: " + totalValue, 20, Color.YELLOW, new Color(0, 0, 0, 0)));
}
```

For this class I have made four constructors. These work by using polymorphism based on what objects and values are passed in.

These create a Greenfoot Image with the respective text and values to display in the correct size and colour.

I have made sure size wise they are not too big, and colour wise they don't stand out too much but are still clear to read. I have made the text black with a transparent background (see below, player left, dealer right).



act():

```
public void act()
{
    if (this.isPlayerMoney){
        this.totalValue = playerCounted.getPlayerMoney();
        setImage(new GreenfootImage("Balance: " + totalValue, 35, Color.BLACK, new Color(0, 0, 0, 0)));
    }
    else if (this.isTotalBet){
        this.handTotal = playerCounted.getTotalBet();
        setImage(new GreenfootImage("Total Bet: " + handTotal, 20, Color.YELLOW, new Color(0, 0, 0, 0)));
    }
    else if (this.isDealer){
        this.handTotal = dealerCounted.getHandTotal();
        setImage(new GreenfootImage("Hand Total: " + handTotal, 20, Color.BLACK, new Color(0, 0, 0, 0)));
    }
    else{
        this.handTotal = handCounted.getHandTotal();
        setImage(new GreenfootImage("Hand Total: " + handTotal, 20, Color.BLACK, new Color(0, 0, 0, 0)));
    }
}
```

I have updated the act method to constantly update the text and value displayed on the counter object(s) (as act is run repeatedly while the program is running). In this method, it checks which type of counter it is (whether it be a counter for the player's money, total bet, hand total or dealer hand total) and then updates it with the correct output.

InformationOutputter Class:

This class will create text on screen that shows the end game results after the dealer's turn.

Defining Methods:

Constructor:

```
import greenfoot.*; // (World, Actor, GreenfootImage, Greenfoot and MouseInfo)

public class InformationOutputter extends Actor
{
    public void act()
    {
        // Add your action code here.
    }

    public InformationOutputter (World world)
    {
        MainGame mainGame = (MainGame)world;
        Player player = mainGame.getPlayer();
        Dealer dealer = mainGame.getDealer();

        int textSize = 40;

        int playerHandTotal = player.getPlayerHand(0).getHandTotal();
        boolean playerHandLost = player.getPlayerHand(0).getHandLost();
        int dealerHandTotal = dealer.getHandTotal();
        boolean dealerLost = dealer.checkDealerLost();
        int winnings = dealer.getWinnings();
        int wager = player.getTotalBet();
        int balance = player.getPlayerMoney();
        boolean blackjack = player.getPlayerHand(0).getBlackjack();
    }
}
```

In the constructor, it creates variables using getter methods from the mainGame, player, hand, and dealer classes. These variables are:

playerHandTotal	playerHandLost	dealerHandTotal	dealerHandLost
winnings	wager	balance	blackjack

(For more information on these variables see stages 2 and 3)

It also creates an Integer variable, **textSize**, and sets it to 40. This will be used when creating text images as they will all be the same size.

```

if (playerHandLost && !dealerLost){
    setImage(new GreenfootImage(" YOU LOST, YOU WENT BUST \n Your Hand Total = "
        + playerHandTotal + ", Dealer hand total = " + dealerHandTotal + " "
        + "\n Wager Lost = " + wager + ", Balance = " + balance + " ", textSize, Color.RED, Color.BLACK));
} // GAME END --- 'PLAYER WENT BUST'

else if ( (!playerHandLost && !dealerLost) && playerHandTotal == dealerHandTotal ){
    setImage(new GreenfootImage(" YOU DREW \n Your Hand Total = "
        + playerHandTotal + ", Dealer hand total = " + dealerHandTotal + " "
        + "\n Wager Returned = " + wager + ", Balance = " + balance + " "
        , textSize, Color.YELLOW, Color.BLACK));
} // GAME END --- 'PUSH'

else if ( (!playerHandLost && dealerLost) || (playerHandTotal > dealerHandTotal) ){
    if (blackjack)
    {
        setImage(new GreenfootImage(" BLACKJACK!! YOU WON \n Your Hand Total = "
            + playerHandTotal + ", Dealer hand total = " + dealerHandTotal + " "
            + "\n Winnings = " + winnings + ", Balance = " + balance + " "
            , textSize, Color.GREEN, Color.BLACK));
    }
    else{
        setImage(new GreenfootImage(" YOU WON \n Your Hand Total = "
            + playerHandTotal + ", Dealer hand total = " + dealerHandTotal + " "
            + "\n Winnings = " + winnings + ", Balance = " + balance + " "
            , textSize, Color.GREEN, Color.BLACK));
    }
} // GAME END --- 'PLAYER WON'

else{
    setImage(new GreenfootImage(" ERROR \n Your Hand Total = " + playerHandTotal + ", Dealer hand total = "
        + dealerHandTotal + " ", textSize, Color.RED, Color.BLACK));
}

```

The rest of the constructor method checks the outcome of the game, such as if:

- The player went bust and loses, it outputs the hand total, the dealer's hand total, *wager lost*, and ending balance.
 - The player and dealer's hand totals were equal, it outputs the hand total, the dealer's hand total, *wager returned*, and ending balance.
 - The player won by either getting more than the dealer or the dealer went bust, it outputs the hand total, the dealer's hand total, *winnings*, and ending balance.
- And if the player got blackjack, the message is changed to specify that.

Depending on what the outcome is, the class creates a Greenfoot image with the correct message/text and values and displays it on screen. I have made it so when the player loses the text is red, if they draw with the dealer the text is yellow, and if they win it is green.

Finally, I have also made another output for if the outcome isn't found it outputs "ERROR" alongside the player's and dealer's hand totals. This would be used for checked for bugs.

Updating the MainGame Class:

Constructor:

```
public MainGame()
{
    super(1200, 700, 1);
    System.out.println();
    System.out.println();
    System.out.println("----- GAME START -----");
    System.out.println();

    // Making objects such as buttons and boxes -----

    MoneyBox moneyBox = new MoneyBox();
    addObject(moneyBox, 1052, 586);

    TotalValueCounter hand1Counter = new TotalValueCounter(hand1);
    addObject(hand1Counter, 500, 500);

    TotalValueCounter dealerHandCounter = new TotalValueCounter(dealer);
    addObject(dealerHandCounter, 695, 190);

    TotalValueCounter playerMoney = new TotalValueCounter(player);
    addObject(playerMoney, 1052, 584);

    TotalValueCounter totalBet = new TotalValueCounter(player, true);
    addObject(totalBet, 495, 540);

    DealButton dealButton = new DealButton();
    addObject(dealButton, 305, 560);

    ResetButton resetButton = new ResetButton();
    addObject(resetButton, 305, 645);

    HitButton hitButton = new HitButton();
    addObject(hitButton, 690, 650);

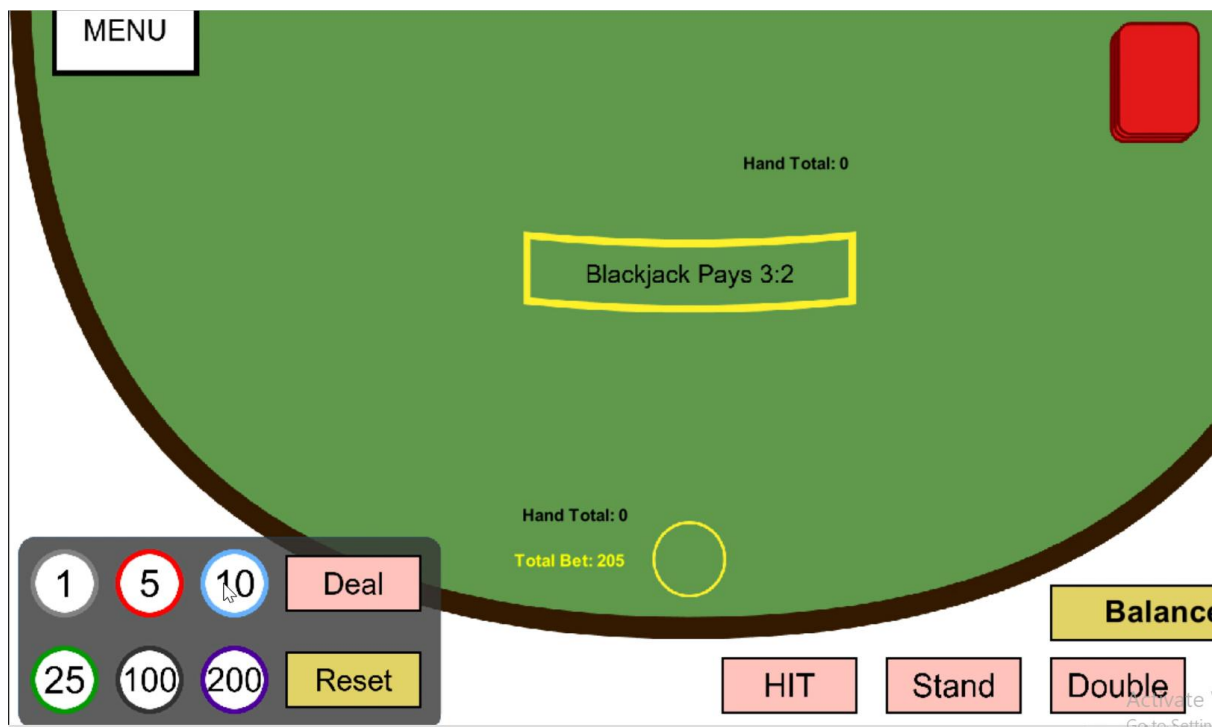
    StandButton standButton = new StandButton();
    addObject(standButton, 835, 650);

    DoubleButton doubleButton = new DoubleButton();
    addObject(doubleButton, 980, 650);

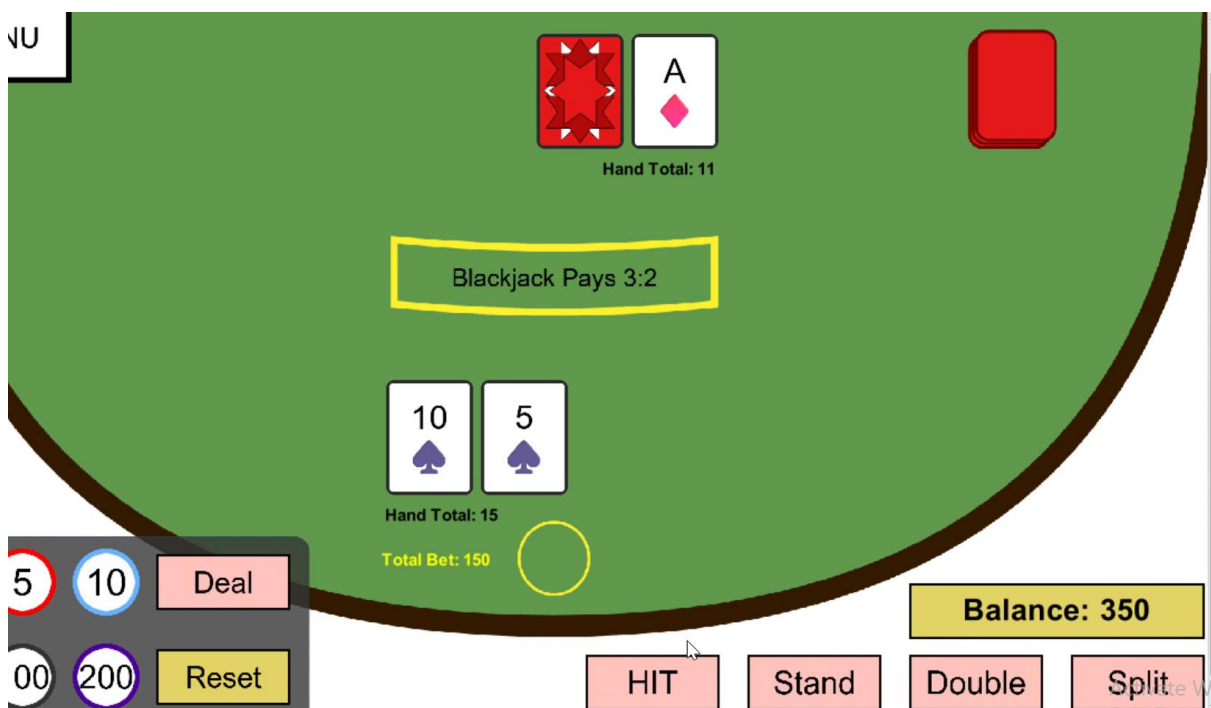
    SplitButton splitButton = new SplitButton();
    addObject(splitButton, 1125, 650);
}
```

I have updated the MainGame class's constructor so that it instantiates all of the GUI related objects (Buttons, Boxes, and Counters) and adds them to the world at the correct positions (see *next page*).

Betting Round:



Player's Turn:



(The above screenshots are the wrong size because they were taken on a laptop, otherwise everything is the correct size for a desktop computer)

Methods:

```
public void displayEndGameInfo()
{
    InformationOutputter infoOutter = new InformationOutputter(this);
    addObject(infoOutter, 600, 350);
    player.emptyBet();
}
```

I have coded a new method for the MainGame class, **displayEndGameInfo()**. This is called at the end of the dealer's method **dealerTurn()** (see stage 3).

This method creates a new InformationOutputter object and adds it to the world, then it runs the player method **emptyBet()** to reset the game.

End of Game:



Testing -----

Test 6a – Testing the Card’s updated **act()** method:

Expected Result: Show the correct image for the card, whether it be face down or face up.

To test this, I will run my game and look at the card objects on screen.

```
public void act()
{
    // Make image the face up card unless face down
    if (faceDown){
        setImage("CardBack.png");
    }
    else{
        String imageFile = cardIs + ".png";
        setImage(imageFile);
    }
    setCardSize();
}
```

Result (Passed):



The player's cards are both face up, as they should be, therefore the player's cards' code has worked.

The dealer has one face up card, and the dealer's second card is face down, this is correct and therefore the dealer's cards' code has worked as intended too.

Test 6b – Testing the Hit Button’s Functionality:

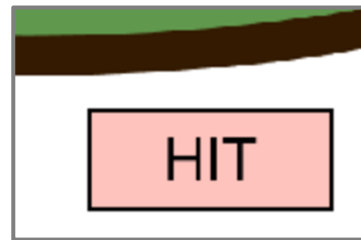
Expected Result: The hit button works as intended and adds a new card on screen and to the player’s current hand.

To test this, I will run my game and look at the card objects on screen before and after I click the hit button.

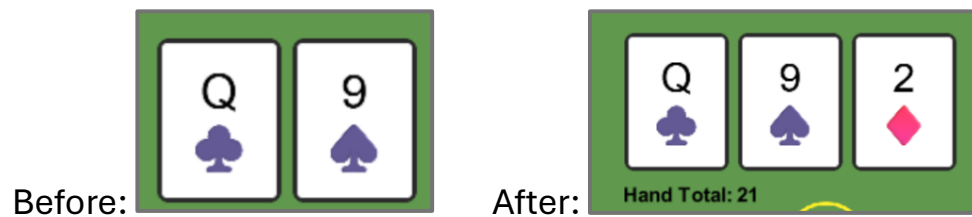
```
public void pressed(){
    World world = getWorld();
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();

    Hand hand = player.getPlayerHand(player.getCurrentHand());

    if (hand.getHandTotal() < 21 && (player.getPlayerTurn() == true)){
        hand.hit(world);
    }
}
```



Result (Passed):



The player has an extra card placed next to their other two, this is correct.

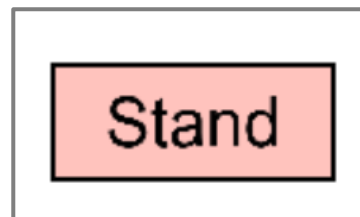
The player’s Hand Total has also been updated, including the new card which means the card object has been correctly added to the player’s hand.

Test 6c – Testing the Stand Button’s Functionality:

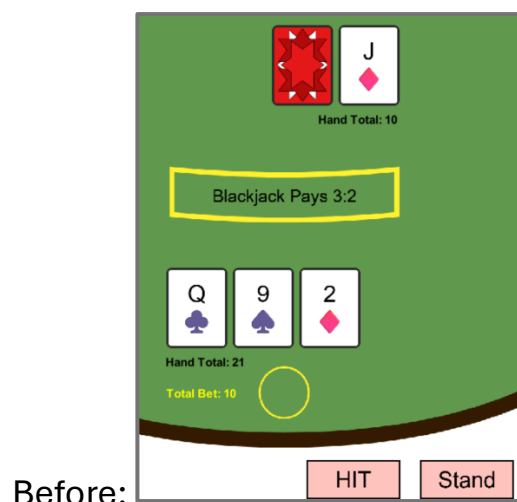
Expected Result: The stand button works as intended and ends the player’s turn.

To test this, I will run my game and look at the game on screen before and after I click the stand button.

```
public void pressed(){  
    World world = getWorld();  
    MainGame mainGame = (MainGame)world;  
    Player player = mainGame.getPlayer();  
    Dealer dealer = mainGame.getDealer();  
  
    Hand hand = player.getPlayerHand(player.getCurrentHand());  
    hand.stand(world);  
}
```



Result (Passed):



After pressing the stand button, the player’s turn ends without drawing another card and the dealer has its turn correctly.

Test 6d – Testing the Double Button’s Functionality:

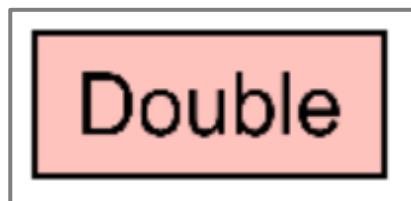
Expected Result: The double down button works as in intended by updating the player’s total bet and balance correctly, dealing the player another card, and then ending the player’s turn.

To test this, I will run my game and look at the game on screen before and after I click the double down button.

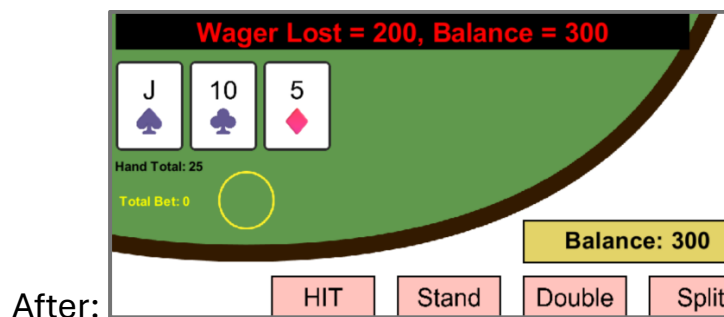
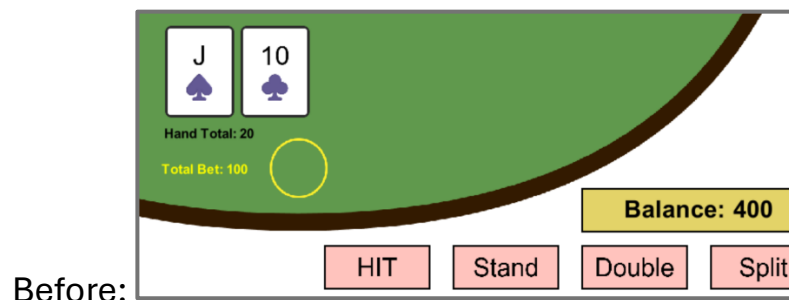
```
public void pressed(){
    World world = getWorld();
    MainGame mainGame = (MainGame)world;
    Player player = mainGame.getPlayer();

    Hand hand = player.getPlayerHand(player.getCurrentHand());

    if ( hand.getHandTotal() < 21 && (player.getPlayerTurn() == true)
    && player.getPlayerMoney() > player.getTotalBet() ){
        player.subPlayerMoney(player.getTotalBet());
        player.addToTotalBet(player.getTotalBet());
        System.out.println("Updated Player Money and Total Bet");
        hand.doubleDown(world);
    }
}
```



Result (Passed):



After pressing the double down button, the player is dealt another card, their bet and balance are updated correctly, and their turn ends. This is correct and has therefore passed the test.

Test 6e – Testing the TotalValueCounter Functionality:

Expected Result: The TotalValueCounter objects work as intended and display the correct value.

To test this, I will run my game and look at the card and counter objects on screen.

Result (Passed):



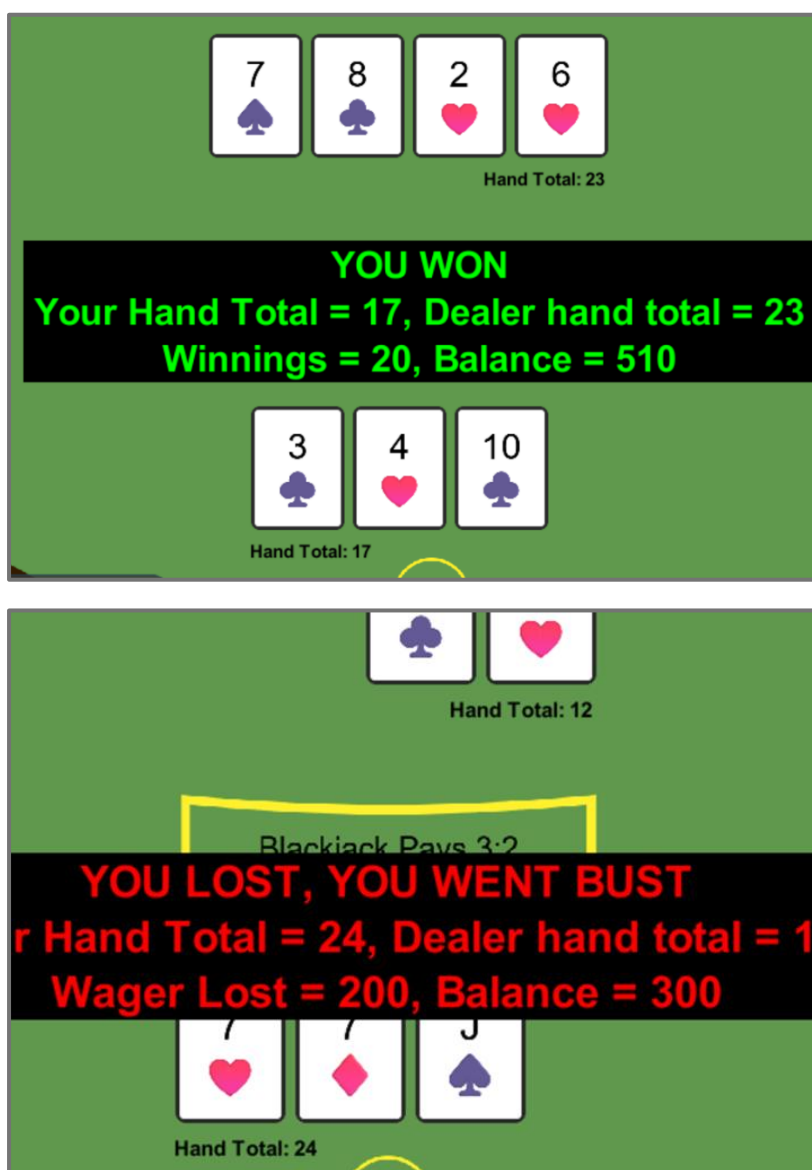
The player hand counter is the correct amount; I can see this by looking at the cards. The same goes for the dealer's hand counter which is the correct value before and after its turn.

Test 6e – Testing the InformationOutputter Functionality:

Expected Result: The InformationOutputter object works as intended and displays the information at the end of the game.

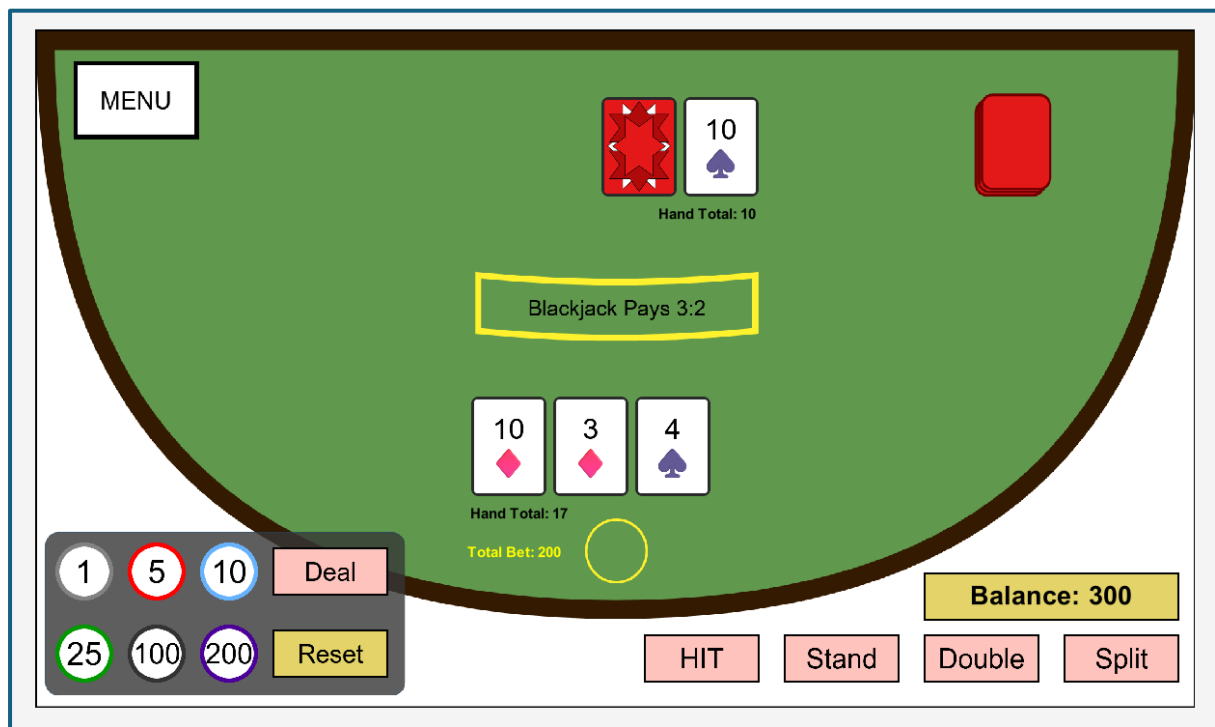
To test this, I will run my game, play through to the end of a game, and see if it creates the object correctly.

Result (Passed):



The object is created successfully and displays an end game message in different scenarios.

The final look of my game's GUI:



For the screen recording of my finished game, see file: “*Blackjack Game – Final Screen recording*”



✓ Blackjack
Game - Final
Screen
recording

END STAGE TESTING -----

Robustness Testing:

Tests I have run to test my game's ability to handle stressful / complicated situations while remaining functional & useable.

Test 1:

Situation: Attempting to use player move buttons outside of turn.

When it is not the player's turn, they should not be able to use their move buttons (*/i.e.: during the betting phase or dealer's turn*).



✓ Robustness
Test 1

After starting the game, I pressed each of the player move buttons. These all, except for the split button as it does not have fully implemented code, caused the game to stop running (see video "Robustness Test 1" for proof).

To fix this, I need to ensure that the buttons check that it is the player's turn before running their **pressed()** methods (see stage 6 for more information) and don't cause any errors.

Outcome: **Fail**

My game has failed this test as it stops running when the player move buttons are used outside of the players turn.

Test 2:

Situation: Attempting to deal the cards (start the game) without betting.

During the betting phase, the player should only be able to click the deal button once they have placed a bet.



✓ Robustness
Test 2

After starting the game, I pressed the deal button without having placed a bet. The game did not deal the cards, and it continued to run (see video “Robustness Test 2” for proof).

Then, after placing a bet, I pressed the deal button, and it started the game correctly.

Outcome: Full Pass

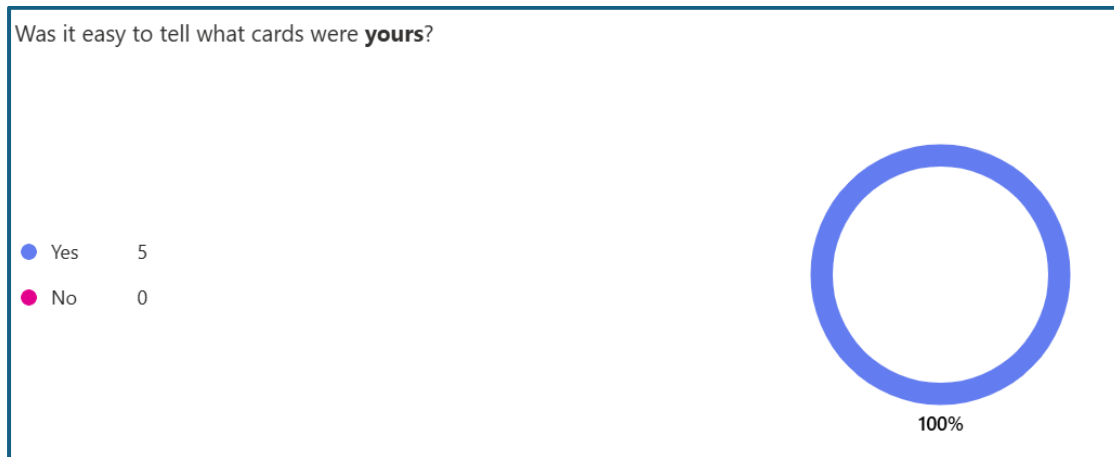
My game has passed this test as it continues to run but stops the game from dealing the beginning cards (starting) when the player attempts to press the deal button during the betting phase without betting first.

Beta Testing & Survey:

For the end stage testing of my game, I got 5 people to beta test it and then fill out a survey that I created.

Here are the responses I got:

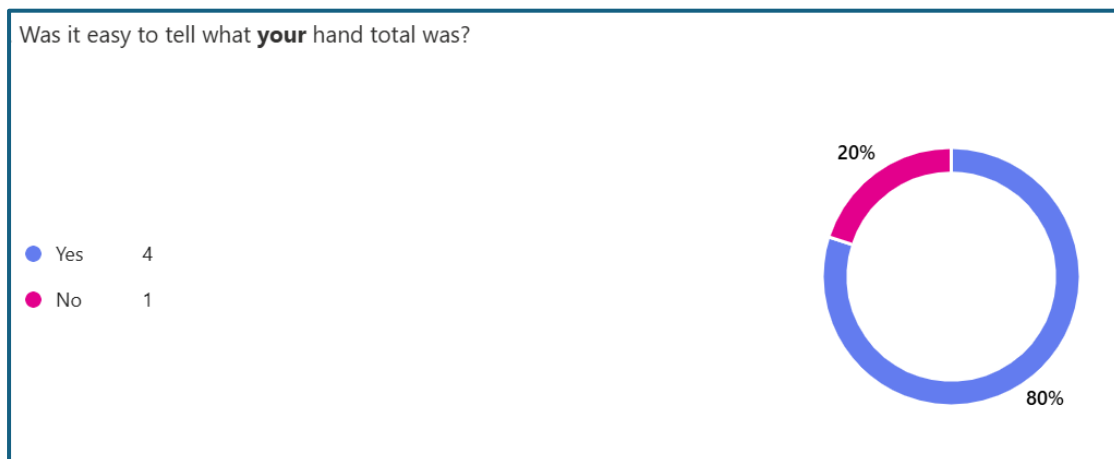
Q1)



I asked this question to check that people knew which cards were theirs. If they didn't, I would have had to make it clearer for the user.

From the responses I can see that people knew which cards were theirs. This means I don't need to make it more clear on what cards belong to the spectator.

Q2)



I asked this question to check that people could tell what their hand total was.

From the responses I can see that the majority of people could tell what their hand total was, however, someone couldn't. To make it clearer, I could change the text in the player and dealer's counters from both being "Hand Total" to being more specific such as "Your Hand Total" and "Dealer's Hand Total".

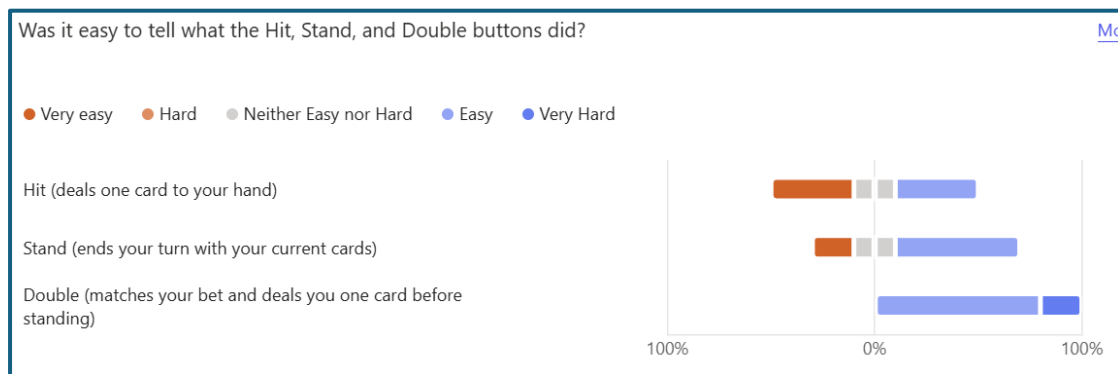
Q3)



I asked this question to check that people could tell what the *dealer's* hand total was.

From the responses I can again see that most people could tell what the dealer's hand total was, however, someone couldn't. To make it clearer, I could make the same change I suggested for Q2's results; Changing the text in the hand total counters to be more specific about who's hand total they are counting.

Q4)

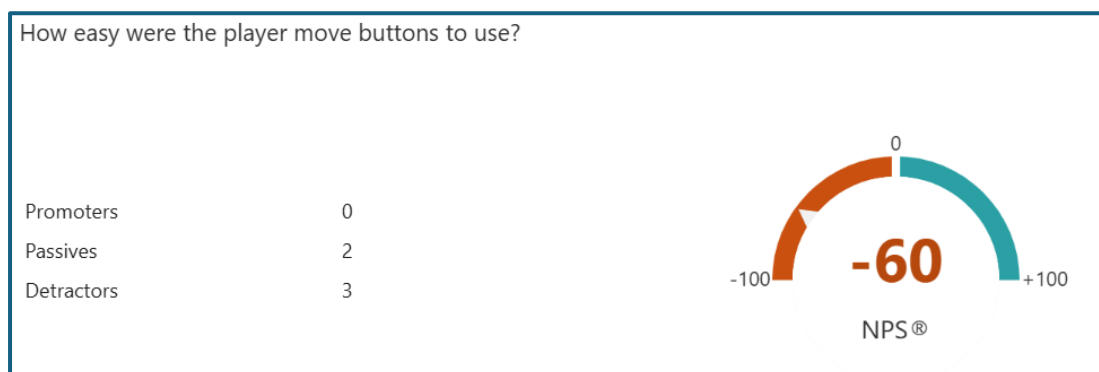


I asked this question to see if people understood what the player's move buttons did. Please also note I made an error when creating the question as the Hard and Easy options should be swapped, however you can still understand the responses based on the colours.

From the responses I can see most people understood what they did, however this may be because they've played Blackjack before, whereas people who haven't played Blackjack might have no clue on what they do.

To make it clearer on what each button does, I could implement a help menu that explains what each move does. This would make my game more player friendly which would greatly improve the usability.

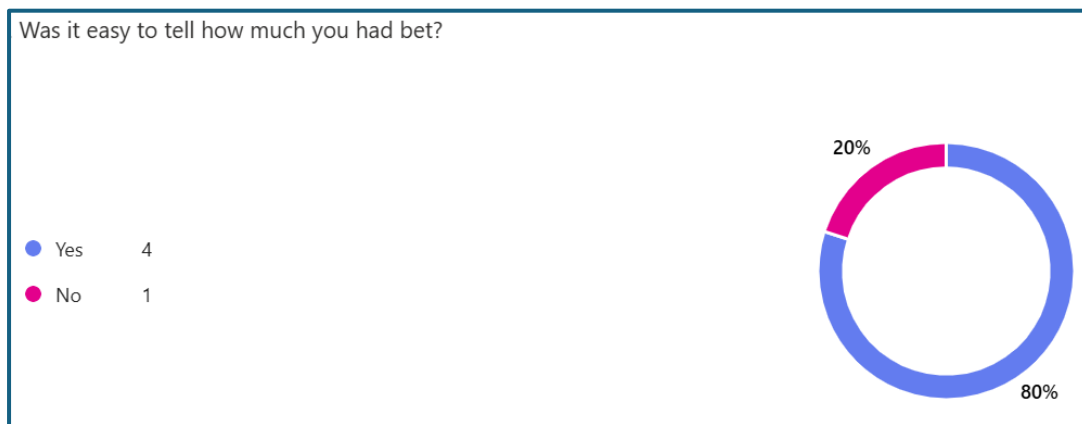
Q5)



I asked this question to find out how easy/hard the player move buttons were to use.

From the responses I can see most people rated it quite low, meaning they found it hard to use. To improve this issue, I could implement a help menu that explains what each move does which might make it easier for the user to use the buttons.

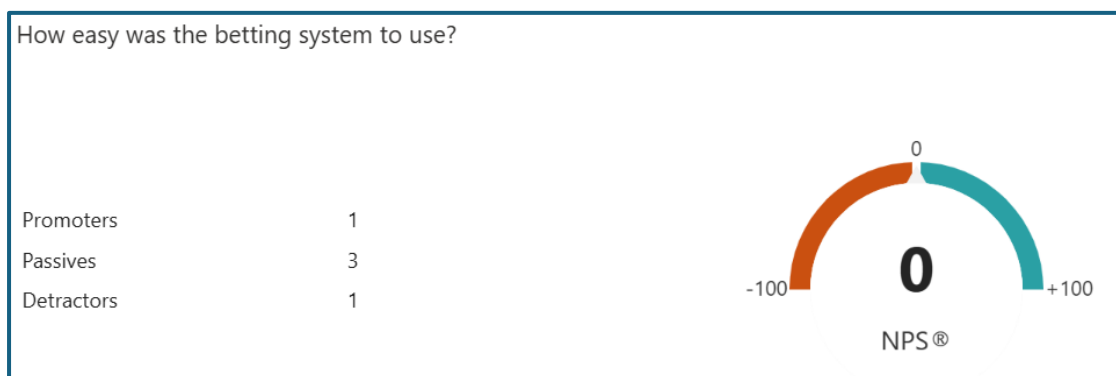
Q6)



I asked this question to check that people could tell what their total bet was.

From the responses I can see that most people, except for a few, could tell what their total bet was. To make it clearer, I could change the text in the total bet counter from both being “Total Bet” to being more specific such as “Your Total Bet” or “Player’s Total Bet”. This seems to be a common issue with the GUI, certain counters not being specific enough for players. Again, changing the GUI to be clearer would fix this issue.

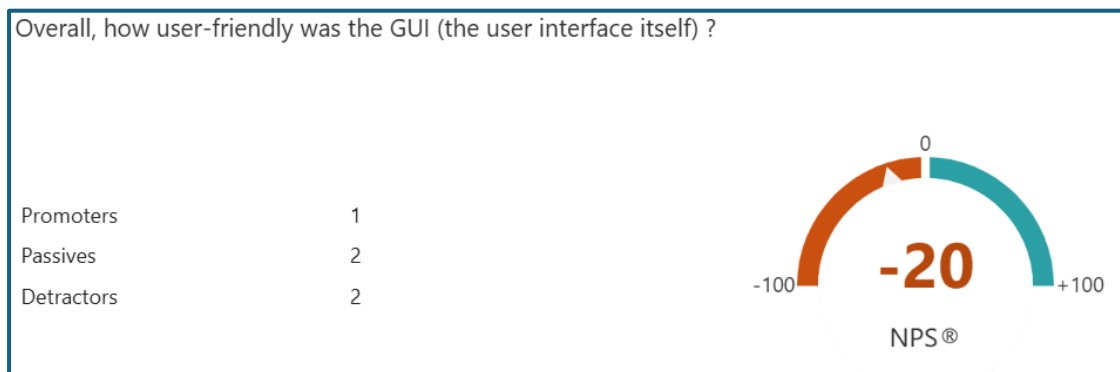
Q7)



I asked this question to find out people’s thoughts on the betting system and how easy it was to use.

From the responses I can see that I got a similar amount of people that found it easy to people that found it hard, and a few others in the middle. In order to make it easier to use, I could implement a help menu explaining how the betting system works.

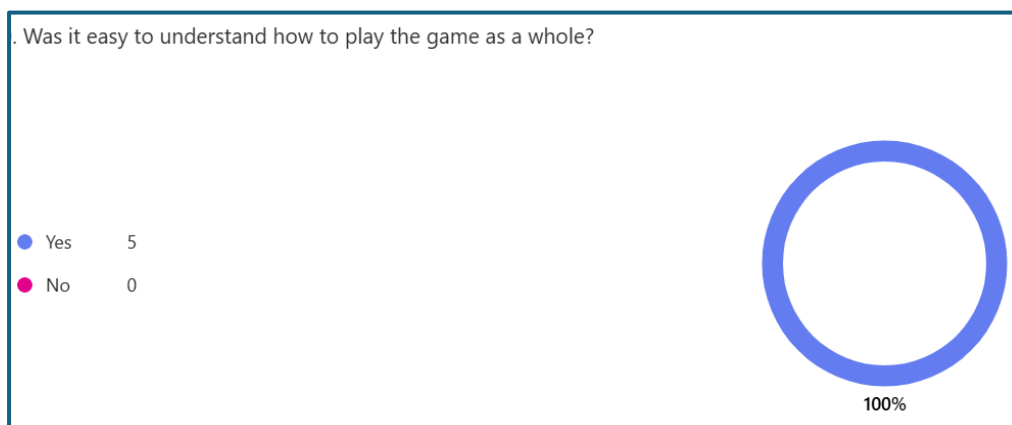
Q8)



I asked this question to find out people's thoughts on the GUI as a whole and how user-friendly they found it.

From the responses I can see that most people thought it was not user-friendly, this is not good. In order to make it more user-friendly, I should implement features that make every part of my game clear on how it works and what it does.

Q9)



I asked this question to find out if people understood how to play the game as a whole.

From the responses I can see that everyone knew how to play it. Because of this I do not need to fully explain all the rules. However, due to the previous questions' responses, it would still be a good idea to have a quick help menu or something similar.

Q10)

. Any bugs/issues you found?		
3 Responses		
ID ↑	Name	Responses
1	anonymous	Not when playing the game as intended
2	anonymous	the game would implode upon itself anytime i did something
3	anonymous	Every time I press hit or stand before play game would crash

I asked this question to find out if any of the beta testers found any bugs/issues.

From the responses (& verbal feedback in person), the issue these three people are referring to is the game crashing when a player move button is pressed during the betting phase (see *Robustness Testing*, 'Test 1'). To fix this, I need to ensure that the buttons check that it is the player's turn before running their **pressed()** methods (see *stage 6 for more information*).

Q11)



I asked this question to find out what the beta testers thought of the games visible design.

From the responses I got positive results, however from verbal feedback I've heard that changing some of the colours of the buttons would look nicer. This is something I could change to improve my game.

Q12)

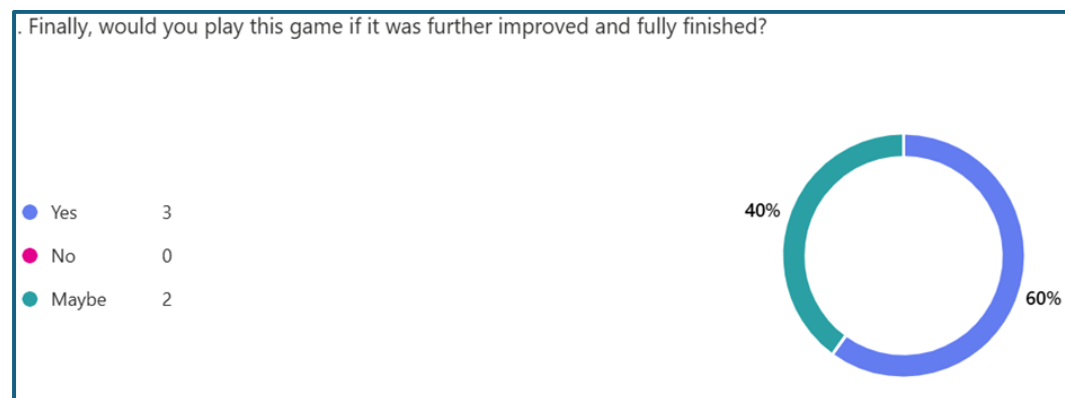
Recommendations for the game's design?

I asked this question to find out if the beta testers had any recommendations for my games design

Some instructions for those who don't know how to play Blackjack might be useful

I got this response for the question. It suggests adding instructions for players in case they don't know how to play Blackjack. This would be an important feature to implement as, even though all my beta testers had played Blackjack before and knew how to, other people who would play my game may not.

Q13)



I asked this question to find out if the beta testers would be interested in playing my game if it were further improved and fully finished.

From the responses I can see that they would want to, this means that the concept of my game is enticing to them.

EVALUATION -----

For the original success criteria for my game, see 'Requirements for my game' pages.

Success Criteria 1 – Menu / Options:

Criterion:

Having a functioning menu screen with these features:

- Being able to exit the game
- Toggleable Perfect Strategy cheat sheet
- Start button
- Title screen
- Toggleable Card Count counter

Fulfillment: NOT MET

Comments:

I was unable to implement a functioning menu due to time constraints. This is because, in my planning, I forgot to include the dealer and the overall flow of the game as stages. This led to me having to cut features from my game that I originally intended to implement so I could ensure my game was playable, with the main one being a menu / options screen (*for more information see 'Development Issue' Page*).

Missing this out will have a negative effect on the users of my game as its purpose is to assist them. Without this, some player may struggle to understand how the game works, which could lead them to not playing it anymore.

However, I have made a concept design for what the Menu would look like and would like to implement it in the future as this would make my game feel more polished.

Development:

For the concept design of the menu, see '*DESIGN, GUI Design, The Menu*'.

Improvements:

If my plan had included a dealer and overall flow of the game stages, I would have given myself more time to ensure that I could implement the menu / options screen as well.

Success Criteria 2 – Help Menu:

Criterion:

2.	Help Menu			Navigational menu on the <u>basic</u> rules of Blackjack. May be part of the main menu or will be a separate menu.	Players who do not already know the rules will need this to play the game. Furthermore, people who know the rules may need reminding.	Important
----	-----------	--	--	--	---	-----------

Fulfillment: NOT MET

Comments:

I could not implement this feature due to the same time restraints as the menu. This is an important feature to have missed as shown by my beta testing (see *End Stage Testing*).

In the beta testing, several people commented on the lack of information provided to the player. The player is never told what to do which is a huge issue as for those who don't know how to play Blackjack will struggle with playing my game.

Development:

N / A – I did not start developing this feature.

Improvements:

By implementing this feature, it would greatly improve the overall usability of my game. It is currently an issue (not just a lack of a feature) that my game does not have a help menu.

This would have a very positive effect on the user-friendliness of the game, and the user will notice if this feature is absent.

Success Criteria 3 – Stacked Poker Chips:

Criterion:

3.	Stacked poker chips			Purely visual. See what you have bet, stacked in front of you.	This would be purely visual as there is already a counter on-screen for the total value you have bet.	Not important
----	---------------------	--	--	--	---	---------------

Fulfillment: NOT MET

Comments:

I did not implement this feature. This was a low priority as it was purely visual, I put it in the success criteria in case I had extra time.

Having this feature in my game would not impact the user much. They won't actively notice it is missing but having it there would subconsciously add to the immersion of the game.

Development:

For the concept design of what the stacked chips would look like, see the '*DESIGN, GUI Design, The Main Game*'

Improvements:

I could implement this feature into the game. This would increase the realism and immersion of that game.

This would also make the game more visually appealing.

Success Criteria 4 – Toggleable Cheat Sheet:

Criterion:

No.	Feature		Sub-features	Explanation	Justification	Importance
4.	Toggleable cheat sheet			Toggleable perfect strategy cheat sheet.	This feature is important as it is what will make my game stand out from other existing similar games.	Important

Fulfillment: NOT MET

Comments:

I couldn't implement this feature due to time constraints. This was because of the same reason as the menu, when planning I did not account for the dealer and the game's overall flow. This was one of the features I had to cut.

This feature was one of the main ideas behind my game as it was planned to be the game **and** a learner. Users can still use my game to practice Blackjack, but this feature would have made it easier.

Development:

N / A – I did not start developing this feature.

Improvements:

By implementing this feature, it would've made my game into a Blackjack *learner* as well by adding a learning aspect of trying to memorise perfect strategy.

This would've set it aside from other existing, similar games. It would also make the game more enjoyable while giving it a learning curve. This would have a positive impact for the user experience.

Success Criteria 5 – Moves The Player can make:

Criterion:

The player can do these Blackjack moves:

- Hit (*Have a new card dealt to their hand*)
- Stand (*End their turn with their current cards*)
- Split (*Split their hand into two, given that their first two cards are equal in value*)
- Double Down (*Match their bet, have one final card dealt to their hand, and then stand*)
- Bust (*Player's hand total goes over 21*)
- Insurance (*An extra option that the player can choose to buy if the dealer's upcard is an Ace*)

Fulfillment: MAINLY MET

Comments:

I have successfully implemented the ability to Hit, Stand, and Double Down (see *Stage 2 and Stage 6*). The player can also go Bust.

I have started to implement the ability to Split, however due to time restraints I was unable to finish it (*for more information see 'Development Issue' Page*).

I did not implement Insurance, also due to the same time constraints as Splitting. As this is quite a small feature of Blackjack, this feature had considerably lower priority. Because of this, I left this feature out so I could make sure more important parts of my games were completed such as the GUI.

Development:

I developed the code for this in 'Stage 2: Player & Hand Classes' and made functioning buttons for the moves in 'Stage 6: Making the GUI'. (*See stage 6 for testing the buttons*)

Improvements:

If I had more time, I would implement Splitting and the option to buy Insurance. These features are in standard Blackjack, so if I ever wanted to release my game it would need these features to be able to compete with other similar, existing versions.

Success Criteria 6 – The GUI:

Criterion:

Having a functional GUI the player can interact with to play the game, it should also be clear on what the player can & should do.

Feature		Sub-features	Explanation	Justification	Importance
GUI	a	Current bet total counter	The total amount that the player has currently bet will be displayed on screen.	This will help the player see more clearly how much they have bet.	Essential
	b	Current upcard total counter for the player	The total amount that the player's upcards add up to will be displayed on screen.	This will help the player see more clearly what their running total is.	Essential

Fulfillment: MET

Comments:

I fully met my original success criteria for the GUI. My game has a total bet counter which displays the player's current bet on screen at all times. My game also has a current upcard total counter for the player on screen.

My GUI also has buttons for each of the player's moves, buttons for betting, clickable poker chips to bet, and displayed values on screen such as balance, hand totals, & total bet.

I believe I have fully met this criterion as I have a fully functioning GUI that the user can interact with. If I was not to have met this criterion, my game would be unplayable for the user as it relies on there being a GUI. Users will notice the quality of the GUI as this is what they are presented with.

Development:

I developed this in 'Stage 6: Making the GUI'. (Also see stage 6 for testing of the GUI)

Improvements:

To improve my GUI, I could make parts of my game clearer as during the beta testing of my game some users were unsure what parts of my game did, such as the player's move buttons (see *End Stage Testing*). Have a quick-help menu would solve this problem.

Success Criteria 7 – Game Rules on the Table:

Criterion:

7.	Game rules functionality & displayed on the table	a	Blackjack pays 3:2	When a player gets a Blackjack, they will be paid 3:1.	This rule is needed to ensure the amount the player wins is a fair amount.	Essential
		b	Dealer must stand on 17 and draw to 16	If the dealer's upcards add to 17, the dealer <u>has to</u> stand. If the dealer does not, the dealer <u>has to</u> draw until they have 16.	This rule is needed to make the game more balanced.	Essential
		c	Insurance pays 2:1	If the player wins the insurance side bet, they get paid 2:1.	This feature is needed for the game to function correctly (If I include insurance as a feature).	Important (if I include Insurance)

Fulfillment: MAINLY MET

Comments:

The game rules functionality has been met. When the player gets Blackjack, they are paid the right amount. The dealer has its turn correctly; it draws to 16 and stands on 17. These are essential features of my game and have been successfully implemented. (see *Stage 3 for more information on both features*)

I have not implemented the insurance payout as I did not implement insurance (*for the reason why I did not implement it, see the Evaluation for Success Criteria 5*).

I only had the game rule of Blackjack pays 3:2 displayed on the table in the game (see *Stage 6*). I did put the game rules of how the dealer plays, and insurance payout on the table.

Development:

I developed the correct payouts, and dealer moves in '*Stage 3 – Dealer Class*' and made the displayed rules in '*Stage 6 – Making the GUI*'.

Improvements:

I could have added the game rules for how the dealer plays and the insurance payout. This would've made my game more realistic and added to the immersion. It also makes the game rules clearer to the player as otherwise they would not know the rules. This is a necessary improvement if I were to release my game.

Evaluation – Extra Notes

Looking back on my code, I found some issues, such as code I could've written more efficiently and things I wanted to change.

*These things affect the **usability** of my game, and some could not be implemented due to **limitations**.*

How I instantiated the deck:

The way I chose to instantiate the Card objects for the Deck of Cards is inefficient and uses lots of code. This is because I instantiated each Card object line by line (see *stage 1*).

From looking back on the code, I have since realised that I could have used iteration to loop through code that would instantiate all the card objects, this would've reduced the amount of redundant code.

Dealer Act Method:

The Dealer's act method (see *stage 3*) could have been a check values method that is called at the end of other methods, like the Hand class's **checkHandStatus()** method (see *stage 2*).

Also, the Dealer's act method currently checks for Blackjack based on if the total is over 21. This is not what Blackjack is, it should check for if the Dealer's *first* and *last* card total to 21.

Player Bust Issue:

When the Player goes bust, the code does not empty the Player's total bet. This is an issue as it would keep their bet to the next round when it should be removed.

Money Box Issue:

The Money Box class should **not** be inherited from the Button subclass as it is not a button. The reason why I coded this way is because it was easier to just code it in the moment, however this could cause future errors such as if a different Button subclass called the Money Box method.

Time Delays:

To further improve my game, I could add time delays. This would make my game a lot more enjoyable as currently the Dealer's turn and end game output happen in a split second; This can stop the user from understanding what has happened in the game and may put them off playing it again.

By adding time delays it would add to the realism and immersion of my game.

Ace's Values:

Currently, Aces in my game only have value of 11, but they can also be 1 in real Blackjack rules. Because of this, if the Player's starting hand is two aces, they will instantly lose which is unfair. This needs to be fixed if I want to release my game.

This is an important feature that I would need to add if I wanted to release my game in the future.

Insurance:

I did not implement Insurance (or the rules for insurance on the table's design) due to time limitations (*see Success Criteria 5 and 7 for more information*).

This again is an important feature that would need to be added if I were to release my game in the future.

Dealer Blackjack Issue:

Currently the dealer does not automatically win if they get Blackjack, the game just counts it as a normal hand and only checks if it is higher, less than, or equal to the players hand (or gone bust).

This is another important feature for the game as it is a rule for Blackjack. Again, I would need to implement this if I wanted to release my game in the future.

InformationOutputter Issue:

Currently, the InformationOutputter class does not have the code in place for if the Dealer has a higher hand total than the player, when neither go bust. This is a very important feature and is needed for my game to work correctly.

It would not take much code to implement this as most of it is already written. The reason it has not been implemented is because I did not notice it was missing when coding my game.

Poker Chip Mini Menu:

During the initial survey for my project (see *Survey Questions, Results and Analysis*), someone suggested to me to make my poker chip mini menu into a pull out / drop down menu.

This would free up space on the screen allowing for a less cluttered UI. It would also give it a more polished feel by having smaller features in my game.

Player Move Buttons Issue:

The player move buttons cause an error when clicked when it is not the player's turn (*i.e.: during the betting phase*). This needs to be changed so that it is similar to how the poker chips work (they don't run any code when they shouldn't be pressable).

In addition to this, as there are no prompts to the player on what to do, I noticed my beta testers tried to press hit before betting. This of course caused the error and led to confusion.

To fix this, I need to edit the code (as described above). Also, the implementation of user prompts would hugely improve my game as currently there are no instructions on what to do at all. Regardless of if the user knows how to play Blackjack, they may still be confused on what to do for this reason.

Maintenance:

Below describes what I have utilised to make sure that my code is maintainable.

Use of Functions:

I made my game using an object-oriented approach, this meant that my program was split into classes and methods. This has a positive impact on the maintainability of my program as it means I can easily find where any bugs or issues are.

This also has reduced repeated code, ensuring there is not redundant code left over. Finally, it allows my game to load quicker and run smoother.

Naming Conventions:

I have named my variables appropriately so that it is easy to understand what they store from just their name. This makes debugging easier as I know what variables are for what data at just a glance.

I chose to write my attributes (variables) in Camel Case to make it clearly readable while not taking up too much line space in the code.

I chose to write my methods (subroutines) in Pascal Case for the same reason as the attributes; It makes it easy to read while again not taking too much line space.

The difference between these naming conventions is that the first letter is not capitalised in Camel Case but is in Pascal Case. This allows me to quickly differentiate between attributes and methods.

Code Indentation:

Greenfoot has an auto indent feature (by pressing CTRL, SHFT, & I) which I utilised to layout my code in a clear, readable way.

I also manually indented certain lines where I wanted spacers such as between old, unused code and newly written code (see *Stage 1*). I chose to leave the code in case I would need it in the future.

Comments and Logging:

I have used code comments in my game to describe what certain pieces of code do. I also used them as notes and reminders for myself to know what to do next if I had to leave a piece of code to come back to it later (*i.e.: checking or modifying one stage while developing another*).

I also have implemented logging in my game. This was extremely useful and important for debugging my code. Because of this, I was able to fix several bugs and issues with my game (see *END STAGE TESTING*).

I wish I had coded more logging in my game as I currently don't have most of the code logged.

Limitations:

Below describes features that I would have liked to have implemented but was not able to.

Time Delays:

Time delays in my game between actions would make a huge difference as it changes the feel it has and makes it feel like a game as opposed to a program. I have gone into more detail about this feature in *Evaluation, Extra Notes, 'Time Delays'*.

Insurance:

Insurance is something I wanted to include in my game but couldn't. This is necessary if I want to release my game in the future. I have also gone into more detail about this feature in *Evaluation, Extra Notes, 'Insurance'*.

Splitting:

Splitting is a necessary part of Blackjack. It allows the player to split their hand into several if they have two cards of the same value.

Menu & Help Menu:

In my original success criteria, I wanted to include a Menu and or a Help Menu. This would ensure players knew what to do as they could check the rules quickly and easily. I have gone into more detail about these features in *'Requirements for my game' and Evaluation, 'Success Criteria 1 & 2'*.

Perfect Strategy Cheat Sheet & Card Count Counter:

Also in my original success criteria, I wanted to implement a perfect strategy cheat sheet and a card count counter. I have gone into more detail about these features in *'Requirements for my game' and Evaluation, 'Success Criteria 1 & 2'*.

Extra Limitations:

For extra limitations not listed here, see *'Identifying Limitations'*.

Conclusion:

To conclude, I am happy with the current state of my game despite not meeting all of my original success criteria or finishing my game fully (See *Evaluation*, 'Success Criteria 1-7'). I still managed to get my game to be playable despite having to make significant changes of plans due to time constraints (see 'Development Issue'). Because of these time constraints, I had to cut a lot of features from my game, meaning that it is unfinished and lacking several important features.

If I were to release my game, I would need to finish it off by fixing the current issues / bugs it has (see *Evaluation*, 'End Stage Testing' and 'Extra Notes'). I also would need to implement the rest of the main features of Blackjack itself; These include but are not limited to: the ability for the player to split, insurance for if the dealer has an ace as an upcard, and different values for an ace (For the full list of these feature and explanations of what they are and why I could not do them, please see *Evaluation*, 'Extra Notes' and 'Limitations').

Finally, I would add extra features such as Time Delays and Customisation into my game before I'd release it. These features would make my game feel more polished and would allow my game to stand out from other similar, existing versions (Please also see *Evaluation*, 'Extra Notes' and 'Limitations' for the full list and explanations of extra features).

Overall, if I could start my project from the beginning, I would spend more time planning and ensuring that everything I needed to do was written out clearly (see my planning under the 'DESIGN' main header and the 'Design' sub-headers for individual stages). As I was planning, I wasn't sure on how to code my game as it was my first independent project coded in an Object-Oriented paradigm. I have since realised that Object-Oriented programs heavily rely on how you plan them, this is because you create lots of code that can be used throughout the program (e.g.: getters, setters, & other methods) and then you tie / link them all together at the end (as opposed to making it as you go along for a procedural programming paradigm).

I would also focus on the user friendliness of my game as the GUI is currently not very user friendly and may cause confusion for the player (for more information please see 'End Stage Testing' and *Evaluation*, Limitations, 'Menu & Help Menu').

Extra References:

[Greenfoot Tutorials Playlist](#) – YouTube

I watched through this playlist before I started this project to refresh myself on how to code in Java in Greenfoot.

[Get Last Value of an ArrayList](#) – Stack Overflow Website

I used this to remind myself how to get last value of ArrayList.

[Java Arrays](#) – W3Schools Website

Originally, I used this to remind myself how to create an array in Java. I needed to create an array for the possible suits and values the cards in my game could have.

I then later realised an ArrayList would be more suitable, so I used that instead (see *Stage 1*).

[Greenfoot.org Topics](#) – Greenfoot Website

I used this to remind myself how to make a random number in Greenfoot. This was needed so I could generate random numbers to then be used to generate random cards. One random number will correspond to a card value, and another random number will correspond to a card suit.

This was for the original code for the Deck of Cards (see *Stage 1*), however when I re-coded the class, I no longer needed a randomly generated number.

[Java Switch Statements](#) – W3Schools Website

I used this to remind myself how to use Switch Case in Java.

This was for the original code for the Deck of Cards (see *Stage 1*), however when I re-coded the class, I no longer needed a switch case in Deck of Cards.

[Java Cast Integer to String](#) – Stack Overflow Website

I used this to remind myself how to cast an integer into a string in Java.

This was for the original code for the Deck of Cards (see *Stage 1*), however when I re-coded the class, I no longer to cast an integer into a string.